

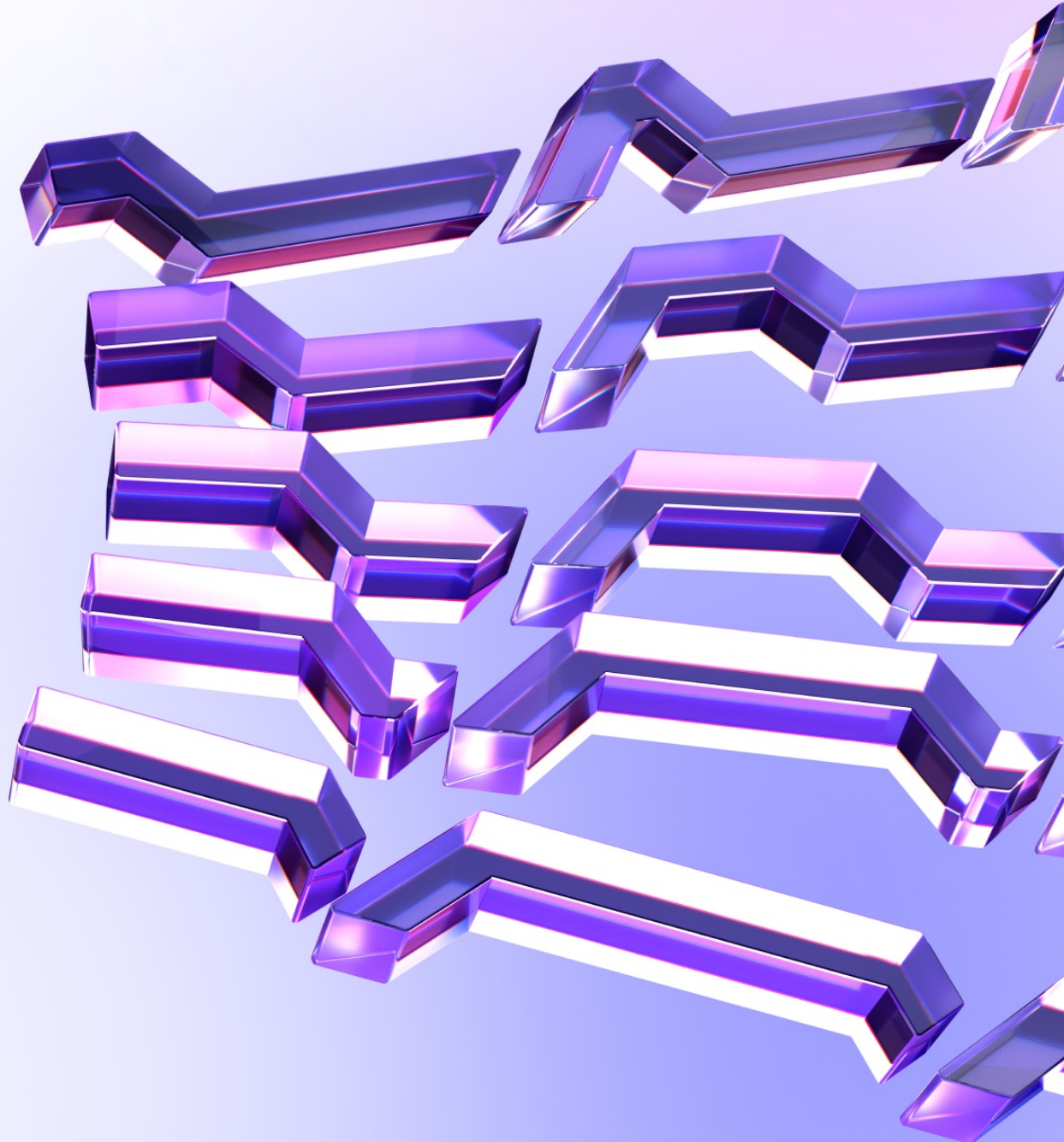


IPOR-Labs

ipor-fusion

December 12, 2025 15:42 UTC

Wake Arena AI Report



Contents

- 1. Document Revisions 3
- 2. Overview 4
 - 2.1. Ackee Blockchain Security 4
 - 2.2. Wake Arena 5
 - 2.3. Disclaimer 5
 - 2.4. Finding Classification 6
- 3. Executive Summary 7
 - Revision 1.0 7
- 4. Findings Summary 12
- Report Revision 1.0 21
 - Findings 21
- Appendix A: How to cite 222

1. Document Revisions

1.0	AI Scan	Dec 12, 2025 15:42 UTC
---------------------	---------	------------------------

2. Overview

This document presents the findings [Wake Arena](#) identified using AI vulnerability checks in the reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling [Wake](#) for Ethereum and [Trident](#) for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the [School of Solana](#) and the [Solana Auditors Bootcamp](#).

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.

Rohanske nabrezi 717/4

186 00 Prague, Czech Republic

<https://ackee.xyz>

hello@ackee.xyz

2.2. Wake Arena

This report was created by [Wake Arena](#), an automated vulnerability analysis tool. It uses [Wake](#) Framework with additional detectors for AI and static analysis of Solidity smart contracts.

To find potential vulnerabilities, Wake uses:

- The code structure and patterns
- Control flow graphs
- Data flow graphs
- Common vulnerability patterns
- Contract interactions

Wake Arena aims for precision, not recall. This means the presented findings are optimized for true positives, not for finding all possible issues, and should always be complemented with additional manual review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, but automated findings shouldn't be considered a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

This executive summary covers the IPOR-Labs/ipor-fusion repository at commit `1e25bb68c4f3b33fd0b6ce20f17f70e2d5b18b40`, focusing on the modular Plasma Vault architecture that delegates to protocol-specific fuses for Aerodrome, Velodrome Superchain (including Slipstream), Balancer, Curve, Gearbox, Lido, and auxiliary async executor flows. Across the codebase, 57 AI-detected issues were identified from the scoped contracts and libraries, spanning valuation logic, access control, and denial-of-service risk surfaces.

Detected issues by impact: critical 0, high 9, medium 18, low 14, warning 2, info 14. The most severe items cluster around authorization gaps in liquidity-management paths, fee and valuation misattribution when LPs are staked in gauges, and accounting mistakes that double-count or misprice positions.

Key high-impact findings include:

- Missing caller authorization on `exit` and `exitTransient` for Slipstream position management allows arbitrary liquidity decreases when the fuse is approved as operator, enabling griefing and forced unwinds without vault consent (`VelodromeSuperchainSlipstreamModifyPositionFuse.exit`, `exitTransient`).
- Gauge-staked LP fees are attributed to the vault address instead of the actual fee owner, inflating balances and distorting TVL/share price; the index delta is multiplied by gauge-held liquidity while indices are read for `address(this)` in `delegatecall` context (`VelodromeSuperchainBalanceFuse._calculateBalanceFromFees`, similar in `AerodromeBalanceFuse`).

- Composable Balancer pools count BPT as if it were an underlying token, causing value double counting and potential pricing reverts if the oracle lacks a BPT feed (`BalancerBalanceFuse.balanceOf`).
- Aerodrome Slipstream pool valuation iterates all Nonfungible Position Manager NFTs owned by the vault and fails to filter by pool, causing cross-pool double-counting and mispricing; with multiple substrates, the same NFTs are included repeatedly (`AerodromeSlipstreamBalanceFuse.balanceOf`).
- Unbounded iteration over externally addable NFTs or gauge-staked positions can push balance updates over gas limits and DoS vault flows that delegatecall fuse `balanceOf` (`VelodromeSuperchainSlipstreamBalanceFuse.balanceOf`, Aerodrome Slipstream variant).
- Misapplied target/selector allowlist in async exits validates ERC20 asset addresses against unrelated selectors, creating both a denial-of-service in normal exits and an ineffective policy check (`AsyncActionFuse.exit`).
- Direct use of `ERC20Upgradeable._burn` inside a fuse bypasses the vault's `_update` hook chain, so vote checkpoints and cap validations are not enforced during burns (`BurnRequestFeeFuse.enter`).

Overall, the security posture shows strong modularization and use of substrate allowlists and oracle-based USD conversions, but multiple identified issues have material operational and financial impact if deployed as-is. The recurring root causes are identity mismatches under `delegatecall` (using `address(this)` where the pool accounts a gauge), missing caller authorization on state-changing `enter/exit` entry points, and valuation pipelines that do not fully mirror protocol accounting (e.g., composable BPT, fee accrual with zero LP balance, cross-pool NFT filtering).

Key Technical Findings

- Authorization gaps on liquidity exits:

`VelodromeSuperchainSlipstreamModifyPositionFuse.exit` and `exitTransient` are callable by anyone; when the fuse is approved for a `tokenId`, any caller can trigger `NonfungiblePositionManager.decreaseLiquidity`, forcing partial/full unwinds without vault authorization. Protect both paths and bind transient inputs to the caller or remove the transient indirection.

- Fee misattribution for gauge-held positions:

`VelodromeSuperchainBalanceFuse._calculateBalanceFromFees` and `AerodromeBalanceFuse._calculateBalanceFromFees` read `supplyIndex* /claimable*` for `address(this)` while multiplying by gauge-held liquidity. Use the actual fee owner on the pool (gauge or `FeesVotingReward`) and prorate by the vault's share of `gauge.totalSupply()`; always include stored `claimable*` even when current LP balance is zero.

- Double counting in Balancer composable pools:

`BalancerBalanceFuse.balanceOf` sums pro-rata token amounts for all pool tokens, including the BPT itself. Skip entries where `address(token) == pool` and use the pool's actual/effective supply (e.g., composable stable's `getActualSupply()` or `totalSupply - poolBptBalance`).

- Cross-pool valuation errors in Slipstream:

`AerodromeSlipstreamBalanceFuse.balanceOf` enumerates all vault-owned NFPM NFTs and values them using the current substrate's `token0/token1/sqrtPriceX96`, causing cross-pool double-counting and mispricing. Filter by the current pool (e.g., `userPositions(owner, pool)` or `positions(tokenId)` checks) and ensure USD conversion uses each position's true token pair.

- Unbounded enumeration leads to DoS: Slipstream balance fuses loop over unbounded arrays (`userPositions`, `stakedValues`) and per-token heavy calls (principal/fees, USD conversions). Spamming NFTs to the vault increases loop cost and can revert `updateMarketsBalances` via gas exhaustion. Maintain curated per-pool position sets and cap per-call processing.

- Misapplied allowlist in async exits: `AsyncActionFuse.exit` calls `_validateTargetsMemory/assets, fetchCallDatas, allowedTargets`), treating ERC20 assets as call targets and enforcing arbitrary 4-byte selectors that are never executed, producing avoidable reverts and an illusory policy. Remove this check or validate what is actually executed (`AsyncExecutor.fetchAssets`).
- Governance/accounting bypass via internal burns: `BurnRequestFeeFuse.enter` calls `_burn` directly from a fuse that inherits `ERC20Upgradeable`, bypassing `PlasmaVaultBase._update` (vote checkpoints and cap checks). Route burns/mints through the vault's `_update` pipeline (e.g., `updateInternal`) instead of local ERC20 internals.
- ERC-4626 withdrawal sizing errors: withdrawal paths use `convertToShares` (deposit-direction, round down) instead of `previewWithdraw` (round up), causing shortfalls or no-ops for small amounts (`FluidInstadappStakingSupplyFuse.instantWithdraw`, `GearboxV3FarmSupplyFuse.instantWithdraw`). Switch to withdrawal previews and add guards when computed shares are zero while `amount > 0`.
- Permissionless `enter/exit` enable front-run griefing: Balancer gauge flows are public and enforce `minBptAmount`; an attacker can front-run to move BPT between wallet and gauge, making the orchestrator's `min` fail and reverting the transaction. Restrict callers or batch atomically through an authorized router.
- Oracle/asset support assumptions: valuation helpers call `PriceOracleMiddleware.getAssetPrice` on every token surfaced by substrates. For composable pools and wrappers, ensure unsupported assets (e.g., BPT) are skipped or have configured feeds to avoid preventable reverts during `balanceOf`.

Conclusion

The codebase exhibits a solid modular architecture and clear integration

boundaries; however, the number and nature of the identified issues—particularly nine high-impact findings affecting authorization, valuation correctness, and operational liveness—indicate that the project is not ready for deployment at this commit. Correctness fixes are required for gauge fee attribution, Slipstream NFT filtering and iteration bounds, Balancer composable-pool handling, async exit validation, and routing ERC20 state changes through the vault's update hooks, alongside tightening access control on all state-changing `enter/exit` entry points. After addressing these items and adding targeted tests, a follow-up review is recommended before mainnet use.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	9	18	14	2	14	57

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
H1: Fee indices misattributed for Gauge-staked LPs inflates USD balance	High	1.0	Reported
H2: Gauge-staked LP fees are attributed to the vault address instead of the actual fee owner, causing mispricing	High	1.0	Reported
H3: Direct ERC20Upgradeable._burn in fuse bypasses PlasmaVaultBase._update hooks (vote checkpoints and caps not enforced)	High	1.0	Reported
H4: Composable pool BPT counted as an underlying token, inflating USD value and risking DoS	High	1.0	Reported
H5: Missing access control on exitTransient enables arbitrary liquidity decrease via global transient storage	High	1.0	Reported

Finding title	Impact	Reported	Status
H6: exit misvalidates assets as target/selecter pairs, causing DoS and ineffective policy checks	High	1.0	Reported
H7: Missing access control on exit allows arbitrary liquidity decreases on vault-managed NFT positions	High	1.0	Reported
H8: Pool path counts all NFPM NFTs regardless of pool, causing cross-pool double-counting and mispricing	High	1.0	Reported
H9: Unbounded iteration over externally added Slipstream NFT positions enables DoS of vault balance updates	High	1.0	Reported
M1: instantWithdraw uses deposit-oriented convertToShares for withdrawals, causing rounding-down shortfalls	Medium	1.0	Reported
M2: Missing access control on enter allows anyone to stake contract-held BPT into granted gauges	Medium	1.0	Reported
M3: Unbounded ERC-721 enumeration lets anyone spam the vault with NFTs and DoS balanceOf	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M4: Deterministic pool address prediction tied to current factory implementation (DoS after upgrades)	Medium	1.0	Reported
M5: Unrestricted exitTransient enables forced withdrawals via transient inputs (no caller authorization)	Medium	1.0	Reported
M6: Missing access control on <code>exit</code> allows anyone to unstake the contract's gauge shares at arbitrary times	Medium	1.0	Reported
M7: Incorrect use of ERC4626 <code>convertToShares</code> in <code>instantWithdraw</code> causes under-withdrawal and brittle downstream flows	Medium	1.0	Reported
M8: Missing per-asset whitelist enforcement in <code>enter</code> enables spending unapproved tokens via <code>Permit2</code>	Medium	1.0	Reported
M9: Unrestricted exit allows anyone to force-withdraw vault-staked NFTs (griefing/DoS)	Medium	1.0	Reported
M10: Permissionless <code>enter/exit</code> enable front-run griefing via <code>minBptAmount</code> reverts and sequencing disruption	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M11: Misleading exit: amounts are accounted (not transferred); collect is omitted	Medium	1.0	Reported
M12: instantWithdraw calls unvalidated stakingPool and token before whitelist enforcement	Medium	1.0	Reported
M13: Fee accrual is skipped when LP balance is zero, undercounting assets and enabling share price manipulation	Medium	1.0	Reported
M14: Unbounded iteration over substrates and positions risks gas exhaustion in <code>balanceOf</code>	Medium	1.0	Reported
M15: <code>getPoolAddress</code> predicts pool using current factory implementation, causing mismatches and DoS after factory upgrades	Medium	1.0	Reported
M16: Deterministic pool address prediction tied to factory's mutable implementation breaks across upgrades, causing allowlist mismatches and denial-of-service	Medium	1.0	Reported
M17: <code>exit</code> allows withdrawing non-whitelisted pool tokens back to the vault, bypassing asset policy	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M18 : Unbounded iteration in balanceOf across substrates and positions can exhaust gas during market balance updates	Medium	1.0	Reported
L1 : Payable enter does not forward or refund ETH, causing stuck funds	Low	1.0	Reported
L2 : Unchecked uint256→uint128 cast in exitTransient can silently truncate liquidity	Low	1.0	Reported
L3 : Claimable fees ignored when current LP balance is zero	Low	1.0	Reported
L4 : previewWithdraw divides by fee rate instead of net factor, causing severe overestimation of required shares	Low	1.0	Reported
L5 : BalancerGaugeFuseEnter emits the requested amount, not the actual deposited amount, producing misleading telemetry	Low	1.0	Reported
L6 : Per-token ERC721 approvals to the gauge are not revoked after withdrawal (least-privilege violation)	Low	1.0	Reported
L7 : Payable exit does not forward or refund ETH, causing stuck funds	Low	1.0	Reported

Finding title	Impact	Reported	Status
L8: instantWithdraw mis-sizes ERC-4626 withdrawals by using convertToShares (deposit path) instead of previewWithdraw/previewRedeem	Low	1.0	Reported
L9: Payable enterTransient does not forward or refund ETH, causing stuck funds	Low	1.0	Reported
L10: Unrestricted enter allows unauthorized approvals and staking for vault-owned NFTs (griefing)	Low	1.0	Reported
L11: Payable exitTransient does not forward or refund ETH, causing stuck funds	Low	1.0	Reported
L12: Inconsistent ETH handling: payable enter ignores msg.value; exit forwards unintended ETH with wethIsEth=false	Low	1.0	Reported
L13: Payable entry points can lock ETH because <code>msg.value</code> is unused and there is no recovery path	Low	1.0	Reported
L14: Mint called with unsorted token pair after sorted pool check causes avoidable reverts	Low	1.0	Reported

Finding title	Impact	Reported	Status
W1: Named return variable <code>finalAmount</code> is shadowed in exit, risking unintended returns after refactors	Warning	1.0	Reported
W2: Named return variable <code>finalAmount</code> is shadowed in enter, risking unintended returns after refactors	Warning	1.0	Reported
I1: Asset allowlist not enforced; error AreodromeSlipstreamModifyPositionFuseUnsupportedAsset is unused	Info	1.0	Reported
I2: Dead code: custom error <code>UnsupportedMethod</code> is declared but never used	Info	1.0	Reported
I3: Dead private helper <code>bytes32ToSubstrate</code> is unused, increasing bytecode size and maintenance risk	Info	1.0	Reported
I4: Dead code: IERC20/SafeERC20 imported and using directive declared but never used	Info	1.0	Reported
I5: Unused immutable MARKET_ID may mislead integrators about market scoping	Info	1.0	Reported

Finding title	Impact	Reported	Status
I6: Dead private helper function <code>substratesToBytes32</code> increases code surface without benefit	Info	1.0	Reported
I7: Unused <code>SafeCast</code> import and using directive increases bytecode and can mislead reviewers	Info	1.0	Reported
I8: <code>instantWithdraw</code> parameter docs mismatch: <code>params[1]</code> must be gauge address (not ERC4626 vault)	Info	1.0	Reported
I9: <code>enterTransient</code> documentation claims outputs are written, but implementation omits <code>setOutputs</code> (transient I/O mismatch)	Info	1.0	Reported
I10: Inaccurate layout description in <code>decodeAllowedTargets</code> (address not left-aligned; selector in least-significant 4 bytes)	Info	1.0	Reported
I11: Unused immutable <code>BALANCER_ROUTER</code> (dead code) increases maintenance and audit risk	Info	1.0	Reported
I12: <code>NatSpec</code> access annotation mismatch: <code>updateMarketsBalances</code> is restricted but documented as public/no restrictions	Info	1.0	Reported

Finding title	Impact	Reported	Status
M13 : Exit data struct mislabels unit/direction (says "to deposit" instead of "to withdraw")	Info	1.0	Reported
M14 : Inaccurate layout description in encodeAllowedTargets (address not left-aligned; unused 7 bytes are most-significant)	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision.
For the complete list of all findings, [Go back to Findings Summary](#)

H1: Fee indices misattributed for Gauge-staked LPs inflates USD balance

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/fuses/aerodrome/AerodromeBalanceFuse.sol	Type:	Logic error

Description

The fee accounting logic in function `pass: [_]calculateBalanceFromFees` reads `supplyIndex0`, `supplyIndex1`, and `claimable0/1` from the pool using the vault address (`address(this)`) as the account, and then applies the index deltas to the current `liquidity` passed from `balanceOf`.

When the substrate is a `Gauge`, the LP tokens are staked in the Gauge contract. Aerodrome pools track fee indices and claimables for the account that holds LP tokens in the pool. In the Gauge case, that account is the Gauge contract, not the vault. Reading indices and claimables for the vault while multiplying by the Gauge-staked `liquidity` misattributes historical fee growth to the vault and overstates the computed fees in USD.

The loop in `balanceOf` treats gauges by taking the pool from the Gauge and the `liquidity` as the Gauge’s LP balance, but it does not adjust the account used in fee index reads:

Listing 1. Code snippet from `contracts/fuses/aerodrome/AerodromeBalanceFuse.sol`

```
90         if (substrate.substrateType == AerodromeSubstrateType.Gauge) {
91             pool = IGauge(substrate.substrateAddress).stakingToken();
92             liquidity = IERC20(substrate.substrateAddress).balanceOf(
93                 address(this)
```

```

94         );
95         } else if (substrate.substrateType ==
    AerodromeSubstrateType.Pool) {
96             pool = substrate.substrateAddress;
97             liquidity = IERC20(pool).balanceOf(address(this));

```

The fee calculation then reads the pool's indices and claimables for the vault and adds index-delta fees proportional to **liquidity**, which in the Gauge case represents LPs held by the Gauge, not the vault:

*Listing 2. Code snippet from
contracts/fuses/aerodrome/AerodromeBalanceFuse.sol*

```

211     address plasmaVault = address(this);
212     uint256 supplyIndex0 = IPool(pool_).supplyIndex0(plasmaVault);
213     uint256 supplyIndex1 = IPool(pool_).supplyIndex1(plasmaVault);
214     uint256 index0 = IPool(pool_).index0();
215     uint256 index1 = IPool(pool_).index1();
216
217     // Calculate index deltas: difference between current global index
    and vault's last checkpoint
218     uint256 delta0 = index0 - supplyIndex0;
219     uint256 delta1 = index1 - supplyIndex1;
220
221     // Get already claimable fees that haven't been claimed yet
222     uint256 claimable0 = IPool(pool_).claimable0(plasmaVault);
223     uint256 claimable1 = IPool(pool_).claimable1(plasmaVault);
224
225     // Add fees accumulated since last checkpoint using index deltas
226     // delta represents fee growth per unit of liquidity, multiplied by
    liquidity to get total fees
227     if (delta0 > 0) {
228         claimable0 += (liquidity_ * delta0) / 1e18;
229     }
230
231     if (delta1 > 0) {
232         claimable1 += (liquidity_ * delta1) / 1e18;
233     }

```

Because **supplyIndexX** and **claimableX** for the vault typically remain at zero when LPs are staked in the Gauge, **deltaX** approximates the full global index value. Multiplying **deltaX** by the current Gauge-staked **liquidity** attributes

the entire historical index growth to the vault, inflating `claimable0/1` and the reported USD balance.

Impact in production:

- Overstated market value returned by `balanceOf` for any configuration that uses Gauge substrates.
- Potential share price mispricing, cap or limit bypassing, or incorrect risk accounting when this fuse feeds vault accounting or risk limits.
- No mitigation is present: the same logic path is used for both Pool and Gauge substrates without switching the account whose indices are read.

Exploit Scenario

A misattribution of fee indices allows overstating the vault's USD balance whenever LP tokens are staked in a Gauge.

Consider the following attack scenario:

1. Alice stakes LP tokens into a configured Gauge substrate so that the Gauge contract holds the pool LPs, while the vault holds none in the pool directly.
2. The pool accrues fee indices and claimables under the Gauge address; the vault's `supplyIndex0/1` and `claimable0/1` in the pool remain near zero.
3. Alice triggers a read path that relies on function `balanceOf` (for example, a rebalance gate, limit check, or NAV computation).
4. Function `pass:[_]calculateBalanceFromFees` reads indices and claimables for the vault and multiplies the Gauge-staked `liquidity` by the near-global index delta, greatly inflating `claimable0/1`.
5. The vault now reports an artificially high USD balance; Bob's protocol, which trusts this balance for limits, pricing, or solvency checks, makes decisions on bad data and can suffer financial loss.

Recommendation

Handle Gauge substrates by reading fee indices and claimables for the correct account (the Gauge), not the vault, when LP tokens are staked in the Gauge.

Recommended approach:

- In `balanceOf`, when `substrateType == AerodromeSubstrateType.Gauge`, set `account = substrate.substrateAddress` (the Gauge) and keep `pool = IGauge(account).stakingToken()` and `liquidity = IERC20(account).balanceOf(address(this))`.
- Modify function `pass:[_]calculateBalanceFromFees` to accept an `account` parameter and use it instead of `address(this)` when reading `supplyIndex0/1` and `claimable0/1` from the pool.
- Only multiply index deltas by `liquidity` that corresponds to the same `account` whose indices are read, ensuring identities match.
- Optionally, for Aerodrome deployments that route fee accruals to `FeesVotingReward`, query the Gauge's linked reward/fees contract if pool-level `claimable0/1` does not include Gauge accruals.

This change preserves correct attribution and prevents inflation of `claimable0/1` when liquidity is staked in the Gauge.

[Go back to Findings Summary](#)

H2: Gauge-staked LP fees are attributed to the vault address instead of the actual fee owner, causing mispricing

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol	Type:	Logic error

Description

The function `balanceOf` values staked gauge positions by multiplying the vault’s gauge balance by the Pool-wide fee index deltas that are looked up for `address(this)`.

When LP tokens are staked in a Velodrome v2 **Gauge**, the Pool accrues per-account fees to the address that holds LP tokens at the Pool (typically the **Gauge** or its linked **FeesVotingReward**), not to the original depositor’s address. Using `address(this)` here attributes pool fees to the Plasma Vault even though the Pool does not track those fees for the vault while LPs are staked in the gauge. This leads to systematic mispricing.

Listing 3. Code snippet from
contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol

```
146 address plasmaVault = address(this);
147 uint256 supplyIndex0 = IPool(pool_).supplyIndex0(plasmaVault);
148 uint256 supplyIndex1 = IPool(pool_).supplyIndex1(plasmaVault);
149 uint256 index0 = IPool(pool_).index0();
150 uint256 index1 = IPool(pool_).index1();
151
152 uint256 delta0 = index0 - supplyIndex0;
153 uint256 delta1 = index1 - supplyIndex1;
154
```

```
155 uint256 claimable0 = IPool(pool_).claimable0(plasmaVault);
156 uint256 claimable1 = IPool(pool_).claimable1(plasmaVault);
```

In the `Gauge` branch of `balanceOf`, the `liquidity` used for fee computation is the vault's stake in the gauge, not the pool LP balance. This is then combined with the Pool indices looked up for `address(this)`, creating a mismatch between the fee accrual owner and the liquidity amount.

Listing 4. Code snippet from

contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol

```
72 if (
73     substrate.substrateType ==
74     VelodromeSuperchainSubstrateType.Gauge
75 ) {
76     pool = ILeafGauge(substrate.substrateAddress).stakingToken();
77     liquidity = IERC20(substrate.substrateAddress).balanceOf(
78         address(this)
79     );
80 }
```

Technical effect:

- Fees are added via `(liquidity * (index - supplyIndex[address(this)]))` even though the Pool accrues those fees to the gauge-side account, not the vault address, when LPs are staked.
- The function never considers gauge-side fee caches such as `ILeafGauge.fees0()` and `ILeafGauge.fees1()` or prorates the gauge's total fees by the vault's share of `gauge.totalSupply()`.

As a result, the returned USD balance for gauge-held positions can be materially over-stated. Since `IMarketBalanceFuse.balanceOf` is used by the Plasma Vault engine to update market valuations, this mispricing propagates into vault TVL/share-price calculations and can produce unfair mint/burn outcomes.

Exploit Scenario

A practical exploitation leverages periods when the vault has LP tokens staked in a `Gauge` while trading activity increases Pool fee indices.

Consider the following attack scenario:

1. Alice observes that the vault stakes LP tokens in a `Gauge` and that `balanceOf` attributes pool fee accrual to `address(this)`.
2. Alice waits for a period of high trading where `index0/index1` increase materially while the vault remains staked.
3. The fuse overvalues the position by adding pool fees to the vault that are actually accruing to the gauge-side fee owner.
4. Alice redeems her shares during this overvaluation window to withdraw more underlying than her fair entitlement; and
5. Bob, the remaining shareholder, is left with the deficit when the accounting later normalizes.

Recommendation

Compute fee value against the address that actually accrues Pool fees for the staked position and apportion it to the vault's share of the gauge.

Suggested approach for gauge-held positions:

- Determine the fee accrual owner on the Pool for the staked LPs, e.g., `address feeOwner = ILeafGauge(gauge).feesVotingReward()` (Velodrome v2 exposes this) or the `gauge` address itself depending on deployment specifics.
- Compute the gauge's total pool-level fee claimables using `IPool(pool).claimable0(feeOwner)` and `IPool(pool).claimable1(feeOwner)` and, if desired, include the lazy accrual increment using the fee owner's indices: `(IERC20(pool).balanceOf(feeOwner) * (index -`

`supplyIndex(feeOwner)))/1e18.`

- Prorate these amounts by the vault's stake:

`ILeafGauge(gauge).balanceOf(vault) / ILeafGauge(gauge).totalSupply().`

- Alternatively, use `ILeafGauge(gauge).fees0()` and

`ILeafGauge(gauge).fees1()` as the gauge-side fee caches and apportion them by the same pro-rata share.

For direct pool-held positions (non-gauge), keep the existing logic that keys per-account indices to the vault address.

Add regression tests that cover both pool-held and gauge-held positions across fee accrual events to prevent reintroduction of this mismatch.

[Go back to Findings Summary](#)

H3: Direct ERC20Upgradeable._burn in fuse bypasses PlasmaVaultBase._update hooks (vote checkpoints and caps not enforced)

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/burn_request_fee/BurnRequestFee Fuse.sol	Type:	Logic error

Description

The `enter` function invokes `ERC20Upgradeable._burn(withdrawManager, amount)` directly from within the fuse. Because the fuse inherits its own `ERC20Upgradeable` implementation and runs via `delegatecall` into `PlasmaVault`, the internal `_update` reached by `_burn` is the one from the fuse’s inheritance chain, not the vault’s overridden `_update` in `PlasmaVaultBase`.

As a result, the vault’s intended hooks are skipped during burns performed by this fuse:

- Voting checkpoints are not updated because `PlasmaVaultBase._update` is not executed, which means `_transferVotingUnits` is never called.
- Supply-cap validations that live in `PlasmaVaultBase._update` are not enforced on mint operations and the voting accounting invariants are not preserved for burns.

Under `delegatecall`, internal calls resolve against the callee’s own inheritance graph. Calling `_burn` from the fuse therefore bypasses `PlasmaVault`’s `_update` pipeline and causes governance state to drift from actual balances, breaking assumptions made by vote queries like `getVotes/getPastVotes` and quorum checks using `getPastTotalSupply`.

Listing 5. Code snippet from

contracts/fuses/burn_request_fee/BurnRequestFeeFuse.sol

```
107 function enter(BurnRequestFeeDataEnter memory data_) public {
108     address withdrawManager = PlasmaVaultStorageLib
109         .getWithdrawManager()
110         .manager;
111
112     if (withdrawManager == address(0)) {
113         revert BurnRequestFeeWithdrawManagerNotSet();
114     }
115
116     if (data_.amount == 0) {
117         return;
118     }
119
120     _burn(withdrawManager, data_.amount);
121
122     emit BurnRequestFeeEnter(VERSION, data_.amount);
123 }
```

The expected vault hook pipeline routes `_update` into `PlasmaVaultBase`, where vote accounting is performed:

Listing 6. Code snippet from contracts/vaults/PlasmaVaultBase.sol

```
253 function _update(
254     address from_,
255     address to_,
256     uint256 value_
257 ) internal virtual override {
258     super._update(from_, to_, value_);
259
260     /// @dev total supply cap validation is disabled when performance and
261     /// management fee is minted
262     if (PlasmaVaultLib.isTotalSupplyCapValidationEnabled()) {
263         /// @dev check total supply cap
264         if (from_ == address(0)) {
265             uint256 maxSupply = cap();
266             uint256 supply = totalSupply();
267             if (supply > maxSupply) {
268                 revert ERC20ExceededCap(supply, maxSupply);
269             }
270         }
271     }
272 }
```

```
269     }  
270 }  
271  
272     _transferVotingUnits(from_, to_, value_);  
273 }
```

Exploit Scenario

The bypass leads to stale governance accounting and broken invariants whenever this fuse burns shares.

Consider the following attack scenario:

1. Alice obtains operational access to trigger fuse actions on the vault and has the `WithdrawManager` delegate its voting power to her address beforehand;
2. Alice calls `enter` on the fuse to burn a large amount of `WithdrawManager`'s shares; and
3. Because the burn uses `ERC20Upgradeable._burn` from the fuse, the vault's `PlasmaVaultBase._update` is not executed, so `_transferVotingUnits` is never called.

Consequences in production:

- Alice retains voting power for shares that were already destroyed, allowing her to influence proposals and quorum that rely on `getVotes` / `getPastVotes` / `getPastTotalSupply`.
- Off-chain indexers and on-chain modules that assume voting units mirror balances will produce inconsistent results, leading to governance decisions based on outdated data.

Recommendation

Ensure all token state changes performed by fuses go through the vault's `_update` pipeline so that vote checkpoints and cap checks execute.

Fix the vulnerability by either:

- Replacing the direct `_burn` with a delegatecall to the base that executes `updateInternal(from, to, value)` using the vault's path, for example calling `PLASMA_VAULT_BASE().functionDelegateCall(abi.encodeWithSelector(IPlasmaVaultBase.updateInternal.selector, withdrawManager, address(0), data_.amount));` or
- Exposing a dedicated external function on `PlasmaVault` (e.g., `burnFrom(address from, uint256 amount)`) that internally routes to `_update` and invoking that from the fuse, instead of using the fuse's own `ERC20Upgradeable._burn`.

Additional hardening:

- Remove `ERC20Upgradeable` inheritance from fuse contracts to prevent accidental use of their local ERC20 internals in a `delegatecall` context;
- Add invariant tests verifying that vote checkpoints and `getPastTotalSupply` stay in sync with balances and total supply after fuse-driven burns and mints.

[Go back to Findings Summary](#)

H4: Composable pool BPT counted as an underlying token, inflating USD value and risking DoS

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/fuses/balancer/BalancerBalanceFuse.sol	Type:	Logic error

Description

The contract sums the USD value of all tokens returned by `IPool(pool).getTokenInfo()` after first computing proportional amounts for an LP share. In composable Balancer pools (for example, Composable Stable), the BPT itself is registered as a pool token and appears in the `tokens` array. Because the loop does not skip the BPT entry, the code adds the USD value of the BPT in addition to the underlying tokens it already represents. This double counts the pool's value.

Affected elements:

- Function `balanceOf` (aggregation of USD value)
- Library function `BalancerSubstrateLib.computeProportionalAmountsOut` (per-token pro-rata amounts)
- Call to `IPriceOracleMiddleware.getAssetPrice` via `_convertToUsd`

The relevant loop iterates over all pool tokens without excluding the BPT (`address(tokens[j]) == pool`), so the BPT is treated as a normal underlying token and priced again.

*Listing 7. Code snippet from
contracts/fuses/balancer/BalancerBalanceFuse.sol*

```
135      (
136          IERC20[] memory tokens,
137          ,
138          ,
139          uint256[] memory lastBalancesLiveScaled18
140      ) = IPool(pool).getTokenInfo();
141
142      uint256 totalSupply = IERC20(pool).totalSupply();
143      uint256[] memory amountsOut = BalancerSubstrateLib
144          .computeProportionalAmountsOut(
145          lastBalancesLiveScaled18,
146          totalSupply,
147          lpBalance
148      );
149
150      uint256 tokensLen = tokens.length;
151      for (uint256 j; j < tokensLen; j++) {
152          balance += _convertToUsd(
153              amountsOut[j],
154              address(tokens[j]),
155              priceOracleMiddleware
156          );
157      }
```

The per-token pro-rata computation multiplies each pool balance by the LP share. It does not special-case BPT.

*Listing 8. Code snippet from
contracts/fuses/balancer/BalancerSubstrateLib.sol*

```
71      uint256 len = balances.length;
72      // Create a new array to hold the amounts of each token to be
73      withdrawn.
74      amountsOut = new uint256[](len);
75      for (uint256 i; i < len; ++i) {
76          // Since we multiply and divide we don't need to use FP math.
77          // Round down since we're calculating amounts out.
78          amountsOut[i] = (balances[i] * bptAmountIn) / bptTotalSupply;
79      }
```

Real-world effects:

- Inflated USD valuation for composable pools because the BPT component represents the same underlying already counted from `lastBalancesLiveScaled18`.
- If the price oracle lacks a BPT price source, `_convertToUsd` may revert with `UnsupportedAsset`, turning `balanceOf` into a denial-of-service for any market that holds such a pool. This follows from `IPriceOracleMiddleware.getAssetPrice`, which is allowed to revert for unsupported assets.

Exploit Scenario

An attacker can profit from the inflated accounting or cause availability issues when this fuse's output is consumed by higher-level logic (limits, thresholds, health checks).

Consider the following attack scenario:

1. Alice selects or influences a market that includes a Balancer composable pool as a substrate;
2. Alice deposits BPT for that pool (directly or via a gauge) so the vault holds LP exposure;
3. Alice ensures the price oracle has a price source for the BPT token; and
4. When `balanceOf` is called, the contract computes pro-rata amounts for every entry in `tokens`, including the BPT, and adds the BPT's USD value on top of the underlying tokens, inflating the reported total.

A failure-mode variant that causes a revert:

1. Alice selects a composable pool whose BPT has no configured price feed;
2. Alice (or any user) triggers a path that calls `balanceOf` (e.g., a periodic check or a view exposed via middleware);

3. `_convertToUsd` attempts to price the BPT and the oracle reverts with `UnsupportedAsset`; and
4. Bob's protocol cannot obtain the total balance, effectively causing a denial-of-service for features that depend on this number.

Recommendation

Harden the accounting so the BPT is not double counted and can't cause pricing-related reverts:

- Skip BPT when summing token values: if `address(tokens[j]) == pool`, continue without pricing that entry.
- For composable pools, use the pool's actual/effective supply when computing proportional amounts, not the raw ERC20 `totalSupply`. In Balancer composable pools the pool holds its own BPT, so the correct denominator is the "actual supply" (e.g., `getActualSupply()` on composable stable pools) or `totalSupply - poolBptBalance`.
- Ensure the price oracle is configured for any token you intend to price. If the BPT is intentionally skipped, it does not need a price source, which removes the DoS vector.

Implementation sketch:

- Add a guard in the token loop to ignore `tokens[j]` whose address equals `pool`.
- For pools known to be composable, either: (a) query the pool's actual BPT supply via a specific interface; or (b) derive an effective supply by subtracting the pool's BPT balance from `totalSupply`.

[Go back to Findings Summary](#)

H5: Missing access control on exitTransient enables arbitrary liquidity decrease via global transient storage

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamModifyPositionFuse.sol	Type:	Access control

Description

The `exitTransient` entry point is externally callable and reads all parameters from transient storage using `TransientStorageLib.getInputs(VERSION)`, where `VERSION` is the fuse contract address. The function performs no caller authorization and forwards the parsed values to `exit`.

Because `TransientStorageLib` writes to EVM transient storage keyed only by the address passed in (here, `VERSION`) and not by the caller, any contract can set inputs for this fuse within the same transaction by calling `TransientStorageLib.setInputs(VERSION, ...)` (or equivalently using `tstore` directly) and then invoke `exitTransient`. If the fuse is approved as operator for the relevant `tokenId`, this results in an unauthorized `decreaseLiquidity` execution, identical to calling `exit` directly.

This design couples critical execution to a globally-shared, caller-agnostic transient storage namespace, which is unsafe for authorization.

Listing 9. Code snippet from

`contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamModifyPositionFuse.sol`

```
368     function exitTransient() external {
369         bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
370
371         VelodromeSuperchainSlipstreamModifyPositionFuseExitData
372             memory data_ =
373             VelodromeSuperchainSlipstreamModifyPositionFuseExitData({
374                 tokenId: TypeConversionLib.toUint256(inputs[0]),
375                 liquidity: uint128(TypeConversionLib.toUint256(inputs[1])),
376                 amount0Min: TypeConversionLib.toUint256(inputs[2]),
377                 amount1Min: TypeConversionLib.toUint256(inputs[3]),
378                 deadline: TypeConversionLib.toUint256(inputs[4])
379             });
380
381         VelodromeSuperchainSlipstreamModifyPositionFuseExitResult
382             memory result = exit(data_);
383
384         bytes32[] memory outputs = new bytes32[](3);
385         outputs[0] = TypeConversionLib.toBytes32(result.tokenId);
386         outputs[1] = TypeConversionLib.toBytes32(result.amount0);
387         outputs[2] = TypeConversionLib.toBytes32(result.amount1);
388         TransientStorageLib.setOutputs(VERSION, outputs);
389     }
```

Exploit Scenario

An attacker can programmatically feed inputs to the fuse and execute a liquidity decrease in a single transaction.

Consider the following attack scenario:

1. Alice deploys a helper contract that calls `TransientStorageLib.setInputs(VERSION, [tokenId, liquidity, 0, 0, deadline])` where `VERSION` is this fuse's address;
2. In the same transaction, Alice's helper contract calls `exitTransient()` on the fuse;

3. The fuse reads the attacker-supplied inputs from transient storage and calls `exit`, which in turn executes `NonfungiblePositionManager.decreaseLiquidity` as the approved operator; and
4. Bob's vault position has its liquidity reduced without Bob's consent, degrading strategy performance and potentially enabling repeated griefing.

Recommendation

Protect the transient path with the same authorization as the non-transient path and remove global, caller-agnostic storage coupling.

Recommended options:

- Add access control to `exitTransient`, e.g., `require(msg.sender == plasmaVault || AccessManager.isAuthorized(msg.sender, selector));`
- Bind transient inputs to the caller by keying them on `msg.sender` instead of `VERSION: bytes32[] memory inputs = TransientStorageLib.getInputs(msg.sender);` so one caller cannot inject inputs for another; and
- Prefer passing parameters directly to functions instead of using shared transient storage for critical state-changing operations.

[Go back to Findings Summary](#)

H6: exit misvalidates assets as target/selector pairs, causing DoS and ineffective policy checks

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/async_action/AsyncActionFuse.sol	Type:	Logic error

Description

The `exit` flow validates ERC20 asset addresses against a target/selector allowlist that was designed for contract calls.

In `exit`, the code calls `_validateTargetsMemory(data_.assets, data_.fetchCallDatas, allowedTargets)`. The validator expects a list of call targets and their ABI-encoded call data; it extracts the first 4 bytes from each `callDatas[i]` as a selector and checks that the pair `(targets[i], selector)` exists in `allowedTargets`.

However, `exit` never performs any external calls based on `fetchCallDatas`. The only external interaction is a single call to `AsyncExecutor.fetchAssets(assets, priceOracle, slippage)`. As a result, the validator is applied to the wrong domain: it interprets token addresses from `data_.assets` as call targets and enforces arbitrary selectors taken from `data_.fetchCallDatas` that are never executed.

This has two security consequences:

- Denial of service: unless governance configures `AllowedTargets` entries that coincidentally pair each asset address with whatever 4-byte selector a caller provides, `_validateTargetsMemory` will revert with `AsyncActionFuseTargetNotAllowed`. Legitimate `exit` calls can be blocked,

leaving assets stranded in the async executor until configuration or code is changed.

- Ineffective policy check: even if a (`asset`, `selector`) pair appears in `AllowedTargets` (for any reason), the selector is not executed by `exit`, so the check does not constrain real behavior. Attackers can supply any 4-byte value in `fetchCallDatas` to satisfy the check, undermining the intent of the allowlist in this code path.

Affected surface:

- Function `exit` in `AsyncActionFuse`;
- Function `exitTransient` in `AsyncActionFuse` (delegates to `exit`);
- Private validator function `_validateTargetsMemory` (applied to assets instead of call targets);
- Substrate `AllowedTargets` (misapplied in the `exit` path).

*Listing 10. Code snippet from
contracts/fuses/async_action/AsyncActionFuse.sol*

```
365     _validateTargetsMemory(  
366         data_.assets,  
367         data_.fetchCallDatas,  
368         allowedTargets  
369     );  
370  
371     // Call fetchAssets on executor  
372     AsyncExecutor(executor).fetchAssets(  
373         data_.assets,  
374         priceOracle,  
375         allowedSlippage.slippage  
376     );
```

The validator `_validateTargetsMemory` enforces target/selector pairs by extracting the selector from the first 4 bytes of each `callData` and matching it against `allowedTargets`.

*Listing 11. Code snippet from
contracts/fuses/async_action/AsyncActionFuse.sol*

```
499     function _validateTargetsMemory(
500         address[] memory targets_,
501         bytes[] memory callDatas_,
502         AllowedTargets[] memory allowedTargets_
503     ) private pure {
504         uint256 targetsLength = targets_.length;
505         uint256 allowedTargetsLength = allowedTargets_.length;
506         bytes memory callData;
507
508         for (uint256 i; i < targetsLength; ++i) {
509             callData = callDatas_[i];
510             if (callData.length < 4) {
511                 revert AsyncActionFuseCallDataTooShort(i);
512             }
513
514             // Extract function selector from first 4 bytes of callData
515             bytes4 selector;
516             assembly {
517                 selector := mload(add(callData, 0x20))
518             }
519             selector = bytes4(selector);
520             address target = targets_[i];
521             bool allowed;
522
523             AllowedTargets memory allowedTarget;
524
525             // Check if target/selector pair exists in the allowed list
526             for (uint256 j; j < allowedTargetsLength; ++j) {
527                 allowedTarget = allowedTargets_[j];
528                 if (
529                     allowedTarget.target == target &&
530                     allowedTarget.selector == selector
531                 ) {
532                     allowed = true;
533                     break;
534                 }
535             }
536
537             if (!allowed) {
538                 revert AsyncActionFuseTargetNotAllowed(target, selector);
539             }
540         }
```

```
541     }
```

By contrast, `exit` invokes a single function on the executor that ignores `fetchCallDatas` entirely and operates only on the provided `assets` array, confirming that target-selector validation does not govern any actual external call in this path.

*Listing 12. Code snippet from
contracts/fuses/async_action/AsyncExecutor.sol*

```
159     function fetchAssets(  
160         address[] calldata assets_,  
161         address priceOracle_,  
162         uint256 slippage_  
163     ) external onlyPlasmaVault {  
164         if (priceOracle_ == address(0)) {  
165             revert AsyncExecutorInvalidPriceOracleAddress();  
166         }  
167  
168         if (slippage_ > WAD) {  
169             revert AsyncExecutorInvalidSlippage();  
170         }  
171  
172         uint256 actualBalance = balance;  
173         uint256 assetsLength = assets_.length;  
174         uint256 totalBalanceUsd;  
175  
176         for (uint256 i; i < assetsLength; ++i) {  
177             totalBalanceUsd += _calculateAssetUsdValue(  
178                 assets_[i],  
179                 priceOracle_  
180             );  
181         }
```

Exploit Scenario

A misapplied validator causes `exit` to either revert unexpectedly or perform an ineffective check that does not constrain behavior.

Consider the following attack scenarios:

1. Alice performs an async strategy that leaves tokens in the executor. Later, Bob, a legitimate operator, calls `exit` to fetch assets `[USDC, WETH]`.
2. The market configuration contains `AllowedTargets` entries for swap targets (e.g., AMMs) but not for pairs `(USDC, <anySelector>)` or `(WETH, <anySelector>)`.
3. The validator `_validateTargetsMemory(data_.assets, data_.fetchCallDatas, allowedTargets)` interprets the asset addresses as call targets and tries to match them against `AllowedTargets`.
4. The check fails and `exit` reverts with `AsyncActionFuseTargetNotAllowed`, leaving assets stranded in the executor until governance changes configuration or the code is fixed.

A second scenario demonstrates the policy check being ineffective:

1. Alice crafts `fetchCallDatas` with arbitrary 4-byte values that match selectors paired with the given asset addresses in `AllowedTargets` (if such pairs exist due to misconfiguration).
2. The validator passes because the `(asset, selector)` pairs are present; however, `exit` does not execute these selectors at all.
3. The only external call is `AsyncExecutor.fetchAssets/assets, priceOracle, slippage)`, which ignores `fetchCallDatas`.
4. The allowlist check gives a false sense of security; it does not restrict what `exit` actually does.

Recommendation

Refactor `exit` to validate what it actually executes, or remove the misapplied validator entirely.

Recommended approaches:

- If `exit` should only fetch token balances: remove `_validateTargetsMemory`

from `exit` and validate the `assets` list against an explicit asset allowlist substrate (for example, `AllowedExitAssets`) or reuse `AllowedAmountToOutside` assets as the allowlist. Reject unknown assets early and drop the unused `fetchCallDatas` parameter from the API to prevent confusion.

- If `exit` is intended to execute calls: extend `exit` to accept proper `targets` and `callDatas` arrays, enforce equal lengths, validate with `_validateTargetsMemory(targets, callDatas, allowedTargets)`, and then perform those calls. Do not mix ERC20 asset addresses with call target validation.

Additional hardening:

- Add explicit array length checks in `exit` to ensure `data_.fetchCallDatas.length == data_.assets.length` if any relationship is kept for auditing purposes.
- Unit test `exit` for both happy-path fetches and misconfigured `AllowedTargets` cases to ensure no unintended reverts occur.

[Go back to Findings Summary](#)

H7: Missing access control on exit allows arbitrary liquidity decreases on vault-managed NFT positions

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamModifyPositionFuse.sol	Type:	Access control

Description

The `exit` function is publicly callable and lacks any caller authentication. It validates only that the derived pool for the `tokenId` is granted in the market configuration via `PlasmaVaultConfigLib.isMarketSubstrateGranted`, which constrains the pool but not who may operate the position.

In typical deployments, this fuse contract is approved as operator for the vault-owned NFT positions in the `NonfungiblePositionManager`. Because the on-chain approval is granted to the fuse contract itself, any external caller can invoke `exit` and cause this contract to execute `decreaseLiquidity` on arbitrary vault-managed positions (subject to the pool allowlist). The parameters `liquidity`, `amount0Min`, `amount1Min`, and `deadline` are attacker-controlled, enabling griefing through forced partial or full liquidity removal and slippage-free execution (e.g., `amount0Min = amount1Min = 0`). Returned tokens are accounted to the position (not the caller), but the strategy’s exposure is reduced, and automation assumptions are broken.

This is a classic missing access control defect: authorization is enforced at the substrate (pool) level but not at the caller level.

*Listing 13. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamModifyPositionFuse.sol*

```

228     function exit(
229         VelodromeSuperchainSlipstreamModifyPositionFuseExitData memory data_
230     )
231     public
232     returns (
233         VelodromeSuperchainSlipstreamModifyPositionFuseExitResult memory
234         result
235     )
236     {
237         (address token0, address token1, int24 tickSpacing) =
238         _getPositionInfo(
239             data_.tokenId
240         );
241         address pool =
242         VelodromeSuperchainSlipstreamSubstrateLib.getPoolAddress(
243             FACTORY,
244             token0,
245             token1,
246             tickSpacing
247         );
248         if (
249             !PlasmaVaultConfigLib.isMarketSubstrateGranted(
250                 MARKET_ID,
251                 VelodromeSuperchainSlipstreamSubstrateLib.substrateToBytes32(
252                     VelodromeSuperchainSlipstreamSubstrate({
253                         substrateType:
254                             VelodromeSuperchainSlipstreamSubstrateType
255                             .Pool,
256                         substrateAddress: pool
257                     })
258                 )
259             ) {
260                 revert
261                 VelodromeSuperchainSlipstreamModifyPositionFuseUnsupportedPool(
262                     pool
263                 );
264             }
265         }
266     }

```



```

263
264     INonfungiblePositionManager.DecreaseLiquidityParams
265         memory params = INonfungiblePositionManager
266             .DecreaseLiquidityParams({
267                 tokenId: data_.tokenId,
268                 liquidity: data_.liquidity,
269                 amount0Min: data_.amount0Min,
270                 amount1Min: data_.amount1Min,
271                 deadline: data_.deadline
272             });
273
274     (uint256 amount0, uint256 amount1) = INonfungiblePositionManager(
275         NONFUNGIBLE_POSITION_MANAGER
276     ).decreaseLiquidity(params);
277
278     result.tokenId = data_.tokenId;
279     result.amount0 = amount0;
280     result.amount1 = amount1;
281
282     emit VelodromeSuperchainSlipstreamModifyPositionFuseExit(
283         VERSION,
284         result.tokenId,
285         result.amount0,
286         result.amount1
287     );
288 }

```

Exploit Scenario

An external account can force liquidity reductions on vault-managed positions as long as this fuse is approved as an operator for those NFTs.

Consider the following attack scenario:

1. Alice observes a vault-managed `tokenId` in the relevant pool and that the fuse is approved as operator for the position;
2. Alice calls `exit` with `liquidity` set to a small value and `amount0Min = amount1Min = 0`, repeating this multiple times to drain liquidity in increments;
3. The fuse contract, acting as the approved operator, calls

`NonfungiblePositionManager.decreaseLiquidity`, accounting tokens owed to the position and reducing its active liquidity; and

4. Bob's vault strategy ends up with materially lower/zero liquidity in the position, losing expected yield and breaking rebalance assumptions, effectively a DoS/griefing vector.

Recommendation

Restrict who can call `exit`.

Recommended options:

- Enforce caller authorization, e.g., `require(msg.sender == plasmaVault || AccessManager.isAuthorized(msg.sender, selector))` so only the orchestrator/vault (or a role) can invoke `exit`;
- If feasible, also assert ownership:
`require(INonfungiblePositionManager(NONFUNGIBLE_POSITION_MANAGER).ownerOf(data_.tokenId) == plasmaVault)` to avoid acting on foreign NFTs; and
- Mirror the same access control on the transient variant (`exitTransient`), ensuring both code paths are protected.

As a defense in depth, consider rate-limiting liquidity decreases per `tokenId` and enforcing non-zero `amount{0,1}Min` parameters via pre-hooks to reduce griefing space.

[Go back to Findings Summary](#)

H8: Pool path counts all NFPM NFTs regardless of pool, causing cross-pool double-counting and mispricing

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/aerodrome_slipstream/AerodromeSlipstreamBalanceFuse.sol	Type:	Logic error

Description

In function `balanceOf` under the `Pool` branch, the code enumerates every position NFT owned by `address(this)` using the Nonfungible Position Manager and never verifies that a given `tokenId` belongs to the current substrate pool being valued.

Next, it fetches `token0`, `token1` and `sqrtPriceX96` from the current substrate pool and uses those values to compute principal and fees for each enumerated `tokenId`. This means positions from unrelated pools are valued with the wrong price and token pair.

Finally, the aggregated `amount0` and `amount1` are converted to USD using the current substrate pool’s tokens, further cementing a cross-pool mismatch.

Effects:

- With multiple `Pool` substrates configured, the same unfiltered set of NFTs is iterated and accounted for in every substrate, inflating balances via cross-pool double-counting.
- Even with a single `Pool` substrate, unrelated NFTs held by the vault are mispriced because amounts are computed with the current substrate’s

`sqrPriceX96` and converted using the current substrate's token addresses, not the actual pair of the NFT.

Listing 14. Code snippet from
contracts/fuses/aerodrome_slipstream/AerodromeSlipstreamBalanceFuse.sol

```
78         tokenIdsLen = INonfungiblePositionManager(  
79             NONFUNGIBLE_POSITION_MANAGER  
80         ).balanceOf(address(this));  
81         tokenIds = new uint256[](tokenIdsLen);  
82  
83         for (uint256 j; j < tokenIdsLen; j++) {  
84             tokenIds[j] = INonfungiblePositionManager(  
85                 NONFUNGIBLE_POSITION_MANAGER  
86             ).tokenOfOwnerByIndex(address(this), j);  
87         }  
88  
89         token0 = ICLPool(substrate.substrateAddress).token0();  
90         token1 = ICLPool(substrate.substrateAddress).token1();  
91         sqrPriceX96 = ICLPool(substrate.substrateAddress)  
92             .slot0()  
93             .sqrPriceX96;  
94  
95         for (uint256 j; j < tokenIdsLen; j++) {  
96             (amount0, amount1) = _addPrincipal(  
97                 amount0,  
98                 amount1,  
99                 tokenIds[j],  
100                 sqrPriceX96  
101             );  
102             (amount0, amount1) = _addFees(  
103                 amount0,  
104                 amount1,  
105                 tokenIds[j]  
106             );  
107         }  
108  
109         balance += _convertToUsd(  
110             amount0,  
111             token0,  
112             priceOracleMiddleware  
113         );  
114         balance += _convertToUsd(  
115             amount1,
```

```
116             token1,  
117             priceOracleMiddleware  
118         );
```

Exploit Scenario

This flaw allows arbitrary NFTs from other pools to be included and priced under the wrong pool context, corrupting balance accounting.

Consider the following attack scenario:

1. Alice mints a cheap or worthless Slipstream position NFT in an unrelated pool and keeps ownership;
2. Alice calls `transferFrom` on the Nonfungible Position Manager to send that NFT to the vault address (contracts cannot block `transferFrom` with `onERC721Received`);
3. The vault later calls `updateMarketsBalances`, which delegatecalls `balanceOf` in this fuse; and
4. For each configured `Pool` substrate, the function enumerates all owner NFTs (including Alice's unrelated NFT) and values it using the substrate's `sqrPriceX96` and token pair, inflating or skewing the reported USD balance.

Realistic impact:

- Reported market balances become incorrect, weakening risk limits and downstream accounting based on `PlasmaVaultMarketsLib.updateMarketsBalances`.
- With several `Pool` substrates, the inclusion happens repeatedly across substrates, amplifying the error.

Recommendation

Filter and value only NFTs that actually belong to the current substrate pool,

and convert amounts using the correct token pair for each position.

Apply the following changes:

- Replace owner-wide enumeration with pool-specific selection. If a pool-filtered method such as `userPositions(owner, pool)` is not available, iterate owner NFTs but verify membership via `INonfungiblePositionManager.positions(tokenId)` and skip NFTs whose `token0/token1` (and tick spacing) do not map to `substrate.substrateAddress`.
- When converting to USD, use the token addresses of the position being valued. If the implementation guarantees only positions from the current pool are processed, using `token0/token1` fetched from that pool is acceptable; otherwise, obtain `token0` and `token1` from `positions(tokenId)` per NFT.
- Avoid cross-substrate double-counting. Ensure each NFT contributes to exactly one substrate (the pool or gauge it belongs to), never all.

Optional hardening:

- Maintain a curated set of position IDs per pool when positions are created/removed by the vault, and have `balanceOf` read that set instead of scanning all owner NFTs. This removes exposure to unsolicited NFT transfers and improves performance predictability.

[Go back to Findings Summary](#)

H9: Unbounded iteration over externally added Slipstream NFT positions enables DoS of vault balance updates

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamBalanceFuse.sol	Type:	Denial of service

Description

The `balanceOf` function iterates over every configured substrate and, for pool and gauge substrates, loops over all NFT positions owned by `address(this)` and performs heavy per-token computations. The arrays returned by `INonfungiblePositionManager.userPositions(address(this), pool)` and `ILeafCLGauge.stakedValues(address(this))` are unbounded and attacker-inflatable for pools because the vault unconditionally accepts ERC721 transfers. Each iteration calls `ISlipstreamSugar.principal` and, for pools, `ISlipstreamSugar.fees`, then performs two USD conversions per substrate. Executed via `functionDelegateCall` during normal vault flows, this design allows a single actor to push the loop into out-of-gas reverts, denying service to balance updates.

Listing 15. Code snippet from `contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamBalanceFuse.sol`

```
101 for (uint256 i; i < len; i++) {
102     substrate = VelodromeSuperchainSlipstreamSubstrateLib
103         .bytes32ToSubstrate(grantedSubstrates[i]);
104     amount0 = 0;
```

```

105     amount1 = 0;
106
107     if (
108         substrate.substrateType ==
109         VelodromeSuperchainSlipstreamSubstrateType.Pool
110     ) {
111         tokenIds = INonfungiblePositionManager(
112             NONFUNGIBLE_POSITION_MANAGER
113         ).userPositions(address(this), substrate.substrateAddress);
114
115         uint256 tokenIdsLen = tokenIds.length;
116         token0 = ICLPool(substrate.substrateAddress).token0();
117         token1 = ICLPool(substrate.substrateAddress).token1();
118         sqrtPriceX96 = ICLPool(substrate.substrateAddress)
119             .slot0()
120             .sqrtPriceX96;
121
122         for (uint256 j; j < tokenIdsLen; j++) {
123             (amount0, amount1) = _addPrincipal(
124                 amount0,
125                 amount1,
126                 tokenIds[j],
127                 sqrtPriceX96
128             );
129             (amount0, amount1) = _addFees(
130                 amount0,
131                 amount1,
132                 tokenIds[j]
133             );
134         }
135
136         balance += _convertToUsd(
137             amount0,
138             token0,
139             priceOracleMiddleware
140         );
141         balance += _convertToUsd(
142             amount1,
143             token1,
144             priceOracleMiddleware
145         );
146     } else if (
147         substrate.substrateType ==
148         VelodromeSuperchainSlipstreamSubstrateType.Gauge
149     ) {
150         tokenIds = ILeafCLGauge(substrate.substrateAddress)

```



```

151         .stakedValues(address(this));
152         uint256 tokenIdsLen = tokenIds.length;
153         token0 = ILeafCLGauge(substrate.substrateAddress).token0();
154         token1 = ILeafCLGauge(substrate.substrateAddress).token1();
155         sqrtPriceX96 = ILeafCLGauge(substrate.substrateAddress)
156             .pool()
157             .slot0()
158             .sqrtPriceX96;
159
160         for (uint256 j; j < tokenIdsLen; j++) {
161             (amount0, amount1) = _addPrincipal(
162                 amount0,
163                 amount1,
164                 tokenIds[j],
165                 sqrtPriceX96
166             );
167         }
168
169         balance += _convertToUsd(
170             amount0,
171             token0,
172             priceOracleMiddleware
173         );
174         balance += _convertToUsd(
175             amount1,
176             token1,
177             priceOracleMiddleware
178         );
179     }
180 }

```

The `balanceOf` function is reached by delegatecall from the vault during state-changing flows, so its gas usage directly determines whether core operations succeed.

*Listing 16. Code snippet from
contracts/vaults/lib/PlasmaVaultMarketsLib.sol*

```

48 balanceFuse = FusesLib.getBalanceFuse(markets[i]);
49
50 wadBalanceAmountInUSD = abi.decode(
51     balanceFuse.functionDelegateCall(
52         abi.encodeWithSignature("balanceOf()")

```

```
53     ),  
54     (uint256)  
55 );
```

The vault unconditionally accepts ERC721 tokens, allowing anyone to transfer Slipstream LP NFTs to it. This enables an attacker to arbitrarily grow `userPositions(address(this), pool)` for any configured pool substrate.

Listing 17. Code snippet from contracts/vaults/PlasmaVault.sol

```
298 function onERC721Received(  
299     address operator,  
300     address from,  
301     uint256 tokenId,  
302     bytes calldata data  
303 ) external returns (bytes4) {  
304     return IERC721Receiver.onERC721Received.selector;  
305 }
```

Exploit Scenario

A malicious actor can force `balanceOf` to iterate over an attacker-controlled number of NFTs and consume excessive gas until vault operations revert.

Consider the following attack scenario:

1. Alice mints a large number of minimal-liquidity Slipstream position NFTs for a pool that the market has configured as a substrate, setting the `recipient` to the vault or subsequently `safeTransferFrom`-ing them to the vault. The vault accepts the NFTs.
2. Alice triggers any operation that leads the vault to update market balances, for example by making a small deposit that calls `updateMarketsBalances`.
3. The vault delegatecalls the fuse `balanceOf`, which loops over all `userPositions(address(this), pool)` and calls `ISlipstreamSugar.principal` and `ISlipstreamSugar.fees` for each token, then performs USD conversions

for both tokens.

4. Gas usage scales linearly with the number of donated NFTs. When the loop exceeds the block gas budget, the call reverts and the user operation fails, effectively denying service for balance updates across that market.

Recommendation

Implement hard bounds and provenance checks so `balanceOf` cannot be forced to process unbounded, attacker-supplied NFT arrays.

Fix the vulnerability by either:

- Rejecting unsolicited NFTs: guard `onERC721Received` to only accept positions originated from authorized fuse flows (e.g., `operator` is a known fuse and `msg.sender` equals the expected `INonfungiblePositionManager`). Otherwise revert. Track accepted `tokenId`'s in storage during the authorized flow.
- Tracking known positions: maintain an internal list or bitmap of vault-owned `tokenId`'s per substrate updated by fuse actions (`mint`, `deposit to gauge`, `burn`, `withdraw`). Replace calls to `userPositions(address(this), pool)` and `stakedValues(address(this))` with iterating this bounded, internally controlled set.
- Paginating computation: add a paged balance function that processes at most `N` positions per call and accumulates results in storage or returns partials consumed by the caller, ensuring gas limits are respected. Avoid requiring a full scan inside a single transaction that is critical for core flows.
- Prefer staked positions only: optionally ignore pool-held NFTs and require positions to be staked in a gauge, combined with the internal tracking above. This reduces the surface for unsolicited donations.

Additionally, consider caching price and pool parameters within a call and

minimizing per-token external calls to reduce gas-per-item if a loop is still necessary.

[Go back to Findings Summary](#)

M1: instantWithdraw uses deposit-oriented convertToShares for withdrawals, causing rounding-down shortfalls

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol	Type:	Logic error

Description

The function `instantWithdraw` interprets `params_[0]` as an amount of the underlying asset, then converts that asset amount to staking token shares using `IERC4626.convertToShares` and passes the result as `fluidTokenAmount` to `_exit`.

Under ERC4626, `convertToShares(assets)` is defined for deposit flows and rounds down. For withdrawals, the correct function is `previewWithdraw(assets)`, which returns the number of shares that must be burned to obtain the requested assets and must round up. Using `convertToShares` here underestimates the shares required, especially around rounding boundaries, which results in withdrawing too few shares from the staking pool. In edge cases, small underlying `amount` values can round down to zero shares, causing the function to perform no withdrawal at all even though a positive `amount` was requested.

Affected elements:

- Function `instantWithdraw` (computes shares using `convertToShares` for a withdrawal intent);
- Function `_exit` (consumes the underestimated `fluidTokenAmount` and

withdraws fewer shares than necessary);

- External interface **IERC4626** as the conversion source.

Technical evidence is shown below.

*Listing 18. Code snippet from
contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol*

```
141    /// @dev params[0] - amount in underlying asset, params[1] - stakingPool
    address
142    function instantWithdraw(bytes32[] calldata params_) external override {
143        uint256 amount = uint256(params_[0]);
144
145        if (amount == 0) {
146            return;
147        }
148
149        address stakingPool = TypeConversionLib.toAddress(params_[1]);
150
151        _exit(
152            FluidInstadappStakingSupplyFuseExitData({
153                stakingPool: stakingPool,
154                fluidTokenAmount: IERC4626(
155                    IFluidLendingStakingRewards(stakingPool).stakingToken()
156                ).convertToShares(amount)
157            }),
158            true
159        );
160    }
```

Impact in production:

- For small **amount** values below one share's worth, **convertToShares(amount)** returns 0, so no shares are withdrawn despite a non-zero underlying request;
- For larger values, systematic rounding down causes shortfalls, so downstream components expecting exact amounts may observe deficits or need additional withdraw iterations, increasing gas and complexity;

- If integrated into a pipeline that assumes exactness, this can surface as reverts or user-visible inconsistencies.

Exploit Scenario

A caller can trigger systematic shortfalls caused by rounding-down share conversion.

Consider the following attack scenario:

1. Alice initiates an instant withdrawal for a very small underlying `amount` (below one share's worth);
2. Alice passes `params_[0] = amount` and a valid `stakingPool` to the fuse;
3. The contract computes `fluidTokenAmount = IERC4626(stakingToken).convertToShares(amount)`, which rounds down to 0 shares;
4. The withdrawal path proceeds with `fluidTokenAmount = 0, _exit` early-returns, and no shares are withdrawn, while Bob's protocol still expects the requested underlying to be covered, causing a shortfall or a follow-up failure.

Recommendation

Replace the deposit-oriented conversion with the withdrawal-oriented preview that rounds up.

Fix the vulnerability by either:

- Using `IERC4626(stakingToken).previewWithdraw(amount)` to compute the required shares for a given underlying `amount` (rounding up); or
- If protocol-specific constraints apply, implement an equivalent rounding-up share calculation that matches the ERC4626 `previewWithdraw` semantics and validate against the current staked share balance.

Additionally:

- Keep the existing min against `balanceOf` in `_exit` to avoid over-withdraw attempts;
- Add a sanity check to revert when the computed shares are 0 while `amount > 0`, to make failure explicit instead of silently doing nothing.

[Go back to Findings Summary](#)

M2: Missing access control on **enter** allows anyone to stake contract-held BPT into granted gauges

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/balancer/BalancerGaugeFuse.sol	Type:	Access control

Description

The function **enter** is publicly callable and lacks any caller authorization. The only runtime check ensures that the provided **gaugeAddress** is a granted substrate for **MARKET_ID**. Because **depositAmount** is computed from the contract’s own BPT balance, any external caller can trigger staking of the vault-held BPT into a granted gauge at arbitrary times.

This enables grieving and sequencing disruption rather than direct theft: a third party can force-deposit the contract’s BPT into the gauge just before other orchestrated actions that expected tokens to be held by the contract. This breaks timing assumptions, can reduce rewards due to unexpected unstaked periods, and forces extra gas expenditure to revert the position.

Listing 19. Code snippet from `contracts/fuses/balancer/BalancerGaugeFuse.sol`

```
128     function enter(  
129         BalancerGaugeFuseEnterData memory data_  
130     ) public payable returns (uint256 depositAmount) {  
131         if (data_.bptAmount == 0) {  
132             return 0;  
133         }  
134  
135         if (  
136             !PlasmaVaultConfigLib.isMarketSubstrateGranted(  

```

```

137         MARKET_ID,
138         BalancerSubstrateLib.substrateToBytes32(
139             BalancerSubstrate({
140                 substrateType: BalancerSubstrateType.GAUGE,
141                 substrateAddress: data_.gaugeAddress
142             })
143         )
144     )
145 } {
146     revert BalancerGaugeFuseUnsupportedGauge(data_.gaugeAddress);
147 }
148 address lpToken = ILiquidityGauge(data_.gaugeAddress).lp_token();
149 uint256 balanceOfPlasmaVault = IERC20(lpToken).balanceOf(address
(this));
150
151 depositAmount = data_.bptAmount > balanceOfPlasmaVault
152     ? balanceOfPlasmaVault
153     : data_.bptAmount;
154
155 if (depositAmount < data_.minBptAmount) {
156     revert BalancerGaugeFuseInsufficientBptAmount(
157         data_.gaugeAddress,
158         data_.bptAmount,
159         data_.minBptAmount
160     );
161 }
162
163 IERC20(lpToken).forceApprove(data_.gaugeAddress, depositAmount);
164 ILiquidityGauge(data_.gaugeAddress).deposit(
165     depositAmount,
166     address(this),
167     false
168 );
169
170 emit BalancerGaugeFuseEnter(
171     VERSION,
172     data_.gaugeAddress,
173     data_.bptAmount
174 );
175 }

```

Exploit Scenario

A malicious third party can manipulate staking state without authorization.

Consider the following attack scenario:

1. Alice observes that `enter` is permissionless for any gauge that is granted for `MARKET_ID`;
2. Alice calls `enter` with `gaugeAddress` set to an approved gauge, `bptAmount` set to a very large value, and `minBptAmount = 0` so that the contract deposits up to its full BPT balance;
3. The contract stakes the contract-held BPT into the gauge via `ILiquidityGauge.deposit` with `beneficiary = address(this)`; and
4. Bob, who operates the orchestrator, finds that subsequent transactions revert or must perform extra operations because BPT is no longer in the wallet when expected, incurring gas costs and reducing yield.

Recommendation

Restrict who can call staking operations and preserve intended sequencing.

Fix the vulnerability by either:

- Adding explicit caller authorization to `enter` (for example, an `onlyOrchestrator/onlyVault` modifier or a role-based check via `AccessControl`);
- Mirroring the same authorization on `enterTransient` which delegates to `enter`; and
- Emitting events with accurate amounts, and keeping `minBptAmount` checks as a safety constraint.

Additionally, consider adding a circuit-breaker (e.g., pause) so the orchestrator can quickly halt permissionless interference if misconfiguration occurs.

[Go back to Findings Summary](#)

M3: Unbounded ERC-721 enumeration lets anyone spam the vault with NFTs and DoS balanceOf

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamBalanceFuse.sol	Type:	Denial of service

Description

The `Pool` branch uses `ERC721Enumerable`-style iteration over all position NFTs owned by `address(this)`. Any external party can transfer arbitrary Nonfungible Position Manager NFTs to this address via `transferFrom`, which does not require `onERC721Received`.

As a result, `tokenIdsLen` can be increased without consent, and the function will loop over every injected token ID on each call, including once per configured `Pool` substrate.

Listing 20. Code snippet from `contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamBalanceFuse.sol`

```
78         tokenIdsLen = INonfungiblePositionManager(
79             NONFUNGIBLE_POSITION_MANAGER
80         ).balanceOf(address(this));
81         tokenIds = new uint256[](tokenIdsLen);
82
83         for (uint256 j; j < tokenIdsLen; j++) {
84             tokenIds[j] = INonfungiblePositionManager(
85                 NONFUNGIBLE_POSITION_MANAGER
86             ).tokenOfOwnerByIndex(address(this), j);
87         }
```

Consequences:

- A malicious user can bloat the number of NFTs owned by the vault, driving $O(S \times N)$ work in `balanceOf` (S = number of `Pool` substrates, N = number of owner NFTs), risking out-of-gas reverts in transactional contexts that call `updateMarketsBalances`.
- Data integrity is also affected because unrelated NFTs are later valued under the current substrate pool context (see separate correctness concerns for valuation).

Exploit Scenario

Consider the following attack scenario:

1. Alice mints many minimal-liquidity Slipstream position NFTs (possibly across arbitrary pools);
2. Alice calls `transferFrom` repeatedly to send those NFTs to the vault address, which cannot block non-safe transfers;
3. Bob later triggers a vault operation that calls `updateMarketsBalances`, which delegatecalls `balanceOf`; and
4. The function must iterate over all injected NFTs, potentially exceeding gas limits and reverting the transaction, resulting in a denial of service for Bob and any users relying on the operation.

This attack does not require any approvals from the vault and is limited only by the attacker's ability to create or obtain NFTs and the chain's per-transaction gas limits.

Recommendation

Eliminate unbounded owner-wide NFT enumeration in `balanceOf` and restrict processing to positions actually tracked for each substrate.

Recommended changes:

- Do not rely on `balanceOf` + `tokenOfOwnerByIndex` for pool positions. Instead, maintain a curated on-chain list of position IDs per pool when the vault opens/closes positions, and have `balanceOf` read from that list.
- If maintaining state is not feasible, at minimum fetch and process only positions that belong to the current substrate pool (e.g., via a pool-filtered view like `userPositions(owner, pool)` if available, or by filtering `positions(tokenId)` results). This reduces the impact of unsolicited NFTs to the filtering step and avoids counting them.
- Add operational safeguards: bound the maximum number of positions considered per call, and/or break work across multiple transactions if many positions exist.

[Go back to Findings Summary](#)

M4: Deterministic pool address prediction tied to current factory implementation (DoS after upgrades)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamLib.sol	Type:	Logic error

Description

The library derives a pool address by predicting a CREATE2 minimal-proxy clone that embeds the factory’s current `poolImplementation` in the init code. Because the init code hash changes whenever `poolImplementation` changes, the predicted address for the same `(token0, token1, tickSpacing)` also changes. Pools deployed before a factory upgrade will therefore no longer match the address predicted after the upgrade, breaking determinism over time.

This is fragile because higher-level fuses gate access using the predicted address rather than the actual deployed pool returned by the factory. After a factory upgrade, legitimate pools can be incorrectly rejected as unsupported, causing a denial of service for valid operations. The library also does not verify that any code exists at the predicted address, making it possible to gate on a non-existent pool if governance accidentally grants such an address.

Listing 21. Code snippet from `contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamLib.sol`

```
82 function _computeAddress(  
83     address factory_,
```

```

84     PoolKey memory key_
85 ) private view returns (address pool) {
86     if (!(key_.token0 < key_.token1)) {
87         revert WrongTokenOrder();
88     }
89
90     pool = _predictDeterministicAddress(
91         ICLFactory(factory_).poolImplementation(),
92         keccak256(abi.encode(key_.token0, key_.token1, key_.tickSpacing)),
93         factory_
94     );
95 }

```

Affected functions and usage:

- Function `_computeAddress` uses the current `ICLFactory(factory_).poolImplementation()` in the init code used for CREATE2 prediction; any change to the implementation changes the predicted address for an unchanged salt.
- Function `getPoolAddress` relies on `_computeAddress` for gating in calling fuses, rather than querying `ICLFactory.getPool(tokenA, tokenB, tickSpacing)`.
- Call sites include checks in `AreodromeSlipstreamModifyPositionFuse.validatePool` and `AreodromeSlipstreamNewPositionFuse.enter`, which gate on the predicted address. After a factory upgrade, these gates can reject legitimate pools.

Exploit Scenario

A practical denial-of-service manifests when the factory updates its pool implementation after pools have already been deployed.

Consider the following attack scenario:

1. Alice, the factory admin, upgrades the factory so that `poolImplementation()` now points to a new implementation contract.

2. Bob, who has a legitimate position in an existing pool (`token0`, `token1`, `tickSpacing`) created before the upgrade, calls a fuse function to modify or close his position.
3. The fuse calls `AreodromeSlipstreamSubstrateLib.getPoolAddress`, which internally predicts an address using the new `poolImplementation` and the same salt.
4. The predicted address no longer matches the existing deployed pool, so the gating check treats the pool as unsupported and reverts. Bob cannot operate on his valid position, resulting in a denial of service.

Recommendation

Replace prediction-based addressing with authoritative lookup from the factory and harden gating logic.

Recommended changes:

- In `getPoolAddress`, call `ICLFactory(factory_).getPool(token0, token1, tickSpacing)` to obtain the actual deployed pool address. Do not derive the address from the current `poolImplementation`.
- If no pool exists, explicitly revert with a clear error; do not accept a predicted address that has no code.
- Where gating relies on pool addresses, ensure the granted substrate list stores and compares the actual pool address returned by the factory.
- Optionally, add a code-existence check (`address(pool).code.length > 0`) as a defense-in-depth measure when interacting with externally derived addresses.

[Go back to Findings Summary](#)

M5: Unrestricted exitTransient enables forced withdrawals via transient inputs (no caller authorization)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamLeafCLGaugeFuse.sol	Type:	Access control

Description

The `exitTransient` function is `external` and implements no caller authorization. It reads parameters from transient storage using `TransientStorageLib.getInputs(VERSION)`, decodes them into `gaugeAddress` and `tokenId`, and then calls `exit(data_)`.

In deployments where transient flows are used (e.g., via `delegatecall` from an orchestrator that pre-populates inputs), the absence of access control means any external caller can trigger a withdrawal when the transient inputs are present. Functionally, `exitTransient` exposes the same withdrawal surface as `exit` but sourced from transient inputs.

Affected function and external interaction:

- Function `exitTransient` (reads transient inputs keyed by `VERSION`, then calls `exit` which calls `ILeafCLGauge.withdraw`).

*Listing 22. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlip
streamLeafCLGaugeFuse.sol*

```
216     function exitTransient() external {
217         bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
218
219         VelodromeSuperchainSlipstreamLeafCLGaugeFuseExitData
220             memory data_ =
221             VelodromeSuperchainSlipstreamLeafCLGaugeFuseExitData({
222                 gaugeAddress: TypeConversionLib.toAddress(inputs[0]),
223                 tokenId: TypeConversionLib.toUint256(inputs[1])
224             });
225
226         VelodromeSuperchainSlipstreamLeafCLGaugeFuseExitResult
227             memory result = exit(data_);
228
229         bytes32[] memory outputs = new bytes32[](2);
230         outputs[0] = TypeConversionLib.toBytes32(result.gaugeAddress);
231         outputs[1] = TypeConversionLib.toBytes32(result.tokenId);
232         TransientStorageLib.setOutputs(VERSION, outputs);
233     }
```

Exploit Scenario

The attack is viable whenever transient inputs are populated as part of normal protocol flows.

Consider the following attack scenario:

1. Alice observes a transaction flow that sets transient inputs for this fuse (e.g., an orchestrator prepares `gaugeAddress` and `tokenId`);
2. Alice front-runs or follows by calling `exitTransient()` directly on the fuse contract in the same block or compatible context;
3. The fuse decodes `gaugeAddress` and `tokenId` from transient storage and calls `exit(data_)`, which withdraws the NFT and stops emissions; and
4. Bob's vault suffers disrupted strategy execution and lost rewards until positions are re-staked.

Recommendation

Add explicit caller authorization to `exitTransient` and, if retained, defensively validate the transient context:

- Gate `exitTransient` with the same `onlyAuthorized` modifier used for `exit` (e.g., only the vault/governance/router may call);
- Validate that transient inputs exist and have the expected length/format before use; and
- Prefer passing parameters explicitly to privileged functions rather than relying on ambient transient state when the function is externally callable.

If transient entry points are only intended for internal orchestration, consider making them `internal` and invoking them via `delegatecall` from a privileged router, or remove public exposure entirely.

[Go back to Findings Summary](#)

M6: Missing access control on `exit` allows anyone to unstake the contract's gauge shares at arbitrary times

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/balancer/BalancerGaugeFuse.sol	Type:	Access control

Description

The function `exit` is publicly callable and lacks any caller authorization. The only gating verifies that the given `gaugeAddress` is granted for `MARKET_ID`. Because `withdrawAmount` is computed from this contract's gauge-share balance, any external caller can trigger an unstake back to the contract at arbitrary times.

This enables griefing and sequencing disruption: a third party can force-unstake just before orchestrated operations tuned for staked balances, interfering with reward accrual and boost mechanics, or causing subsequent transactions with non-zero `minBptAmount` to revert.

*Listing 23. Code snippet from
contracts/fuses/balancer/BalancerGaugeFuse.sol*

```
201     function exit(  
202         BalancerGaugeFuseExitData memory data_  
203     ) public payable returns (uint256 withdrawAmount) {  
204         if (data_.bptAmount == 0) {  
205             return 0;  
206         }  
207  
208         if (  
209             !PlasmaVaultConfigLib.isMarketSubstrateGranted(  
210                 MARKET_ID,  
211                 BalancerSubstrateLib.substrateToBytes32(  

```

```

212         BalancerSubstrate({
213             substrateType: BalancerSubstrateType.GAUGE,
214             substrateAddress: data_.gaugeAddress
215         })
216     )
217 ) {
218     revert BalancerGaugeFuseUnsupportedGauge(data_.gaugeAddress);
219 }
220
221
222 uint256 balanceOfPlasmaVault = IERC20(data_.gaugeAddress).balanceOf(
223     address(this)
224 );
225
226 withdrawAmount = data_.bptAmount > balanceOfPlasmaVault
227     ? balanceOfPlasmaVault
228     : data_.bptAmount;
229
230 if (withdrawAmount < data_.minBptAmount) {
231     revert BalancerGaugeFuseInsufficientBptAmount(
232         data_.gaugeAddress,
233         data_.bptAmount,
234         data_.minBptAmount
235     );
236 }
237
238 ILiquidityGauge(data_.gaugeAddress).withdraw(withdrawAmount, false);
239
240 emit BalancerGaugeFuseExit(VERSION, data_.gaugeAddress,
    withdrawAmount);

```

Exploit Scenario

An arbitrary account can force-withdraw the strategy's gauge shares.

Consider the following attack scenario:

1. Alice observes that `exit` is permissionless for gauges granted to `MARKET_ID`;
2. Alice calls `exit` with `bptAmount` set to a very large value and `minBptAmount = 0`, causing the contract to withdraw up to its full gauge-share balance;
3. The contract executes `ILiquidityGauge.withdraw` and returns BPT to the

contract balance; and

4. Bob's scheduled actions that assume the position remains staked now revert due to `minBptAmount` checks or accrue fewer rewards until restaked, wasting gas and time.

Recommendation

Apply caller authorization to unstaking operations.

Fix the vulnerability by either:

- Adding explicit caller authorization to `exit` (e.g., `onlyOrchestrator` / `onlyVault` or `AccessControl` roles);
- Mirroring the same authorization on `exitTransient`; and
- Keeping the `minBptAmount` check to enforce slippage/amount expectations.

As an additional hardening measure, consider a short cool-down or rate-limit for `exit` if operationally acceptable, to reduce griefing surface even if the authorization is temporarily misconfigured.

[Go back to Findings Summary](#)

M7: Incorrect use of ERC4626 convertToShares in instantWithdraw causes under-withdrawal and brittle downstream flows

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/curve_gauge/CurveChildLiquidityGaugeSupplyFuse.sol	Type:	Logic error

Description

The function `instantWithdraw` calculates the number of LP shares to withdraw from the Curve child gauge using `ERC4626.convertToShares` on the vault token. In ERC4626, `convertToShares/assets` is intended for the deposit path and is allowed to round down, while determining how many shares must be burned to withdraw a given amount of underlying should use `previewWithdraw/assets`, which must round up. Using `convertToShares` here can underestimate the shares required to later redeem the intended amount of underlying, leading to withdrawing too few LP shares from the gauge.

Affected code paths:

- Function `instantWithdraw` computes `lpTokenToWithdraw` via `ERC4626Upgradeable(lpToken).convertToShares(amount);`
- Internal function `_exit` withdraws exactly that many shares from the gauge without compensating for rounding; and
- Private function `_performWithdraw` performs `IChildLiquidityGauge.withdraw(finalAmount_, address(this), false)` and does not correct for under-withdrawal.

Effect in production:

- If the withdrawn shares are later used to call `ERC4626.withdraw(amount, ...)` or to satisfy an exact-underlying requirement in a composed flow, the call may revert (insufficient shares) or deliver less than requested. This creates brittle execution and potential denial-of-service on higher-level operations that expect exact amounts.

Listing 24. Code snippet from

contracts/fuses/curve_gauge/CurveChildLiquidityGaugeSupplyFuse.sol

```
127 /// @dev Could be used only if lpToken is ERC4626.
128 /// @dev params[0] - amount in underlying asset, params[1] - vault address
129 function instantWithdraw(bytes32[] calldata params_) external {
130     uint256 amount = uint256(params_[0]);
131
132     address gauge = PlasmaVaultConfigLib.bytes32ToAddress(params_[1]);
133     address lpToken = IChildLiquidityGauge(gauge).lp_token();
134     uint256 lpTokenToWithdraw = ERC4626Upgradeable(lpToken).convertToShares(
135         amount
136     );
137
138     _exit(
139         CurveChildLiquidityGaugeSupplyFuseExitData(
140             gauge,
141             lpTokenToWithdraw
142         ),
143         true
144     );
145 }
```

Exploit Scenario

A miscalculation of shares via `convertToShares` can be exercised to cause under-withdrawal and revert a composed operation that expects an exact amount of underlying.

Consider the following attack scenario:

1. Alice prepares a multi-step transaction that needs exactly 100 units of the ERC4626 vault's underlying to repay a debt in a downstream step;

2. Alice calls `instantWithdraw` with `params_[0] = 100` and `params_[1] = <gauge>` so the contract first withdraws LP shares from the Curve child gauge;
3. The contract computes `lpTokenToWithdraw = ERC4626Upgradeable(lpToken).convertToShares(100)`, which rounds down below `previewWithdraw(100)` and withdraws too few shares from the gauge;
4. In the next step, the flow attempts `ERC4626.withdraw(100, ...)` or otherwise assumes 100 underlying will be available, but the contract now holds fewer shares than required; and
5. Bob's protocol call reverts or receives less than 100 underlying, causing the whole transaction to fail or execute with a shortfall.

Recommendation

Replace the share calculation with the ERC4626 withdrawal-side preview that rounds up, and guard the flow with explicit checks.

Fix the vulnerability by either:

- Using `ERC4626Upgradeable(lpToken).previewWithdraw(amount)` to compute the required number of shares to withdraw from the gauge so that `withdraw(amount, ...)` will succeed; or
- If the next step will redeem shares directly, compute the target underlying from shares via `previewRedeem/convertToAssets` consistently and accept a caller-provided slippage parameter.

Additionally:

- Before calling the gauge, cap the calculated shares by the staked share balance and revert with a clear error if insufficient to fulfill `amount` (instead of silently under-withdrawing);
- Propagate and enforce a minimum-out or exact-underlying invariant for downstream redemption; and

- Consider emitting the calculated shares and intended underlying in the event to aid monitoring and debugging.

[Go back to Findings Summary](#)

M8: Missing per-asset whitelist enforcement in enter enables spending unapproved tokens via Permit2

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/balancer/BalancerLiquidityUnbalancedFuse.sol	Type:	Logic error

Description

The `enter` function validates only the pool substrate and that the provided tokens belong to the chosen pool. It does not verify that each token is granted as an allowed asset for `MARKET_ID` using `PlasmaVaultConfigLib.isSubstrateAsAssetGranted`.

Within the same transaction, the function force-approves `PERMIT2` and then grants the Balancer router a temporary allowance for each provided token and amount. Because per-asset grants are never checked, any token held by the calling context that is also registered in the target pool can be spent by the router up to `exactAmountsIn[i]`, bypassing the market’s asset policy. This undermines per-asset risk controls that are otherwise available in the configuration layer.

The contract defines a dedicated error for asset restriction violations but never uses it, which strongly indicates the missing validation.

Listing 25. Code snippet from `contracts/fuses/balancer/BalancerLiquidityUnbalancedFuse.sol`

```
160     function enter(  
161         BalancerLiquidityUnbalancedFuseEnterData memory data_  
162     ) public payable returns (uint256 bptAmountOut) {
```

```

163         if (data_.pool == address(0)) {
164             revert BalancerLiquidityUnbalancedFuseInvalidParams();
165         }
166         if (data_.tokens.length != data_.exactAmountsIn.length) {
167             revert BalancerLiquidityUnbalancedFuseInvalidParams();
168         }
169
170         if (
171             !PlasmaVaultConfigLib.isMarketSubstrateGranted(
172                 MARKET_ID,
173                 BalancerSubstrateLib.substrateToBytes32(
174                     BalancerSubstrate({
175                         substrateType: BalancerSubstrateType.POOL,
176                         substrateAddress: data_.pool
177                     })
178                 )
179             )
180         ) {
181             revert
182             BalancerLiquidityUnbalancedFuseUnsupportedPool(data_.pool);
183         }
184
185         BalancerSubstrateLib.checkTokensInPool(data_.pool, data_.tokens);
186
187         uint256 len = data_.tokens.length;
188         for (uint256 i; i < len; ++i) {
189             uint256 amountIn = data_.exactAmountsIn[i];
190             if (amountIn > 0) {
191                 IERC20(data_.tokens[i]).forceApprove(
192                     PERMIT2,
193                     type(uint256).max
194                 );
195                 IPermit2(PERMIT2).approve(
196                     data_.tokens[i],
197                     BALANCER_ROUTER,
198                     amountIn.toUint160(),
199                     uint48(block.timestamp + 1)
200                 );
201             }
202         }
203
204         bptAmountOut = IRouter(BALANCER_ROUTER).addLiquidityUnbalanced(
205             data_.pool,
206             data_.exactAmountsIn,
207             data_.minBptAmountOut,
208             false,

```

```

208         """
209     );
210
211     emit BalancerLiquidityUnbalancedFuseEnter(
212         VERSION,
213         data_.pool,
214         bptAmountOut,
215         data_.exactAmountsIn
216     );
217
218     for (uint256 i; i < len; ++i) {
219         if (data_.exactAmountsIn[i] > 0) {
220             IERC20(data_.tokens[i]).forceApprove(PERMIT2, 0);
221         }
222     }
223 }

```

*Listing 26. Additional evidence from
contracts/fuses/balancer/BalancerLiquidityUnbalancedFuse.sol*

```

106     /// @notice Thrown when a token is not granted as an asset for this
    market
107     /// @param token The address of the token that was not granted
108     /// @custom:error BalancerLiquidityUnbalancedFuseUnsupportedAsset
109     error BalancerLiquidityUnbalancedFuseUnsupportedAsset(address token);

```

Exploit Scenario

A malicious caller can cause the vault to approve and spend non-whitelisted assets as long as they are included in the selected pool.

Consider the following attack scenario:

1. Alice controls the privileged executor that can call the fuse via `PlasmaVault.execute`;
2. Alice selects a pool that is whitelisted at the substrate (pool) level and constructs `tokens/exactAmountsIn` where at least one token is not granted as an asset for `MARKET_ID` but is part of the pool;

3. The fuse force-approves `PERMIT2` and then approves `BALANCER_ROUTER` to spend those token amounts, without checking `isSubstrateAsAssetGranted` for each token; and
4. The router pulls the non-whitelisted token(s) from the vault and mints BPT, resulting in exposure built from assets that violate the market's asset policy and potentially creating accounting and risk-management issues.

Recommendation

Validate every provided token against the market's asset whitelist before setting approvals or calling the router.

Fix the vulnerability by:

- For each `data_.tokens[i]` with `data_.exactAmountsIn[i] > 0`, require `PlasmaVaultConfigLib.isSubstrateAsAssetGranted(MARKET_ID, data_.tokens[i])` to be true; otherwise, revert with `BalancerLiquidityUnbalancedFuseUnsupportedAsset(data_.tokens[i])`.
- Perform this check before `IERC20(...).forceApprove(PERMIT2, ...)` and `IPermit2(PERMIT2).approve(...)` are executed.
- Keep the approval cleanup (`forceApprove(PERMIT2, 0)`) after the operation.

This ensures that only market-granted assets can be approved and spent during `enter`, preserving per-asset risk controls.

[Go back to Findings Summary](#)

M9: Unrestricted exit allows anyone to force-withdraw vault-staked NFTs (griefing/DoS)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamLeafCLGaugeFuse.sol	Type:	Griefing

Description

The `exit` function is `public` and enforces only a substrate whitelist via `PlasmaVaultConfigLib.isMarketSubstrateGranted`. It performs no caller authorization.

Any external address can call `exit` with a `tokenId` staked by this contract in a whitelisted gauge. The call to `ILeafCLGauge.withdraw(tokenId)` succeeds because the gauge tracks deposits by this contract and returns the NFT back to this contract. While funds are not directly stolen, unauthorized callers can force positions out of gauges at arbitrary times, stopping emissions/compounding and disrupting strategy execution. This creates a griefing/DoS vector against the vault’s yield.

Affected function and external interaction:

- Function `exit` (state reads `MARKET_ID`, performs external call `ILeafCLGauge.withdraw`);
- External dependency `ILeafCLGauge` exposes views such as `stakedValues(address)`, enabling discovery of the vault’s staked `tokenId`s.

*Listing 27. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlip
streamLeafCLGaugeFuse.sol*

```
156     function exit(  
157         VelodromeSuperchainSlipstreamLeafCLGaugeFuseExitData memory data_  
158     )  
159     public  
160     returns (  
161         VelodromeSuperchainSlipstreamLeafCLGaugeFuseExitResult memory  
162         result  
163     )  
164     {  
165         if (  
166             !PlasmaVaultConfigLib.isMarketSubstrateGranted(  
167                 MARKET_ID,  
168                 VelodromeSuperchainSlipstreamSubstrateLib.substrateToBytes32(  
169                     VelodromeSuperchainSlipstreamSubstrate({  
170                         substrateType:  
171                             VelodromeSuperchainSlipstreamSubstrateType  
172                             .Gauge,  
173                             substrateAddress: data_.gaugeAddress  
174                         })  
175                     ) {  
176             revert VelodromeSuperchainSlipstreamLeafCLGaugeUnsupportedGauge(  
177                 data_.gaugeAddress  
178             );  
179         }  
180  
181         result.gaugeAddress = data_.gaugeAddress;  
182         result.tokenId = data_.tokenId;  
183  
184         ILeafCLGauge(data_.gaugeAddress).withdraw(data_.tokenId);  
185  
186         emit VelodromeSuperchainSlipstreamLeafCLGaugeExit(  
187             result.gaugeAddress,  
188             result.tokenId  
189         );  
190     }
```

Exploit Scenario

A realistic attack requires no special privileges beyond knowing a staked `tokenId` and its gauge (enumerable via gauge view functions).

Consider the following attack scenario:

1. Alice queries the gauge for positions staked by this contract using `stakedValues(address(this))` to identify a `tokenId` and its associated gauge;
2. Alice calls `exit({gaugeAddress, tokenId})` directly on the fuse contract;
3. The fuse validates the gauge is whitelisted and then calls `ILeafCLGauge.withdraw(tokenId)`, returning the NFT back to the fuse contract and stopping emissions; and
4. Bob's vault stops accruing rewards for that position and higher-level strategy logic may become desynchronized, causing operational disruption and reduced yield.

Recommendation

Implement explicit access control on `exit` so that only a trusted orchestrator (for example, the vault contract, governance, or a designated router) can invoke withdrawals.

Recommended hardening steps:

- Introduce an `onlyAuthorized` modifier (e.g., checking `msg.sender` against a stored privileged address) and apply it to `exit`;
- If multiple components must interact, manage an allowlist of callers that can perform `enter/exit` operations;
- Validate that `data_.tokenId` is actually staked by this contract in `data_.gaugeAddress` before withdrawing to avoid misuse; and

- Emit detailed events on access-denied attempts to aid monitoring.

Do not rely solely on substrate whitelists for security; they constrain which gauges are used but do not limit who can trigger withdrawals.

[Go back to Findings Summary](#)

M10: Permissionless **enter/exit** enable front-run griefing via **minBptAmount** reverts and sequencing disruption

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/balancer/ BalancerGaugeFuse.sol	Type:	Denial of service

Description

All state-changing entry/exit paths are permissionless, allowing a third party to front-run or interleave **enter/exit** operations and manipulate where the position resides (in-gauge vs. in-wallet) just before an orchestrated transaction executes. Because both paths enforce **minBptAmount** thresholds, an attacker can desynchronize available amounts and force reverts by making the orchestrator’s expected amount unavailable when the transaction runs.

The behavior is enabled by public/external **payable** entry points with no caller checks, combined with amount computations based on the contract’s current balances at execution time.

Listing 28. Code snippet from `contracts/fuses/balancer/BalancerGaugeFuse.sol` (`enter`)

```
128     function enter(  
129         BalancerGaugeFuseEnterData memory data_  
130     ) public payable returns (uint256 depositAmount) {  
131         if (data_.bptAmount == 0) {  
132             return 0;  
133         }  
134  
135         if (  
136             !PlasmaVaultConfigLib.isMarketSubstrateGranted(  

```

```

137         MARKET_ID,
138         BalancerSubstrateLib.substrateToBytes32(
139             BalancerSubstrate({
140                 substrateType: BalancerSubstrateType.GAUGE,
141                 substrateAddress: data_.gaugeAddress
142             })
143         )
144     )
145 } {
146     revert BalancerGaugeFuseUnsupportedGauge(data_.gaugeAddress);
147 }
148 address lpToken = ILiquidityGauge(data_.gaugeAddress).lp_token();
149 uint256 balanceOfPlasmaVault = IERC20(lpToken).balanceOf(address
(this));
150
151 depositAmount = data_.bptAmount > balanceOfPlasmaVault
152     ? balanceOfPlasmaVault
153     : data_.bptAmount;
154
155 if (depositAmount < data_.minBptAmount) {
156     revert BalancerGaugeFuseInsufficientBptAmount(

```

*Listing 29. Code snippet from
contracts/fuses/balancer/BalancerGaugeFuse.sol (exit)*

```

201 function exit(
202     BalancerGaugeFuseExitData memory data_
203 ) public payable returns (uint256 withdrawAmount) {
204     if (data_.bptAmount == 0) {
205         return 0;
206     }
207
208     if (
209         !PlasmaVaultConfigLib.isMarketSubstrateGranted(
210             MARKET_ID,
211             BalancerSubstrateLib.substrateToBytes32(
212                 BalancerSubstrate({
213                     substrateType: BalancerSubstrateType.GAUGE,
214                     substrateAddress: data_.gaugeAddress
215                 })
216             )
217         )
218     ) {
219         revert BalancerGaugeFuseUnsupportedGauge(data_.gaugeAddress);
220     }

```

```

221
222     uint256 balanceOfPlasmaVault = IERC20(data_.gaugeAddress).balanceOf(
223         address(this)
224     );
225
226     withdrawAmount = data_.bptAmount > balanceOfPlasmaVault
227         ? balanceOfPlasmaVault
228         : data_.bptAmount;
229
230     if (withdrawAmount < data_.minBptAmount) {
231         revert BalancerGaugeFuseInsufficientBptAmount(

```

Exploit Scenario

A front-running attacker can turn `minBptAmount` into a denial-of-service trigger.

Consider the following attack scenario:

1. Alice monitors mempool for Bob's orchestrated call that will `enter` with `minBptAmount = X`;
2. Just before Bob's transaction is mined, Alice calls `enter` herself with `minBptAmount = 0`, staking the contract's remaining BPT into the gauge so that only $< X$ remains in-wallet;
3. Bob's transaction executes and reverts because `depositAmount < minBptAmount` at execution time; and
4. Bob must resubmit or perform compensating transactions, repeatedly paying gas while Alice can continue to grief for minimal cost.

Recommendation

Eliminate permissionless manipulation of the strategy's staking state.

Fix the vulnerability by either:

- Restricting `enter`, `exit`, `enterTransient`, and `exitTransient` to an authorized orchestrator or vault role (e.g., `AccessControl` or an allowlist);

- If public access is required, batch operations into a single atomic call that cannot be front-run (e.g., via a multicall entry point) and use internal functions for the raw `enter/exit` primitives; and
- Add optional pausing to rapidly mitigate observed griefing in production.

Do not remove `minBptAmount` protections; they are valuable safety checks. Address the root cause by controlling who can move the position and when.

[Go back to Findings Summary](#)

M11: Misleading exit: amounts are accounted (not transferred); collect is omitted

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol	Type:	Logic error

Description

The `exit` function calls `INonfungiblePositionManager.decreaseLiquidity` and returns/emits the resulting amounts as if they were received. However, `decreaseLiquidity` only accounts amounts to the position’s tokens owed and does not transfer tokens to the caller or this contract. A separate `collect` call is required to actually move funds. This discrepancy makes the function and its event misleading and can break integrator assumptions about token availability immediately after `exit`.

Listing 30. Code snippet from `contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol`

```
355     function exit(
356         AreodromeSlipstreamModifyPositionFuseExitData memory data_
357     ) public returns (uint256 tokenId, uint256 amount0, uint256 amount1) {
358         if (data_.liquidity == 0) {
359             revert InvalidAmount();
360         }
361
362         validatePool(data_.tokenId);
363
364         INonfungiblePositionManager.DecreaseLiquidityParams
365             memory params = INonfungiblePositionManager
366             .DecreaseLiquidityParams({
```



```

367         tokenId: data_.tokenId,
368         liquidity: data_.liquidity,
369         amount0Min: data_.amount0Min,
370         amount1Min: data_.amount1Min,
371         deadline: data_.deadline
372     });
373
374     (amount0, amount1) = INonfungiblePositionManager(
375         NONFUNGIBLE_POSITION_MANAGER
376     ).decreaseLiquidity(params);
377
378     emit AreodromeSlipstreamModifyPositionFuseExit(
379         VERSION,
380         data_.tokenId,
381         amount0,
382         amount1
383     );
384
385     return (data_.tokenId, amount0, amount1);
386 }

```

The interface explicitly documents that `decreaseLiquidity` accounts amounts to tokens owed, not that tokens are transferred.

Listing 31. Code snippet from
contracts/fuses/aerodrome_slipstream/ext/INonfungiblePositionManager.sol

```

118     /// @notice Decreases the amount of liquidity in a position and accounts
119     it to the position
120     /// @param params tokenId The ID of the token for which liquidity is
121     being decreased,
122     /// amount The amount by which liquidity will be decreased,
123     /// amount0Min The minimum amount of token0 that should be accounted for
124     the burned liquidity,
125     /// amount1Min The minimum amount of token1 that should be accounted for
126     the burned liquidity,
127     /// deadline The time by which the transaction must be included to
128     effect the change
129     /// @return amount0 The amount of token0 accounted to the position's
130     tokens owed
131     /// @return amount1 The amount of token1 accounted to the position's
132     tokens owed
133     /// @dev The use of this function can cause a loss to users of the

```

Exploit Scenario

A user or integrator can be misled into believing tokens were transferred to the contract or caller, while they remain only as tokens-owed in the position. Downstream operations that assume immediate availability may revert or cause accounting mismatches.

Consider the following attack scenario:

1. Alice triggers `exit` on a managed position and reads the returned `amount0` and `amount1` values as if tokens were received;
2. Alice's integration then forwards those amounts to a user or to subsequent protocol steps, assuming the balances are present on-chain;
3. The contract never called `collect`, so no tokens were transferred; and
4. Bob's protocol ends up with failed transfers or negative accounting because the expected tokens are not in its custody.

Recommendation

Align behavior and documentation to avoid misleading integrators.

Fix the vulnerability by either:

- Performing a `collect` immediately after `decreaseLiquidity` so that tokens are actually transferred to a chosen recipient (e.g., `address(this)` or an explicit recipient parameter). Example:
- Call `INonfungiblePositionManager.collect` with `CollectParams({ tokenId: data_.tokenId, recipient: address(this), amount0Max: type(uint128).max, amount1Max: type(uint128).max })` and emit/return the collected amounts; or

- Updating the function NatSpec and `AreodromeSlipstreamModifyPositionFuseExit` event documentation to state that the amounts are accounted to the position's tokens owed and not received, and avoid implying immediate transfer.

When using the first option, ensure allowances and asset handling are consistent with vault custody rules, and that the recipient is appropriately controlled.

[Go back to Findings Summary](#)

M12: instantWithdraw calls unvalidated stakingPool and token before whitelist enforcement

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol	Type:	Trust model

Description

The function `instantWithdraw` performs external calls to a user-supplied `stakingPool` before any whitelist verification. Specifically, it calls `IFluidLendingStakingRewards(stakingPool).stakingToken()` and then `IERC4626(stakingToken).convertToShares(amount)` using the returned token address. Only later, inside `_exit`, is `stakingPool` checked against the whitelist.

Because the EVM does not enforce `view`, a malicious `stakingPool` can implement these functions to perform arbitrary state changes or reentrancy attempts. Even though funds are not moved at this stage, interacting with unvalidated addresses widens the attack surface and violates the intended trust boundaries.

Affected elements:

- Function `instantWithdraw` (external calls on unvalidated `stakingPool` and `stakingToken`);
- Function `_exit` (contains the whitelist check, but only after prior external interactions);
- State variable `MARKET_ID` indirectly controls the whitelist scope.

The ordering issue is evidenced below.

Listing 32. Code snippet from

contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol

```
151         _exit(  
152             FluidInstadappStakingSupplyFuseExitData({  
153                 stakingPool: stakingPool,  
154                 fluidTokenAmount: IERC4626(  
155                     IFluidLendingStakingRewards(stakingPool).stakingToken()  
156                 ).convertToShares(amount)  
157             }),  
158             true  
159         );
```

The whitelist is enforced only later inside `_exit`.

Listing 33. Code snippet from

contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol

```
166     ) internal returns (address stakingPool, uint256 withdrawAmount) {  
167         stakingPool = data_.stakingPool;  
168         if (  
169             !PlasmaVaultConfigLib.isSubstrateAsAssetGranted(  
170                 MARKET_ID,  
171                 stakingPool  
172             )  
173         ) {  
174             revert FluidInstadappStakingSupplyFuseUnsupportedStakingPool(  
175                 "exit",  
176                 stakingPool  
177             );  
178         }
```

Impact in production:

- Passing a malicious `stakingPool` causes the fuse to make external calls into attacker-controlled code before any whitelist gate, enabling reentrancy attempts or unexpected side effects;

- While later checks prevent fund movement for unwhitelisted pools, this ordering still increases risk and complicates analysis and auditing of call graphs.

Exploit Scenario

A malicious caller can force pre-check external interactions with attacker-controlled contracts.

Consider the following attack scenario:

1. Alice deploys a contract that implements the `stakingToken()` function and returns a token whose `convertToShares` triggers callbacks;
2. Alice calls `instantWithdraw` and sets `params_[1]` to her malicious `stakingPool` address;
3. Before any whitelist validation, the fuse executes `stakingToken()` and then `convertToShares`, allowing Alice's contracts to reenter or manipulate state in Bob's surrounding protocol context;
4. The subsequent `_exit` reverts on the whitelist check, but the pre-check interactions have already occurred, exposing Bob's protocol to unnecessary risk.

Recommendation

Enforce whitelist checks before any external interaction and minimize pre-check calls.

Fix the vulnerability by either:

- Moving the `PlasmaVaultConfigLib.isSubstrateAsAssetGranted(MARKET_ID, stakingPool)` check to the very beginning of `instantWithdraw` and reverting on failure before calling `stakingToken()` or any ERC4626 function; or
- Refactoring to compute `fluidTokenAmount` only after the whitelist check

and using a cached, validated `stakingToken` address.

Additionally:

- Consider adding a reentrancy guard on the entry point if it is reachable from external execution paths;
- Follow the checks-effects-interactions pattern to reduce reentrancy surface area.

[Go back to Findings Summary](#)

M13: Fee accrual is skipped when LP balance is zero, undercounting assets and enabling share price manipulation

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol	Type:	Logic error

Description

The function `balanceOf` only values fees when `liquidity > 0`, skipping `_calculateBalanceFromFees` entirely when the current LP balance for a substrate is zero. In Solidly/Velodrome-style pools, per-account fees are tracked lazily via `supplyIndex{0,1}` and `claimable{0,1}`. An account can retain positive `claimable0/claimable1` even after its LP balance becomes zero (e.g., after rebalancing or unstaking) because outstanding fees are stored to `claimable*` on transfer/burn.

By gating fee valuation on `liquidity > 0`, the fuse omits these owed fees from the reported USD balance, leading to underestimation during common operational flows.

Listing 34. Code snippet from `contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol`

```
89 if (liquidity > 0) {
90     balance += _calculateBalanceFromLiquidity(
91         pool,
92         priceOracleMiddleware,
93         liquidity
94     );
95     balance += _calculateBalanceFromFees(
96         pool,
```



```
97         priceOracleMiddleware,  
98         liquidity  
99     );  
100 }
```

Production impact:

- Understated TVL/share price while fees remain unclaimed and LP balance is temporarily zero.
- Adversaries can time deposits during these windows to mint disproportionately more shares, extracting value from existing shareholders when fees are later realized.

Exploit Scenario

Consider the following attack scenario:

1. Alice monitors vault operations and sees that positions are temporarily moved such that the vault's LP balance in a pool/gauge becomes zero while unclaimed fees remain recorded in the pool as `claimable0/claimable1` for the vault.
2. During this window, `balanceOf` returns no fee value because `liquidity == 0`, underpricing the vault.
3. Alice deposits into the vault and receives more shares per unit of value than she should.
4. Later, when the vault re-adds liquidity or claims fees, the valuation normalizes.
5. Bob, an existing shareholder, suffers dilution as Alice exits at a profit.

Recommendation

Do not gate fee valuation on current LP balance. Always include per-account fee claimables regardless of `liquidity`.

Concrete changes:

- Always read and value `IPool(pool).claimable0(account)` and `IPool(pool).claimable1(account)` even when `liquidity == 0`.
- For gauge-held positions, also include gauge-side fee accrual (e.g., `ILeafGauge(gauge).fees0()/fees1()`) apportioned by the vault's share of `gauge.totalSupply()`.
- Keep liquidity-based index deltas for incremental accruals, but ensure outstanding `claimable*` already stored for the account are never skipped due to a zero current LP balance.

Add tests that simulate: accrue fees, burn/withdraw to zero, then verify `_calculateBalanceFromFees` still contributes positive value.

[Go back to Findings Summary](#)

M14: Unbounded iteration over substrates and positions risks gas exhaustion in **balanceOf**

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamBalanceFuse.sol	Type:	Denial of service

Description

Function **balanceOf** iterates over all granted substrates and, for each **Pool** substrate, performs two loops over all owner NFTs: one to collect token IDs and another to accumulate principal and fees. This results in $O(S \times N)$ complexity (S = number of **Pool** substrates, N = number of owner NFTs) and makes the call cost scale poorly with configuration and holdings.

In transactional contexts where **updateMarketsBalances** is called on-chain, this can lead to out-of-gas reverts, blocking dependent operations.

Listing 35. Code snippet from contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamBalanceFuse.sol

```
68     for (uint256 i; i < len; i++) {
69         substrate = AreodromeSlipstreamSubstrateLib.bytes32ToSubstrate(
70             grantedSubstrates[i]
71         );
72         amount0 = 0;
73         amount1 = 0;
74
75         if (
76             substrate.substrateType ==
77             AreodromeSlipstreamSubstrateType.Pool
78         ) {
79             tokenIdLen = INonfungiblePositionManager(
79                 NONFUNGIBLE_POSITION_MANAGER
```

```

80         ).balanceOf(address(this));
81         tokenIds = new uint256[](tokenIdsLen);
82
83         for (uint256 j; j < tokenIdsLen; j++) {
84             tokenIds[j] = INonfungiblePositionManager(
85                 NONFUNGIBLE_POSITION_MANAGER
86             ).tokenOfOwnerByIndex(address(this), j);
87         }
88
89         token0 = ICLPool(substrate.substrateAddress).token0();
90         token1 = ICLPool(substrate.substrateAddress).token1();
91         sqrtPriceX96 = ICLPool(substrate.substrateAddress)
92             .slot0()
93             .sqrtPriceX96;
94
95         for (uint256 j; j < tokenIdsLen; j++) {
96             (amount0, amount1) = _addPrincipal(
97                 amount0,
98                 amount1,
99                 tokenIds[j],
100                 sqrtPriceX96
101             );
102             (amount0, amount1) = _addFees(
103                 amount0,
104                 amount1,
105                 tokenIds[j]
106             );
107         }

```

Exploit Scenario

Consider the following attack scenario:

1. Alice ensures the vault has many NFTs (e.g., by transferring in a large number of position NFTs);
2. Alice or any user calls a function path that invokes `updateMarketsBalances` (for example, a pre-hook, deposit, or withdrawal that updates market balances);
3. The delegatecall into `balanceOf` must enumerate S substrates and N positions, making many external view calls; and

4. The transaction runs out of gas and reverts, causing a denial of service for the operation and any dependent user flows.

Even without malicious action, a configuration with many `Pool` substrates and many owned positions can hit gas limits unintentionally.

Recommendation

Reduce the asymptotic work and bound per-call processing cost.

Concrete steps:

- Avoid owner-wide enumeration per substrate. Fetch only the positions belonging to the current substrate pool (e.g., via a pool-filtered view or by filtering `positions(tokenId)` once, grouping token IDs by pool, and processing each group exactly once).
- Cache or maintain a curated list of position IDs per substrate in vault storage and iterate that list; update it on mint/burn and gauge stake/unstake. This makes `balanceOf` $O(S + N_{\text{pool}})$ instead of $O(S \times N)$.
- If very large sets are expected, introduce batching or pagination semantics for balance updates, or precompute/carry forward partial aggregates to avoid scanning every position on each call.

[Go back to Findings Summary](#)

M15: `getPoolAddress` predicts pool using current factory implementation, causing mismatches and DoS after factory upgrades

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamLib.sol	Type:	Logic error

Description

`getPoolAddress` returns a pool by predicting the CREATE2 address of a minimal proxy constructed from the factory’s current `poolImplementation` and a salt `keccak256(abi.encode(token0, token1, tickSpacing))`. Since the minimal-proxy init code embeds the implementation address, changing `poolImplementation` modifies the init code hash and therefore the predicted address for the same salt. Pools deployed before a factory upgrade will then no longer be returned by `getPoolAddress`, breaking assumptions and gating logic that depends on it.

This design couples address derivation to the factory’s current implementation rather than the actual deployment history. In this codebase, callers use `getPoolAddress` to verify a pool is granted as a substrate before entering or modifying positions. After a factory upgrade, legitimate pools can be incorrectly rejected as unsupported. The function also does not verify that code exists at the predicted address, so if governance accidentally grants a predicted-but-nonexistent address, the gate could allow interactions that later fail unpredictably.

Listing 36. Code snippet from `contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamLib.sol`

```
49 function getPoolAddress(
```

```

50     address factory_,
51     address tokenA_,
52     address tokenB_,
53     int24 tickSpacing_
54 ) internal view returns (address pool) {
55     PoolKey memory key = _getPoolKey(tokenA_, tokenB_, tickSpacing_);
56     pool = _computeAddress(factory_, key);
57 }

```

The `getPoolAddress` call flows into `_computeAddress`, which uses `ICLFactory(factory_).poolImplementation()` to build the init code for prediction. This is the root cause of the mismatch after upgrades.

Exploit Scenario

A realistic failure occurs when governance rotates the pool implementation used by the factory.

Consider the following attack scenario:

1. Alice, acting as protocol governance, upgrades `poolImplementation()` on the factory to a new contract to roll out improvements.
2. Bob attempts to open a new position through the fuse for a known pool `(token0, token1, tickSpacing)` that already exists on-chain.
3. The fuse checks whether the pool is granted by calling `AreodromeSlipstreamSubstrateLib.getPoolAddress` and gating on the returned address.
4. Because `getPoolAddress` now predicts the address using the new implementation, it returns a different address than the one of the existing pool. The gate treats the pool as unsupported and the transaction reverts, preventing Bob from using a legitimate market.

Recommendation

Decouple pool discovery from the current implementation by querying the

factory for the actual deployed pool.

Recommended changes:

- Make `getPoolAddress` call `ICLFactory(factory_).getPool(token0, token1, tickSpacing)` and return that value. Do not compute a predicted address from `poolImplementation`.
- If the factory reports no pool (returns zero address), revert with an explicit error and avoid using any predicted address.
- As defense-in-depth, where external addresses are accepted or derived, assert `address(pool).code.length > 0` before proceeding.

[Go back to Findings Summary](#)

M16: Deterministic pool address prediction tied to factory’s mutable implementation breaks across upgrades, causing allowlist mismatches and denial-of-service

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamSubstrateLib.sol	Type:	Logic error

Description

The library derives a pool address at runtime using `Clones.predictDeterministicAddress` and the factory’s current `poolImplementation`. In CREATE2-based minimal proxy deployments, the predicted address includes the init code hash, which itself encodes the implementation address. After a factory upgrade changes `poolImplementation`, the same `(token0, token1, tickSpacing)` key produces a different predicted address than the already deployed pool, breaking determinism across upgrades.

This derived address is subsequently used for authorization gates. Both new-position and modify-position fuses compute the pool with `getPoolAddress` and check `PlasmaVaultConfigLib.isMarketSubstrateGranted` on that address. When the factory upgrades `poolImplementation`, these checks begin to fail for legitimate, previously granted pools (or worse, may pass for a different version), resulting in denial-of-service or misrouting.

Affected code surface:

- Library `VelodromeSuperchainSlipstreamSubstrateLib.getPoolAddress` and `_computeAddress`;
- Contract `VelodromeSuperchainSlipstreamNewPositionFuse.enter` substrate gate; and
- Contract `VelodromeSuperchainSlipstreamModifyPositionFuse.enter` and `exit` substrate gates.

Listing 37. Code snippet from `contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamSubstrateLib.sol`

```

89 function _computeAddress(
90     address factory_,
91     PoolKey memory key_
92 ) private view returns (address pool) {
93     if (key_.token0 >= key_.token1) revert TokensNotSorted();
94     pool = Clones.predictDeterministicAddress({
95         implementation: ICLFactory(factory_).poolImplementation(),
96         salt: keccak256(
97             abi.encode(key_.token0, key_.token1, key_.tickSpacing)
98         ),
99         deployer: factory_
100     });
101 }
```

The address computed above changes whenever `ICLFactory(factory_).poolImplementation()` changes, because the CREATE2 init code hash used by `Clones` depends on that implementation.

The computed address is used to build a substrate identifier for allowlist checks:

Listing 38. Code snippet from `contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamNewPositionFuse.sol`

```

131 if (
```

```

132     !PlasmaVaultConfigLib.isMarketSubstrateGranted(
133         MARKET_ID,
134         VelodromeSuperchainSlipstreamSubstrateLib.substrateToBytes32(
135             VelodromeSuperchainSlipstreamSubstrate({
136                 substrateType: VelodromeSuperchainSlipstreamSubstrateType
137                     .Pool,
138                 substrateAddress: VelodromeSuperchainSlipstreamSubstrateLib
139                     .getPoolAddress(
140                         FACTORY,
141                         data_.token0,
142                         data_.token1,
143                         data_.tickSpacing
144                     )
145             })
146         )
147     )
148 ) {
149     /// @dev this is to avoid stack too deep error
150     revert VelodromeSuperchainSlipstreamNewPositionFuseUnsupportedPool(
151         VelodromeSuperchainSlipstreamSubstrateLib.getPoolAddress(
152             FACTORY,
153             data_.token0,
154             data_.token1,
155             data_.tickSpacing
156         )
157     );
158 }

```

Technical explanation:

- With CREATE2, an address is `keccak256(0xff deployer salt ++ init_code_hash)[12:]`.
- For minimal proxies, `init_code_hash` depends on the implementation address embedded in the creation code.
- Changing `poolImplementation` after some pools are deployed makes historical addresses non-recoverable via the current implementation, so `getPoolAddress` diverges from the canonical, deployed pool for the same key.
- Because allowlists are stored as `bytes32(substrateType << 160 |`

`poolAddress`), the computed address no longer matches the granted substrate and authorization fails.

Impact in production:

- Legitimate calls that create, increase, or decrease liquidity for previously granted pools will revert after the upstream factory upgrades `poolImplementation`.
- Operators may mistakenly re-grant a new, different pool address for the same key, fragmenting liquidity across versions and potentially misrouting funds or accounting.
- No on-chain safeguard in the fuses verifies that code actually exists at the predicted address.

Exploit Scenario

An upstream change to the factory's `poolImplementation` is sufficient to trigger denial-of-service and potential misrouting for this market integration.

Consider the following attack scenario:

1. Alice, the factory maintainer, upgrades `poolImplementation` to a new version using the factory's own governance;
2. Bob, a regular user, calls the fuse `enter` to create a position in an already deployed and granted pool (`token0`, `token1`, `tickSpacing`);
3. The fuse computes `pool = getPoolAddress(FACTORY, token0, token1, tickSpacing)` using the new implementation and builds a substrate id from that address;
4. The allowlist check `isMarketSubstrateGranted(MARKET_ID, substrate)` fails because governance had granted the original pool address; and
5. Bob's transaction reverts, blocking all position management for that pool until governance is reconfigured.

A variation that can misroute interactions:

1. Alice proposes and executes allowlist updates that replace the old pool address with the newly predicted one for the same key;
2. Subsequent calls are authorized but now reference a different pool version than existing positions and accounting expect; and
3. Liquidity and rewards become fragmented across versions, increasing operational risk and causing unexpected outcomes.

Recommendation

Prefer resolving the pool address from a canonical registry that is independent of the current implementation, and avoid recomputing CREATE2 addresses with mutable inputs.

Actionable fixes:

- Replace `Clones.predictDeterministicAddress` with a call to a factory or registry getter that returns the canonical pool address for `(token0, token1, tickSpacing)` (for example, `factory.getPool(token0, token1, tickSpacing)`). Extend `ICLFactory` accordingly; and
- Treat the factory's `initCodeHash` as an immutable deployment-time constant rather than reading `poolImplementation` at runtime. If you must compute, derive `address = keccak256(0xff factory salt ++ initCodeHash)` using the exact `initCodeHash` that was used to deploy pools, not the current implementation.

Additional hardening:

- In fuses, verify that bytecode exists at the derived address `(extcodesize(pool) > 0)` and revert with a clear error if not. Do not authorize against a non-existent or unexpected target;
- For modify-position flows, avoid recomputation entirely: derive the pool

from a canonical mapping (e.g., `factory.getPool`), or embed the exact pool address in the substrate id granted by governance and compare against that value directly;

- If upgrades are expected, version the substrate id by including a `version` or the factory's `initCodeHash` in the substrate encoding to prevent silent cross-version mismatches; and
- Document and test a governance playbook: on factory upgrades, update allowlists atomically using canonical addresses before re-enabling user flows.

[Go back to Findings Summary](#)

M17: exit allows withdrawing non-whitelisted pool tokens back to the vault, bypassing asset policy

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	contracts/fuses/balancer/BalancerLiquidityUnbalancedFuse.sol	Type:	Logic error

Description

The `exit` function validates that the pool substrate is granted but never verifies that the tokens redeemed from the pool are granted as assets for `MARKET_ID`.

It approves the pool token (BPT) to the router and calls `removeLiquidityCustom`, which returns arbitrary underlying tokens as defined by the pool’s registration order. If some of those tokens are not asset-granted for the market, they will be transferred into the calling context (e.g., the `PlasmaVault` via `delegatecall`), violating the vault’s asset policy and undermining risk controls.

Listing 39. Code snippet from `contracts/fuses/balancer/BalancerLiquidityUnbalancedFuse.sol`

```
267     function exit(  
268         BalancerLiquidityUnbalancedFuseExitData memory data_  
269     )  
270     public  
271     payable  
272     returns (uint256 bptAmountIn, uint256[] memory amountsOut)  
273     {  
274         if (data_.pool == address(0)) {  
275             revert BalancerLiquidityUnbalancedFuseInvalidParams();  
276         }  
277     }
```

```

278         if (
279             !PlasmaVaultConfigLib.isMarketSubstrateGranted(
280                 MARKET_ID,
281                 BalancerSubstrateLib.substrateToBytes32(
282                     BalancerSubstrate({
283                         substrateType: BalancerSubstrateType.POOL,
284                         substrateAddress: data_.pool
285                     })
286                 )
287             ) {
288                 revert
289                 BalancerLiquidityUnbalancedFuseUnsupportedPool(data_.pool);
290             }
291
292             if (data_.maxBptAmountIn == 0) {
293                 amountsOut = new uint256[](0);
294                 return (0, amountsOut);
295             }
296
297             // Approve BPT (pool token) to router for burning
298             IERC20(data_.pool).forceApprove(BALANCER_ROUTER,
299                 data_.maxBptAmountIn);
300
301             (bptAmountIn, amountsOut, ) = IRouter(BALANCER_ROUTER)
302                 .removeLiquidityCustom(
303                     data_.pool,
304                     data_.maxBptAmountIn,
305                     data_.minAmountsOut,
306                     false,
307                     "");
308
309             emit BalancerLiquidityUnbalancedFuseExit(
310                 VERSION,
311                 data_.pool,
312                 bptAmountIn,
313                 amountsOut
314             );
315
316             IERC20(data_.pool).forceApprove(BALANCER_ROUTER, 0);
317         }

```


Exploit Scenario

A privileged caller can bring non-whitelisted assets into the vault by burning BPT from a whitelisted pool that contains such assets.

Consider the following attack scenario:

1. Alice controls the privileged executor that can call the fuse via `PlasmaVault.execute;`
2. Alice selects a pool that is substrate-whitelisted but that contains token `X` which is not asset-whitelisted for `MARKET_ID`;
3. Alice calls `exit` with `maxBptAmountIn > 0` and suitable `minAmountsOut`;
4. The fuse burns BPT and the router transfers underlying tokens back to the vault, including non-whitelisted token `X`, thereby bypassing the market's asset policy and potentially breaking accounting, pricing, or downstream integrations.

Recommendation

Before burning BPT, validate that all tokens registered in the pool are asset-granted for the market.

Fix the vulnerability by:

- Querying the pool's token list (e.g., via `IPool.getTokenInfo()` as used in `BalancerSubstrateLib.checkTokensInPool`) and, for each token, requiring `PlasmaVaultConfigLib.isSubstrateAsAssetGranted(MARKET_ID, token) == true`; otherwise, revert with `BalancerLiquidityUnbalancedFuseUnsupportedAsset(token)`.
- Only proceed to approve and call `removeLiquidityCustom` when every pool token passes the asset-whitelist check.
- Maintain the approval cleanup after the call.

This prevents `exit` from transferring non-whitelisted assets into the vault and preserves per-asset risk controls.

[Go back to Findings Summary](#)

M18: Unbounded iteration in balanceOf across substrates and positions can exhaust gas during market balance updates

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamBalanceFuse.sol	Type:	Denial of service

Description

The `balanceOf` function scans all granted substrates for a market and, for each substrate, iterates over every LP NFT owned or staked by the vault. These loops depend on `grantedSubstrates.length` and on arrays returned by external contracts, with no upper bound in the fuse logic. Each iteration triggers external view calls (`ISlipstreamSugar.principal`, and for pools also `ISlipstreamSugar.fees`) and then two USD conversions, making the overall cost scale linearly with the number of positions.

Because the vault calls `balanceOf` via `functionDelegateCall` as part of routine state-changing flows, an accumulation of many positions or many substrates can cause updates to run out of gas and revert.

Listing 40. Code snippet from `contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamBalanceFuse.sol`

```
101 for (uint256 i; i < len; i++) {
102     substrate = VelodromeSuperchainSlipstreamSubstrateLib
103         .bytes32ToSubstrate(grantedSubstrates[i]);
104     amount0 = 0;
105     amount1 = 0;
```

```

106
107     if (
108         substrate.substrateType ==
109         VelodromeSuperchainSlipstreamSubstrateType.Pool
110     ) {
111         tokenIds = INonfungiblePositionManager(
112             NONFUNGIBLE_POSITION_MANAGER
113         ).userPositions(address(this), substrate.substrateAddress);
114
115         uint256 tokenIdsLen = tokenIds.length;
116         token0 = ICLPool(substrate.substrateAddress).token0();
117         token1 = ICLPool(substrate.substrateAddress).token1();
118         sqrtPriceX96 = ICLPool(substrate.substrateAddress)
119             .slot0()
120             .sqrtPriceX96;
121
122         for (uint256 j; j < tokenIdsLen; j++) {
123             (amount0, amount1) = _addPrincipal(
124                 amount0,
125                 amount1,
126                 tokenIds[j],
127                 sqrtPriceX96
128             );
129             (amount0, amount1) = _addFees(
130                 amount0,
131                 amount1,
132                 tokenIds[j]
133             );
134         }
135
136         balance += _convertToUsd(
137             amount0,
138             token0,
139             priceOracleMiddleware
140         );
141         balance += _convertToUsd(
142             amount1,
143             token1,
144             priceOracleMiddleware
145         );
146     } else if (
147         substrate.substrateType ==
148         VelodromeSuperchainSlipstreamSubstrateType.Gauge
149     ) {
150         tokenIds = ILeafCLGauge(substrate.substrateAddress)
151             .stakedValues(address(this));

```

```

152     uint256 tokenIdsLen = tokenIds.length;
153     token0 = ILeafCLGauge(substrate.substrateAddress).token0();
154     token1 = ILeafCLGauge(substrate.substrateAddress).token1();
155     sqrtPriceX96 = ILeafCLGauge(substrate.substrateAddress)
156         .pool()
157         .slot0()
158         .sqrtPriceX96;
159
160     for (uint256 j; j < tokenIdsLen; j++) {
161         (amount0, amount1) = _addPrincipal(
162             amount0,
163             amount1,
164             tokenIds[j],
165             sqrtPriceX96
166         );
167     }
168
169     balance += _convertToUsd(
170         amount0,
171         token0,
172         priceOracleMiddleware
173     );
174     balance += _convertToUsd(
175         amount1,
176         token1,
177         priceOracleMiddleware
178     );
179 }
180 }

```

*Listing 41. Code snippet from
contracts/vaults/lib/PlasmaVaultMarketsLib.sol*

```

48 balanceFuse = FusesLib.getBalanceFuse(markets[i]);
49
50 wadBalanceAmountInUSD = abi.decode(
51     balanceFuse.functionDelegateCall(
52         abi.encodeWithSignature("balanceOf()")
53     ),
54     (uint256)
55 );

```

Exploit Scenario

An attacker can induce worst-case gas usage by proliferating many small positions that the vault later enumerates, causing balance updates to revert.

Consider the following attack scenario:

1. Alice repeatedly causes the system to create or accrue many small LP NFTs for a configured market (e.g., by triggering fuse flows that mint new positions) or transfers compatible NFTs to the vault if accepted.
2. Alice or any user then submits an action that triggers market balance updates.
3. The vault delegate calls the fuse `balanceOf`, which loops over all configured substrates and all token IDs returned for the vault, performing multiple external computations per token.
4. When enough positions exist, the transaction exceeds the gas budget and reverts, blocking updates and dependent operations until the number of positions is reduced or the configuration is changed.

Recommendation

Bound the amount of work performed by `balanceOf` and avoid reliance on unbounded external arrays.

Fix the vulnerability by either:

- Maintain an internal, bounded registry of vault-owned `tokenId`s` per substrate that is updated by fuse actions; iterate this registry instead of ``userPositions/stakedValues`.
- Add pagination parameters to the balance query and restructure callers to process balances in chunks rather than requiring a full scan in a single transaction.
- Add acceptance controls in `onERC721Received` (if the vault accepts NFTs) to

prevent unsolicited NFT donations that bloat scans.

- Introduce per-market caps on the number of positions the vault is allowed to hold and enforce consolidation of dust positions in operational flows.

[Go back to Findings Summary](#)

L1: Payable enter does not forward or refund ETH, causing stuck funds

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol	Type:	Logic error

Description

The function `enter` is declared `payable` but the implementation never forwards `msg.value` to the Balancer Router nor otherwise consumes or refunds it. The router call is made without `{value: msg.value}` and with `wethIsEth` set to `false`, so any ETH sent alongside the transaction is accepted by this contract and then stranded.

There is no `receive` or `fallback` function and no withdrawal mechanism in the contract, which makes any mistakenly attached ETH unrecoverable.

Listing 42. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (`payable` function signature)

```
136 function enter(  
137     BalancerLiquidityProportionalFuseEnterData memory data_  
138 ) public payable returns (uint256[] memory amountsIn) {  
139     if (data_.pool == address(0)) {  
140         revert BalancerLiquidityProportionalFuseInvalidParams();
```

Listing 43. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (router call without forwarding ETH)

```
176 amountsIn = IRouter(BALANCER_ROUTER).addLiquidityProportional(
```



```
177     data_.pool,  
178     data_.maxAmountsIn,  
179     data_.exactBptAmountOut,  
180     false,  
181     ""  
182 );
```

Technical evidence:

- No usages of `msg.value` in this file, and no `receive/fallback` handlers are defined for this contract.
- The router call explicitly sets `wethIsEth` to `false` and is not invoked with a value.

Exploit Scenario

A caller can inadvertently strand ETH by attaching it to the transaction.

Consider the following attack scenario:

1. Alice calls `enter` to add liquidity and mistakenly sends 1 ETH with the transaction;
2. The function accepts the payment because it is `payable` but never forwards `{value: msg.value}` and sets `wethIsEth` to `false` in the router call;
3. The ETH remains credited to this contract with no code path to refund or withdraw it; and
4. Bob, the protocol operator or integrator, cannot recover the 1 ETH, resulting in permanent loss of funds.

Recommendation

Make these functions non-payable unless ETH is intentionally supported.

Enforce explicit value handling to prevent stuck funds.

Fix the vulnerability by either:

- Remove `payable` and add `require(msg.value == 0)` to defend against accidental value; or
- If ETH support is intended, forward value to the router and enable ETH handling:
- Validate `msg.value` against the expected total ETH to be used;
- Call the router with `{value: msg.value}` and set `wethIsEth` to `true` when appropriate; and
- Revert on any unexpected leftover value (`require(msg.value == expected)`), or return unused ETH to the caller in the same transaction.

Additionally, consider adding a `receive()` that reverts to prevent accidental ETH transfers to the contract address.

[Go back to Findings Summary](#)

L2: Unchecked uint256→uint128 cast in exitTransient can silently truncate liquidity

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol	Type:	Overflow/Underflow

Description

In `exitTransient`, liquidity is read from transient storage as `bytes32`, converted to `uint256`, and then cast to `uint128` without bounds checking. In Solidity 0.8+, explicit narrowing casts do not revert on overflow; they truncate the upper bits. Values greater than `type(uint128).max` will be silently truncated, potentially producing a smaller-than-intended liquidity value.

Listing 44. Code snippet from
contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol

```
398         uint256 tokenId = TypeConversionLib.toUint256(tokenIdBytes32);
399         uint128 liquidity = uint128(
400             TypeConversionLib.toUint256(liquidityBytes32)
401         );
```

This can make the on-chain decrease use a different (smaller) liquidity than intended by the caller, causing confusing behavior or downstream accounting discrepancies when integrators rely on inputs rather than the value actually used.

Exploit Scenario

A large transient input can be truncated, changing the effective liquidity decrease without reverting.

Consider the following attack scenario:

1. Alice writes a very large 256-bit liquidity value to transient storage for the call to `exitTransient`;
2. The contract performs `uint128(liquidity256)`, silently truncating to the lower 128 bits;
3. `exit` proceeds with a smaller nonzero liquidity decrease than intended; and
4. Bob's off-chain systems or subsequent protocol logic mis-account based on the original (untruncated) input, leading to inconsistencies.

Recommendation

Validate the range before casting and provide a clear revert on overflow.

Fix the vulnerability by either:

- Reading the 256-bit value into a local `uint256`, requiring `val <= type(uint128).max`, then casting to `uint128`; or
- Using a dedicated safe conversion helper (e.g., `toUint128`) that reverts on overflow.

Additionally, keep the existing `liquidity > 0` check in `exit`, and consider validating upper bounds against the position's current liquidity for stronger guarantees.

[Go back to Findings Summary](#)

L3: Claimable fees ignored when current LP balance is zero

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/aerodrome/AerodromeBalanceFuse.sol	Type:	Logic error

Description

Function `balanceOf` suppresses fee valuation entirely when `liquidity == 0`. However, Aerodrome pools expose `claimable0/1(account)`, which can remain positive after LP tokens are withdrawn until explicitly claimed. Gating fee computation by `liquidity > 0` omits these already claimable amounts and underreports the vault’s market value.

The offending gate in `balanceOf` prevents calling fee computation when `liquidity` is zero:

Listing 45. Code snippet from `contracts/fuses/aerodrome/AerodromeBalanceFuse.sol`

```
102         if (liquidity > 0) {
103             balance += _calculateBalanceFromLiquidity(
104                 pool,
105                 priceOracleMiddleware,
106                 liquidity
107             );
108             balance += _calculateBalanceFromFees(
109                 pool,
110                 priceOracleMiddleware,
111                 liquidity
112             );
113         }
```

As a result, if the vault exited a position but has unclaimed fees recorded in pool accounting for its account (or for the Gauge account when using Gauges), these fees are not reflected in the reported USD balance until they are claimed. This creates inaccurate reporting around rebalances, migrations, or transient zero-liquidity states.

Impact in production:

- Understated market value whenever there are unclaimed fees but current LP balance is zero.
- Potentially incorrect limit checks, fee calculations, or NAV-dependent flows that trust the returned balance.

Exploit Scenario

By timing withdrawals to make **liquidity** zero, an attacker can temporarily hide accrued fees from the reported balance.

Consider the following attack scenario:

1. Alice provides liquidity and accrues trading fees attributed to her account.
2. Before a snapshot, gate check, or accounting call that reads **balanceOf**, Alice withdraws or migrates LP tokens so that **liquidity** becomes zero while unclaimed **claimable0/1** remain recorded.
3. The **balanceOf** call skips **pass:[_]calculateBalanceFromFees** and returns a lower USD balance than the true realizable amount.
4. Alice later re-deposits or simply claims the fees, realizing value that was not counted at snapshot time; Bob's protocol, which relied on the underreported balance for decisions, suffers from mispriced or mis-gated actions.

Recommendation

Decouple fee valuation from the `liquidity > 0` gate.

Recommended approach:

- Always call function `pass:[_]calculateBalanceFromFees` for each substrate to value `claimable0/1(account)`, even when `liquidity == 0`. The additional index-delta component will naturally contribute zero when `liquidity == 0`, but already claimable fees should still be included.
- Keep function `pass:[_]calculateBalanceFromLiquidity` gated by `liquidity > 0` to avoid division by zero and unnecessary work.
- If Gauges are used, ensure the correct `account` (Gauge vs vault) is used when reading per-account claimables in the pool so zero-liquidity periods still include pending fees for that account.

This change ensures the reported USD balance includes realizable fees regardless of the current LP token balance.

[Go back to Findings Summary](#)

L4: previewWithdraw divides by fee rate instead of net factor, causing severe overestimation of required shares

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/vaults/PlasmaVault.sol	Type:	Logic error

Description

The implementation of `previewWithdraw` applies the withdrawal fee incorrectly. When a withdraw fee is charged in shares (WAD precision), the number of shares required to withdraw a target `assets_` amount must be scaled by $1 / (1 - \text{fee})$ (i.e., add the proportional fee on top of the pre-fee shares).

Instead, the code multiplies the pre-fee shares by $1e18 / \text{fee}$, which is dimensionally wrong. For a typical fee of 1% ($1e16$ WAD), this inflates the result by ~100x rather than the intended ~1.0101x. For very small, non-zero fees, the returned value can become extremely large, making integrator balance/allowance pre-checks fail spuriously.

Affected elements:

- Function `previewWithdraw` in contract `PlasmaVault`;
- Variable `withdrawManager` and return value of `WithdrawManager.getWithdrawFee (WAD, 1e18 = 100%)`.

Correctness cross-check:

- `previewRedeem` correctly applies the net factor with `Math.mulDiv(shares_, 1e18 - withdrawFee, 1e18)` before delegating to the parent; `previewWithdraw` should use the algebraic inverse for shares, i.e. divide by

(1 - fee), not by fee.

Listing 46. Code snippet from contracts/vaults/PlasmaVault.sol

```
835 function previewWithdraw(  
836     uint256 assets_  
837 ) public view override returns (uint256) {  
838     address withdrawManager = PlasmaVaultStorageLib  
839         .getWithdrawManager()  
840         .manager;  
841  
842     if (withdrawManager != address(0)) {  
843         /// @dev get withdraw fee in shares with 18 decimals  
844         uint256 withdrawFee = WithdrawManager(withdrawManager)  
845             .getWithdrawFee();  
846  
847         if (withdrawFee > 0) {  
848             return  
849                 Math.mulDiv(  
850                     super.previewWithdraw(assets_),  
851                     1e18,  
852                     withdrawFee  
853                 );  
854         }  
855     }  
856     return super.previewWithdraw(assets_);  
857 }
```

Exploit Scenario

This bug is exploitable in practice through off-chain integrations that rely on `previewWithdraw` for quoting, pre-checks, or allowance sizing. Because the function overestimates required shares by orders of magnitude when a non-zero fee is configured, integrators and routers may either block valid user withdrawals or require excessive share approvals.

Consider the following attack scenario:

1. Alice, who controls fee configuration, enables a small withdrawal fee (for example, 1%).

2. Bob uses a third-party router that calls `previewWithdraw(assets)` to check if Bob has enough shares and to decide the allowance to request.
3. The router receives an overinflated share requirement (~100x for a 1% fee) and concludes Bob lacks sufficient shares, reverting or refusing to build the transaction.
4. Bob is unable to withdraw through that integrator, or is asked to approve/burn far more shares than necessary, degrading UX and potentially increasing approval risk.

Realistic impact:

- Denial-of-service in integrator paths that depend on `previewWithdraw` for balance/allowance validation once any non-zero fee is turned on.
- Excessive approvals requested from users, which may expand the blast radius in case of an approval misuse outside of this contract.
- Inconsistent quoting compared to `previewRedeem`, confusing end users and automated systems.

Recommendation

Replace the divisor with the net factor and align the logic with the `previewRedeem` approach:

- Compute the pre-fee shares with `super.previewWithdraw(assets_)`.
- Scale by $1e18 / (1e18 - \text{withdrawFee})$ (rounding up) to add the proportional fee on top of the pre-fee shares.

Concretely, change the calculation to:

- `Math.mulDiv(super.previewWithdraw(assets_), 1e18, 1e18 - withdrawFee).`

Additional hardening:

- If `withdrawFee == 1e18` (100%), withdrawing is effectively impossible; consider reverting in `previewWithdraw` for clarity instead of returning a finite share value.
- Add unit tests that assert realistic ratios, e.g., for a 1% fee, `previewWithdraw_with_fee ↪ previewWithdraw_without_fee * 1.0101...` within rounding tolerance, and that `previewRedeem` remains the algebraic inverse.
- Document fee semantics in NatSpec to prevent future regressions.

[Go back to Findings Summary](#)

L5: BalancerGaugeFuseEnter emits the requested amount, not the actual deposited amount, producing misleading telemetry

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/balancer/BalancerGaugeFuse.sol	Type:	Logging

Description

The `BalancerGaugeFuseEnter` event claims to log “the amount of BPT tokens that were deposited”, but the implementation emits `data_.bptAmount` (the requested amount) rather than `depositAmount` (the actual amount deposited). When the contract’s balance is below the requested amount, the function deposits only the available balance yet still emits the larger requested value, producing misleading telemetry for off-chain monitoring and accounting.

Listing 47. Code snippet from `contracts/fuses/balancer/BalancerGaugeFuse.sol` (actual amount calculation)

```
148     address lpToken = ILiquidityGauge(data_.gaugeAddress).lp_token();
149     uint256 balanceOfPlasmaVault = IERC20(lpToken).balanceOf(address
      (this));
150
151     depositAmount = data_.bptAmount > balanceOfPlasmaVault
152         ? balanceOfPlasmaVault
153         : data_.bptAmount;
154
155     if (depositAmount < data_.minBptAmount) {
156         revert BalancerGaugeFuseInsufficientBptAmount(
```

*Listing 48. Code snippet from
contracts/fuses/balancer/BalancerGaugeFuse.sol (event emission)*

```
170         emit BalancerGaugeFuseEnter(  
171             VERSION,  
172             data_.gaugeAddress,  
173             data_.bptAmount  
174         );
```

Exploit Scenario

An attacker or even a benign user can cause systematic over-reporting in logs.

Consider the following attack scenario:

1. Alice calls `enter` with `bptAmount = 1_000e18` when the contract's BPT balance is only `400e18`;
2. The function deposits `depositAmount = 400e18` (the available balance) but emits an event with `bptAmount = 1_000e18`;
3. Off-chain systems used by Bob for monitoring and accounting record a `1_000e18` deposit based on the event; and
4. Bob's dashboards and alerts misrepresent the position, potentially triggering incorrect operational decisions until the discrepancy is reconciled.

Recommendation

Emit the actual amount deposited in the event.

Fix the vulnerability by either:

- Changing the `BalancerGaugeFuseEnter` emission to pass `depositAmount` instead of `data_.bptAmount`; or

- If the input is desired for auditing, emit both values in distinct fields (e.g., `requestedBptAmount` and `depositedBptAmount`) with clear NatSpec.

Ensure the event's NatSpec and name remain aligned with the values they log.

[Go back to Findings Summary](#)

L6: Per-token ERC721 approvals to the gauge are not revoked after withdrawal (least-privilege violation)

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamLeafCLGaugeFuse.sol	Type:	Trust model

Description

During `enter`, the fuse sets a per-token approval on the `NonfungiblePositionManager` for `gaugeAddress`. After a subsequent `exit` (withdraw), the code does not explicitly revoke that approval.

Leaving approvals in place extends the authorization window unnecessarily. If a whitelisted gauge is later compromised or misconfigured, it could transfer or re-deposit the NFT without going through the fuse’s intended orchestration. While substrate whitelisting reduces the likelihood, avoiding stale approvals adheres to least privilege and reduces blast radius.

Affected code locations:

- Approval in `enter` via `INonfungiblePositionManager.approve(gaugeAddress, tokenId);`
- No corresponding approval revocation after `exit`.

*Listing 49. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlip
streamLeafCLGaugeFuse.sol*

```
139         INonfungiblePositionManager(ILeafCLGauge(data_.gaugeAddress).nft())  
140             .approve(data_.gaugeAddress, data_.tokenId);  
141  
142         ILeafCLGauge(data_.gaugeAddress).deposit(data_.tokenId);
```

Exploit Scenario

The risk materializes if the gauge becomes malicious or is upgraded with unintended behavior.

Consider the following attack scenario:

1. Alice compromises a previously whitelisted gauge or finds a misconfiguration giving it broader powers;
2. Alice triggers the gauge to call `transferFrom/deposit` on the `NonfungiblePositionManager` for a `tokenId` that the fuse contract has approved earlier;
3. The transfer succeeds due to the lingering approval, moving the NFT without invoking the fuse logic; and
4. Bob's vault loses control over position management ordering and observability, potentially leading to missed accounting updates or unintended restakes.

Recommendation

Revoke per-token approvals once they are no longer needed:

- After a successful `withdraw`, call `INonfungiblePositionManager.approve(address(0), tokenId)` to clear the approval; or

- Use a scoped approval pattern that sets approval immediately before a transfer and clears it immediately after, ensuring the approval window is minimal.

Additionally, consider defense-in-depth:

- Validate in `exit` that the position is staked by this contract in the expected `gaugeAddress` before proceeding; and
- If supported, prefer pull-less patterns where the fuse initiates transfers explicitly rather than relying on third-party pulls.

[Go back to Findings Summary](#)

L7: Payable exit does not forward or refund ETH, causing stuck funds

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol	Type:	Logic error

Description

The function `exit` is declared `payable` but never forwards `msg.value` to the router nor otherwise uses or refunds it. The proportional removal call uses `wethIsEth = false` and omits `{value: msg.value}`. Any ETH sent with `exit` is therefore accepted and stranded because the contract exposes no recovery mechanism.

Listing 50. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (`payable` function signature)

```
237 function exit(  
238     BalancerLiquidityProportionalFuseExitData memory data_  
239 ) public payable returns (uint256[] memory amountsOut) {  
240     if (data_.pool == address(0)) {  
241         revert BalancerLiquidityProportionalFuseInvalidParams();
```

Listing 51. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (router call without forwarding ETH)

```
268 amountsOut = IRouter(BALANCER_ROUTER).removeLiquidityProportional(  
269     data_.pool,  
270     data_.exactBptAmountIn,  
271     data_.minAmountsOut,
```

```
272     false,  
273     ""  
274 );
```

Technical evidence:

- No usages of `msg.value` in this file, and no `receive/fallback` exists.
- The `removeLiquidityProportional` call is not supplied with value and explicitly disables ETH handling via `wethIsEth = false`.

Exploit Scenario

A user can accidentally lock ETH when withdrawing liquidity.

Consider the following attack scenario:

1. Alice calls `exit` to remove liquidity and unintentionally includes 0.2 ETH in `msg.value`;
2. The function accepts the payment because it is `payable` but never forwards it to the router and leaves `wethIsEth` as `false`;
3. The ETH becomes trapped in the contract with no withdrawal function; and
4. Bob, who expected a normal withdrawal, finds the 0.2 ETH cannot be recovered, realizing a loss.

Recommendation

Eliminate acceptance of ETH unless the flow truly requires it.

Fix the vulnerability by either:

- Remove `payable` and assert `msg.value == 0`; or
- If ETH withdrawals are intended, forward `{value: msg.value}` and set `wethIsEth = true` in the router call, while validating that the supplied ETH

matches the expected amount and refunding any surplus within the same transaction.

Also include a reverting `receive()` to block stray transfers to the contract address.

[Go back to Findings Summary](#)

L8: `instantWithdraw` mis-sizes ERC-4626 withdrawals by using `convertToShares` (deposit path) instead of `previewWithdraw/previewRedeem`

Low impact finding

Impact:	Low	Confidence:	High
Target:	<code>./contracts/fuses/gearbox_v3/GearboxV3FarmSupplyFuse.sol</code>	Type:	Logic error

Description

The `instantWithdraw` function computes the number of `dToken` shares to unstake from the farm by calling `IERC4626.convertToShares(amount)` on the staking token returned by `IFarmingPool.stakingToken`.

Under EIP-4626, `convertToShares` is a pure, linear conversion intended for the deposit direction and must round down. It explicitly ignores withdrawal fees and the rounding direction required to achieve a target `assets` amount on withdrawal. For the withdrawal path, the correct functions are `previewWithdraw(assets)` or `previewRedeem(shares)`, which account for fees and round up as necessary to ensure enough shares are burned to obtain the requested `assets`.

Because `instantWithdraw` uses `convertToShares`, it can underestimate the shares required to unlock a target amount of underlying. In vaults with withdrawal fees or non-trivial rounding, the fuse may unstake fewer `dToken` shares than needed, causing the Plasma Vault to free less than the requested amount and potentially forcing liquidity to be sourced elsewhere or causing best-effort withdrawals to fall short.

*Listing 52. Code snippet from
contracts/fuses/gearbox_v3/GearboxV3FarmSupplyFuse.sol*

```
138         _exit(  
139             GearboxV3FarmdSupplyFuseExitData({  
140                 farmdToken: farmdToken,  
141                 /// @dev dToken 1:1 Farm dToken  
142                 dTokenAmount:  
143                     IERC4626(IFarmingPool(farmdToken).stakingToken())  
144                         .convertToShares(amount)  
145             })),  
146             true  
147         );
```

Exploit Scenario

A subtle but realistic scenario in which the sizing error results in a shortfall during withdrawals.

Consider the following attack scenario:

1. Alice deposits or redeems in the ERC-4626 **dToken** so that a withdrawal fee applies and the share-to-asset ratio causes rounding up on withdrawals.
2. Alice triggers **instantWithdraw** with **amount** equal to the target underlying assets the Plasma Vault should free.
3. The fuse computes **dTokenAmount** via **convertToShares(amount)**, which rounds down and ignores fees, and unstakes only that many shares from the farm.
4. When those **dToken** shares are later redeemed to underlying to satisfy the withdrawal, the resulting assets are less than **amount** due to fees/rounding, forcing Bob's protocol to fetch the remainder elsewhere or accept a shortfall.

Recommendation

Replace **convertToShares** with the ERC-4626 withdrawal-path previews so that sizing honors fees and rounding rules.

Fix the vulnerability by either:

- Using `IERC4626(stakingToken).previewWithdraw(amount)` to compute the number of shares that must be burned to obtain `amount` assets; or
- If working from shares, using `previewRedeem` appropriately, ensuring rounding up when targeting assets.

Additionally:

- Consider adding a small safety buffer to the computed shares to cover potential price movement between sizing and execution.
- Document the expectation that the staking token complies with ERC-4626 rounding semantics.

[Go back to Findings Summary](#)

L9: Payable enterTransient does not forward or refund ETH, causing stuck funds

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol	Type:	Logic error

Description

The function `enterTransient` is declared `payable` but never forwards `msg.value` to `enter(data)` nor consumes or refunds it. As a result, any ETH sent with this call is accepted and becomes stuck because the inner `enter` also does not handle ETH and the contract exposes no withdrawal path.

Listing 53. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (*payable function signature*)

```
200 function enterTransient() external payable {
201     bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
```

Listing 54. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (*internal call without forwarding ETH*)

```
225 uint256[] memory amountsIn = enter(data);
```

Technical evidence:

- No usages of `msg.value` exist within this function, and the callee `enter` does not process ETH either.

- The contract defines no `receive/fallback` nor any withdrawal function, so sent ETH cannot be recovered.

Exploit Scenario

A user or integrator can unintentionally send ETH when calling the transient entrypoint.

Consider the following attack scenario:

1. Alice calls `enterTransient` from an integration that automatically attaches ETH for gas stipend and mistakenly includes 0.5 ETH as `msg.value`;
2. The function is `payable`, accepts the ETH, and calls `enter(data)` without `{value: msg.value}`;
3. The ETH remains stuck in the contract with no refund path; and
4. Bob, operating the integration, discovers the 0.5 ETH cannot be withdrawn, resulting in a permanent loss.

Recommendation

Harden value handling for the transient entrypoint.

Fix the vulnerability by either:

- Remove `payable` and add `require(msg.value == 0)` to reject accidental value; or
- If ETH support is required, forward the value and enable ETH mode;
- Validate and forward `{value: msg.value}` to the underlying `enter` router call with `wethIsEth = true` when appropriate;
- Revert on mismatch between `msg.value` and the expected ETH requirement; and
- Prevent residual ETH by fully consuming or refunding unused value within

the transaction.

Also consider a reverting `receive()` function to block stray transfers.

[Go back to Findings Summary](#)

L10: Unrestricted enter allows unauthorized approvals and staking for vault-owned NFTs (griefing)

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamLeafCLGaugeFuse.sol	Type:	Griefing

Description

The `enter` function is `public` and enforces only that the gauge is whitelisted as a market substrate. It performs no caller authorization.

When called with a `tokenId` owned by this contract, `enter` approves the gauge to transfer the NFT via the `NonfungiblePositionManager` and then deposits it into the gauge. Any external caller can thus force staking of the vault’s NFT positions at arbitrary times, interfering with rebalancing, compounding cadence, or other higher-level strategy logic.

Affected function and external interaction:

- Function `enter` (state reads `MARKET_ID`, calls `INonfungiblePositionManager.approve` then `ILeafCLGauge.deposit`).

Listing 55. Code snippet from `contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamLeafCLGaugeFuse.sol`

```
103     function enter(  
104         VelodromeSuperchainSlipstreamLeafCLGaugeFuseEnterData memory data_
```

```

105     )
106     public
107     returns (
108         VelodromeSuperchainSlipstreamLeafCLGaugeFuseEnterResult memory
109         result
110     )
111     {
112         if (
113             !PlasmaVaultConfigLib.isMarketSubstrateGranted(
114                 MARKET_ID,
115                 VelodromeSuperchainSlipstreamSubstrateLib.substrateToBytes32(
116                     VelodromeSuperchainSlipstreamSubstrate({
117                         substrateType:
118                             VelodromeSuperchainSlipstreamSubstrateType
119                             .Gauge,
120                             substrateAddress: data_.gaugeAddress
121                         })
122                     )
123                 ) {
124                     revert VelodromeSuperchainSlipstreamLeafCLGaugeUnsupportedGauge(
125                         data_.gaugeAddress
126                     );
127                 }
128
129                 result.gaugeAddress = data_.gaugeAddress;
130                 result.tokenId = data_.tokenId;
131
132                 if (data_.tokenId == 0) {
133                     emit VelodromeSuperchainSlipstreamLeafCLGaugeEnter(
134                         result.gaugeAddress,
135                         result.tokenId
136                     );
137                     return result;
138                 }
139
140                 INonfungiblePositionManager(ILeafCLGauge(data_.gaugeAddress).nft())
141                     .approve(data_.gaugeAddress, data_.tokenId);
142
143                 ILeafCLGauge(data_.gaugeAddress).deposit(data_.tokenId);
144
145                 emit VelodromeSuperchainSlipstreamLeafCLGaugeEnter(
146                     result.gaugeAddress,
147                     result.tokenId
148                 );

```

Exploit Scenario

An unprivileged caller can change staking state as long as the NFT is held by the fuse contract.

Consider the following attack scenario:

1. Alice monitors the vault and identifies an NFT `tokenId` currently owned by the fuse contract and a corresponding whitelisted gauge;
2. Alice calls `enter({gaugeAddress, tokenId})` directly on the fuse contract;
3. The fuse approves the gauge on the `NonfungiblePositionManager` and deposits the NFT into the gauge; and
4. Bob's strategy timing is disrupted (e.g., deposits occur before fee collection or across rebalance windows), reducing realized yield or complicating accounting.

Recommendation

Restrict `enter` to trusted callers and add invariant checks:

- Gate `enter` with an `onlyAuthorized` modifier limiting access to the vault, governance, or a designated router;
- Before approving, check that `tokenId` is actually owned by this contract and not already staked in `gaugeAddress`;
- Consider implementing time/role-based orchestration so deposits occur only when intended (e.g., post-fee collection); and
- Log access-denied attempts for monitoring.

[Go back to Findings Summary](#)

L11: Payable exitTransient does not forward or refund ETH, causing stuck funds

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol	Type:	Logic error

Description

The function `exitTransient` is declared `payable` but does not forward `msg.value` to `exit(data)` nor otherwise consume or refund it. The callee `exit` also ignores ETH and the contract lacks any withdrawal or refund logic. Any ETH sent to this entrypoint will therefore be stranded.

Listing 56. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (*payable function signature*)

```
288 function exitTransient() external payable {
289     bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
```

Listing 57. Code snippet from `contracts/fuses/balancer/BalancerLiquidityProportionalFuse.sol` (*internal call without forwarding ETH*)

```
307 uint256[] memory amountsOut = exit(data);
```

Technical evidence:

- No references to `msg.value` exist within this function and the called `exit` also does not forward ETH to the router.

- The contract has no `receive/fallback` handler and no withdrawal path.

Exploit Scenario

An integration may inadvertently attach value to the transient exit call.

Consider the following attack scenario:

1. Alice invokes `exitTransient` from an adapter that mistakenly attaches 0.05 ETH as a value parameter;
2. The function is `payable` and accepts the ETH, then calls `exit(data)` without `{value: msg.value}`;
3. The ETH remains stuck in the contract with no refund mechanism; and
4. Bob, the integrator, cannot recover the 0.05 ETH and must account for the loss.

Recommendation

Prevent acceptance of ETH on transient flows unless explicitly handled.

Fix the vulnerability by either:

- Remove `payable` and revert on `msg.value > 0`; or
- If ETH should be supported, validate and forward `{value: msg.value}` downstream with `wethIsEth = true` where applicable, and ensure that unused ETH is refunded or the transaction reverts.

Implement a reverting `receive()` to block accidental ETH transfers to the contract.

[Go back to Findings Summary](#)

L12: Inconsistent ETH handling: payable enter ignores msg.value; exit forwards unintended ETH with wethIsEth=false

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	./contracts/fuses/balancer/BalancerSingleTokenFuse.sol	Type:	Logic error

Description

The contract declares `enter` and `exit` as `payable`, but ETH is not consistently or intentionally handled. This creates two classes of issues:

- `enter` accepts ETH yet never reads `msg.value` nor forwards ETH to the Balancer router. Any ETH sent to `enter` becomes trapped in the contract balance because there is no withdrawal path.
- `exit` accepts ETH and forwards `msg.value` to the external router call while hard-coding `wethIsEth=false`. If a caller mistakenly sends ETH, that value is still propagated to the router despite ETH being disabled at the parameter level. Depending on the router implementation, this may cause reverts, unaccounted ETH retention, or other undefined behavior.

The router interface confirms both relevant methods are `payable`, so the forwarded value will be accepted at the ABI level. However, because `wethIsEth` is always `false`, ETH should not be required by design.

Listing 58. Code snippet from `contracts/fuses/balancer/BalancerSingleTokenFuse.sol`

```
154     function enter(  
155         BalancerSingleTokenFuseEnterData memory data_
```



```

156     ) public payable returns (uint256 amountIn) {
157         if (data_.pool == address(0) || data_.tokenIn == address(0)) {
158             revert BalancerSingleTokenFuseInvalidParams();
159         }
160
161         if (
162             !PlasmaVaultConfigLib.isMarketSubstrateGranted(
163                 MARKET_ID,
164                 BalancerSubstrateLib.substrateToBytes32(
165                     BalancerSubstrate({
166                         substrateType: BalancerSubstrateType.POOL,
167                         substrateAddress: data_.pool
168                     })
169                 )
170             )
171         ) {
172             revert BalancerSingleTokenFuseUnsupportedPool(data_.pool);
173         }
174
175         if (data_.maxAmountIn == 0) {
176             return 0;
177         }
178
179         address[] memory tokens = new address[](1);
180         tokens[0] = data_.tokenIn;
181
182         BalancerSubstrateLib.checkTokensInPool(data_.pool, tokens);
183
184         IERC20(data_.tokenIn).forceApprove(PERMIT2, data_.maxAmountIn);
185         IPermit2(PERMIT2).approve(
186             data_.tokenIn,
187             BALANCER_ROUTER,
188             data_.maxAmountIn.toUint160(),
189             uint48(block.timestamp + 1)
190         );
191
192         amountIn = IRouter(BALANCER_ROUTER).addLiquiditySingleTokenExactOut(
193             data_.pool,
194             IERC20(data_.tokenIn),
195             data_.maxAmountIn,
196             data_.exactBptAmountOut,
197             false,
198             ""
199         );

```

In `enter`, the call to `addLiquiditySingleTokenExactOut` passes `wethIsEth=false` and does not forward value with `{value: ...}`. Any ETH sent remains in this contract, which has no sweep or withdrawal mechanism.

*Listing 59. Code snippet from
contracts/fuses/balancer/BalancerSingleTokenFuse.sol*

```
238     function exit(  
239         BalancerSingleTokenFuseExitData memory data_  
240     ) public payable returns (uint256 bptAmountIn) {  
241         if (data_.pool == address(0) || data_.tokenOut == address(0)) {  
242             revert BalancerSingleTokenFuseInvalidParams();  
243         }  
244  
245         if (  
246             !PlasmaVaultConfigLib.isMarketSubstrateGranted(  
247                 MARKET_ID,  
248                 BalancerSubstrateLib.substrateToBytes32(  
249                     BalancerSubstrate({  
250                         substrateType: BalancerSubstrateType.POOL,  
251                         substrateAddress: data_.pool  
252                     })  
253                 )  
254             )  
255         ) {  
256             revert BalancerSingleTokenFuseUnsupportedPool(data_.pool);  
257         }  
258  
259         if (data_.maxBptAmountIn == 0) {  
260             return 0;  
261         }  
262  
263         IERC20(data_.pool).forceApprove(BALANCER_ROUTER,  
data_.maxBptAmountIn);  
264  
265         bptAmountIn = IRouter(BALANCER_ROUTER)  
266             .removeLiquiditySingleTokenExactOut{value: msg.value}(  
267             data_.pool,  
268             data_.maxBptAmountIn,  
269             IERC20(data_.tokenOut),  
270             data_.exactAmountOut,  
271             false,  
272             ""
```

```
273         );
```

In `exit`, `msg.value` is forwarded to the router even though `wethIsEth=false` explicitly disables ETH handling. The router signatures are indeed `payable`:

Listing 60. Code snippet from `contracts/fuses/balancer/ext/IRouter.sol`

```
58     function addLiquiditySingleTokenExactOut(  
59         address pool,  
60         IERC20 tokenIn,  
61         uint256 maxAmountIn,  
62         uint256 exactBptAmountOut,  
63         bool wethIsEth,  
64         bytes memory userData  
65     ) external payable returns (uint256 amountIn);
```

Listing 61. Code snippet from `contracts/fuses/balancer/ext/IRouter.sol`

```
161    function removeLiquiditySingleTokenExactOut(  
162        address pool,  
163        uint256 maxBptAmountIn,  
164        IERC20 tokenOut,  
165        uint256 exactAmountOut,  
166        bool wethIsEth,  
167        bytes memory userData  
168    ) external payable returns (uint256 bptAmountIn);
```

Additionally, `enterTransient` and `exitTransient` are both `payable` and simply delegate to `enter` and `exit` respectively, inheriting the same inconsistencies. In practice, typical integrations will call with zero ETH, which avoids immediate loss, but the current surface invites mistakes and complicates integration assumptions about value handling.

Exploit Scenario

Two realistic mis-use scenarios demonstrate impact even without a direct theft vector.

Consider the following attack scenario:

1. Alice interacts directly with the fuse and calls `enter` with a non-zero `msg.value` by mistake.
2. The contract accepts ETH due to `payable` but never consumes or forwards it.
3. The call succeeds using only ERC20 transfers; `msg.value` remains in the fuse contract balance.
4. Bob, operating the protocol, has no withdrawal path in the fuse and cannot recover Alice's ETH without a governance-level migration.

Consider another attack scenario:

1. Alice (or an integration) mistakenly calls `exit` with a non-zero `msg.value` while requesting a token-out path that is not ETH, and `wethIsEth` is hard-coded to `false`.
2. The contract forwards `msg.value` to the router call `{value: msg.value}` even though ETH should not be used.
3. The router function is `payable` and may accept the value; depending on its internal logic, ETH can be retained, ignored, or cause a revert.
4. Bob's protocol experiences unpredictable behavior: either silent ETH leakage to the router, integration breakage via revert, or operational overhead to reconcile stray value.

Recommendation

Make ETH handling explicit and consistent, and fail fast on unexpected value.

Recommended changes:

- Remove `payable` from `enter`, `enterTransient`, `exit`, and `exitTransient` unless native ETH is a supported flow.
- If native ETH is not supported, add `require(msg.value == 0, "No ETH accepted")` at the top of each function as a defensive assertion.

- If native ETH is intended for specific flows, introduce an explicit flag in input data (for example, `useEth`) and wire it to the router's `wethIsEth` argument. Only forward `{value: msg.value}` when `useEth` is true and validate the corresponding token is WETH on the correct side of the trade.
- In `exit`, remove `{value: msg.value}` when `wethIsEth=false`, or revert on `msg.value > 0` to prevent accidental value forwarding.
- Optionally, add a governance-controlled `sweepNative(address to)` function to recover any stray ETH that may have been sent before the fix. Ensure it cannot sweep user funds owed to the vault by mistake and document its intended use.

These changes eliminate silent ETH trapping in `enter` and unintended value forwarding in `exit`, making integration behavior predictable and safe.

[Go back to Findings Summary](#)

L13: Payable entry points can lock ETH because `msg.value` is unused and there is no recovery path

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	contracts/fuses/balancer/ BalancerGaugeFuse.sol	Type:	Code quality

Description

The functions `enter`, `enterTransient`, `exit`, and `exitTransient` are declared `payable` but never read or use `msg.value`, and the contract exposes no `receive` /`fallback` or administrative withdrawal for native ETH. Any ETH sent alongside these calls becomes permanently locked in the contract, creating avoidable value loss risk for integrators who mistakenly attach ETH.

*Listing 62. Code snippet from
contracts/fuses/balancer/BalancerGaugeFuse.sol (`enter`)*

```
128     function enter(  
129         BalancerGaugeFuseEnterData memory data_  
130     ) public payable returns (uint256 depositAmount) {
```

*Listing 63. Code snippet from
contracts/fuses/balancer/BalancerGaugeFuse.sol (`enterTransient`)*

```
179     function enterTransient() external payable {  
180         bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
```

*Listing 64. Code snippet from
contracts/fuses/balancer/BalancerGaugeFuse.sol (`exit`)*

```
201     function exit(  
202         BalancerGaugeFuseExitData memory data_
```

```
203     ) public payable returns (uint256 withdrawAmount) {
```

Listing 65. Code snippet from

contracts/fuses/balancer/BalancerGaugeFuse.sol (exitTransient)

```
245     function exitTransient() external payable {  
246         bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
```

Exploit Scenario

A user can inadvertently burn ETH by attaching it to calls that ignore `msg.value`.

Consider the following attack scenario:

1. Alice integrates the fuse and mistakenly sends **0.5 ETH** with a call to `enter`;
2. The contract does not use `msg.value`, does not emit it, and cannot transfer it out later; and
3. Bob, who operates the vault, cannot recover the ETH via this contract and must rely on out-of-band rescue mechanisms (if any), resulting in locked value in production.

Recommendation

Avoid locking native ETH by removing `payable` where not needed or adding controlled recovery.

Fix the vulnerability by either:

- Removing `payable` from `enter`, `enterTransient`, `exit`, and `exitTransient` since they do not use `msg.value`; or
- Implementing a controlled native-asset sweep (e.g., an `onlyOwner` or `onlyVault recoverETH(address to)` function) and a `receive()/fallback()` to handle stray ETH.

Favor removal of `payable` to reduce accidental misuse unless a future use of `msg.value` is planned.

[Go back to Findings Summary](#)

L14: Mint called with unsorted token pair after sorted pool check causes avoidable reverts

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamNewPositionFuse.sol	Type:	Data validation

Description

The `enter` function validates that the target pool is a granted substrate using `getPoolAddress`, which sorts the token pair internally before computing the pool address. Immediately after, it calls `INonfungiblePositionManager.mint` with the raw, user-supplied `token0` and `token1` without enforcing the same ordering. In concentrated-liquidity managers, `mint` typically requires `token0 < token1`; passing an unsorted pair reverts.

This mismatch allows calls that pass the substrate check to later revert in `mint`, wasting gas and breaking integrator assumptions. The code path below shows the substrate check using a sorted pair, followed by the `mint` call that forwards unsorted inputs.

Listing 66. Code snippet from `contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamNewPositionFuse.sol`

```
131         if (
132             !PlasmaVaultConfigLib.isMarketSubstrateGranted(
133                 MARKET_ID,
134                 VelodromeSuperchainSlipstreamSubstrateLib.substrateToBytes32(
```

```

135         VelodromeSuperchainSlipstreamSubstrate({
136             substrateType:
137             VelodromeSuperchainSlipstreamSubstrateType
138             .Pool,
139             substrateAddress:
140             VelodromeSuperchainSlipstreamSubstrateLib
141             .getPoolAddress(
142                 FACTORY,
143                 data_.token0,
144                 data_.token1,
145                 data_.tickSpacing
146             )
147         })
148     ) {
149         /// @dev this is to avoid stack too deep error
150         revert
151         VelodromeSuperchainSlipstreamNewPositionFuseUnsupportedPool(
152             VelodromeSuperchainSlipstreamSubstrateLib.getPoolAddress(
153                 FACTORY,
154                 data_.token0,
155                 data_.token1,
156                 data_.tickSpacing
157             )
158         );
159     }

```

The subsequent `mint` forwards `data_.token0` and `data_.token1` directly, without sorting or validating the order.

*Listing 67. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamNewPositionFuse.sol*

```

174         ) = INonfungiblePositionManager(NONFUNGIBLE_POSITION_MANAGER).mint(
175             INonfungiblePositionManager.MintParams({
176                 token0: data_.token0,
177                 token1: data_.token1,
178                 tickSpacing: data_.tickSpacing,
179                 tickLower: data_.tickLower,
180                 tickUpper: data_.tickUpper,
181                 amount0Desired: data_.amount0Desired,

```

```

182         amount1Desired: data_.amount1Desired,
183         amount0Min: data_.amount0Min,
184         amount1Min: data_.amount1Min,
185         recipient: address(this),
186         deadline: data_.deadline,
187         sqrtPriceX96: data_.sqrtPriceX96
188     })
189 );

```

The substrate library explicitly sorts tokens and reverts if order is invalid when computing the pool address, confirming the ordering expectation.

*Listing 68. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamSubstrateLib.sol*

```

71     function _getPoolKey(
72         address tokenA_,
73         address tokenB_,
74         int24 tickSpacing_
75     ) private pure returns (PoolKey memory) {
76         if (tokenA_ > tokenB_) (tokenA_, tokenB_) = (tokenB_, tokenA_);
77         return
78             PoolKey({
79                 token0: tokenA_,
80                 token1: tokenB_,
81                 tickSpacing: tickSpacing_
82             });
83     }

```

*Listing 69. Code snippet from
contracts/fuses/velodrome_superchain_slipstream/VelodromeSuperchainSlipstreamSubstrateLib.sol*

```

89     function _computeAddress(
90         address factory_,
91         PoolKey memory key_
92     ) private view returns (address pool) {
93         if (key_.token0 >= key_.token1) revert TokensNotSorted();
94         pool = Clones.predictDeterministicAddress({
95             implementation: ICLFactory(factory_).poolImplementation(),

```

```

96         salt: keccak256(
97             abi.encode(key_.token0, key_.token1, key_.tickSpacing)
98         ),
99         deployer: factory_
100     });
101 }

```

As a result, a caller supplying `token0` and `token1` in reverse order can pass the substrate check (since it sorts internally) but still hit a revert in `mint`. In production, this manifests as avoidable reverts and gas loss for users or integrators mistakenly providing unsorted inputs.

Exploit Scenario

A user can grief themselves or upstream integrators by calling `enter` with an unsorted token pair. The substrate check succeeds because it sorts the pair internally, but the subsequent `mint` reverts on order mismatch. The end result is wasted gas and unpredictability for routing systems that do not pre-normalize token order.

Consider the following attack scenario:

1. Alice prepares inputs for a permitted pool but passes `token0` and `token1` in reverse (i.e., `token0 > token1` by address);
2. Alice calls `enter` with these unsorted addresses and valid amounts/ticks;
3. The substrate check succeeds because `getPoolAddress` sorts the pair and finds the granted pool;
4. The contract approves tokens and forwards the unsorted pair to `NonfungiblePositionManager.mint`;
5. The `mint` call reverts due to invalid token ordering constraints; and
6. Bob, operating an aggregator that routed Alice's order, eats the gas cost and experiences degraded reliability.

Recommendation

Enforce a consistent token ordering invariant before calling `mint`, aligned with how the substrate is validated.

Fix the vulnerability by either:

- Validating input order and reverting early with a clear error if unsorted; or
- Sorting the pair locally and adjusting the amounts accordingly before `mint`.

Practical options:

- Add a precondition in `enter`: `require(data_.token0 < data_.token1, "TokensNotSorted");`. This gives immediate, deterministic feedback to integrators.
- Alternatively, normalize inputs before use:
- If `data_.token0 > data_.token1`, swap `token0/token1` locally;
- Also swap `amount0Desired` with `amount1Desired`, and `amount0Min` with `amount1Min` to keep semantics consistent; and
- If `sqrtPriceX96` is used to initialize a new pool, invert it when swapping tokens to preserve the intended price (otherwise set it to 0 for existing pools).

In all cases, use the same (sorted) token order for both the substrate check and the `mint` call to eliminate this class of avoidable reverts and align behavior across code paths.

[Go back to Findings Summary](#)

W1: Named return variable `finalAmount` is shadowed in `exit`, risking unintended returns after refactors

Impact:	Warning	Confidence:	High
Target:	contracts/fuses/chains/et hereum/lido/StEthWrapp e rFuse.sol	Type:	Code quality

Description

Function `exit` declares named return variables (`finalAmount`, `stEthAmount`) but then redeclares a new local variable `uint256 finalAmount` inside the function body. The local declaration shadows the named return variable, which can mislead maintainers and tooling and makes future refactors prone to subtle bugs if implicit `return;` is ever used or assignments are made under the assumption that they target the named return variable.

This pattern is risky in flows that orchestrate unwrapping and asset accounting. A mismatch between the value used for external calls and the value returned to callers can cascade into incorrect balance updates or bypassed limits in integrating systems that rely on the function’s return values.

*Listing 70. Code snippet from
contracts/fuses/chains/ethereum/lido/StEthWrapperFuse.sol*

```
116    /// @notice Exits the strategy by unwrapping wstETH to stETH
117    /// @param wstEthAmount Amount of wstETH to unwrap
118    /// @return finalAmount Amount of wstETH actually unwrapped
119    /// @return stEthAmount Amount of stETH received
120    function exit(
121        uint256 wstEthAmount
122    ) public returns (uint256 finalAmount, uint256 stEthAmount) {
123        if (wstEthAmount == 0) {
124            return (0, 0);
```

```

125     }
126     _validateSubstrates("exit");
127
128     uint256 finalAmount = IporMath.min(
129         ERC20(WST_ETH).balanceOf(address(this)),
130         wstEthAmount
131     );
132
133     if (finalAmount == 0) {
134         return (0, 0);
135     }
136
137     stEthAmount = IWstETH(WST_ETH).unwrap(finalAmount);
138
139     emit StEthWrapperFuseExit(VERSION, finalAmount, stEthAmount);
140
141     return (finalAmount, stEthAmount);

```

Exploit Scenario

If a future change removes the explicit tuple `return (finalAmount, stEthAmount);` and relies on implicit `return;` while keeping the local `uint256 finalAmount`, the named return variable `finalAmount` would not be updated. The function would still unwrap based on the local value but return zero for `finalAmount`. Downstream consumers could make incorrect accounting or authorization decisions.

Consider the following attack scenario:

1. Alice calls `exit` with a non-zero `wstEthAmount` after a refactor introduced an implicit `return;` while keeping the shadowed local `finalAmount`;
2. Alice receives the correct `stEthAmount` from the actual local `finalAmount` (unwrapping succeeded);
3. The integrating system reads the returned `finalAmount` as zero and incorrectly interprets that no `wstETH` was unwrapped, potentially skipping balance and limit checks; and
4. Bob's protocol suffers financial loss due to inconsistent accounting,

enabling Alice to withdraw more than permitted or bypass fees/limits.

Recommendation

Eliminate the shadowing to align the returned data with the values used for external calls.

Fix the vulnerability by either:

- Rename the local `finalAmount` to a distinct identifier (for example, `amountToUnwrap`) and continue using explicit tuple returns; or
- Drop the names from the return variables and only return explicit local variables; or
- Keep named returns and stop redeclaring `finalAmount` locally, assigning directly to the named return variable.

Also enforce a lint rule that forbids variable shadowing for state- and accounting-related identifiers.

[Go back to Findings Summary](#)

W2: Named return variable `finalAmount` is shadowed in `enter`, risking unintended returns after refactors

Impact:	Warning	Confidence:	High
Target:	contracts/fuses/chains/ethereum/lido/StEthWrapperFuse.sol	Type:	Code quality

Description

Function `enter` declares named return variables (`finalAmount`, `wstEthAmount`) but later redeclares a new local variable `uint256 finalAmount` inside the function body. The local declaration shadows the named return variable, creating ambiguous semantics and increasing the risk of future logic bugs if the function is refactored to rely on the named return variables or implicit `return`;

Shadowing named return variables is a known footgun. If future maintainers assume that assignments to `finalAmount` affect the named return variable, while in reality a different local identifier is being written, the function may return an unintended value (e.g., zero or a stale value). This is particularly dangerous in wrapper flows that interact with external tokens, approvals, and integrations that rely on accurate accounting of how much asset was actually wrapped.

Listing 71. Code snippet from `contracts/fuses/chains/ethereum/lido/StEthWrapperFuse.sol`

```
71    /// @notice Enters the strategy by wrapping stETH to wstETH
72    /// @param stEthAmount Amount of stETH to wrap
73    /// @return finalAmount Amount of stETH actually wrapped
74    /// @return wstEthAmount Amount of wstETH received
75    function enter(
76        uint256 stEthAmount
```

```

77     ) public returns (uint256 finalAmount, uint256 wstEthAmount) {
78         if (stEthAmount == 0) {
79             return (0, 0);
80         }
81         _validateSubstrates("enter");
82
83         uint256 finalAmount = IporMath.min(
84             ERC20(ST_ETH).balanceOf(address(this)),
85             stEthAmount
86         );
87
88         if (finalAmount == 0) {
89             return (0, 0);
90         }
91
92         ERC20(ST_ETH).forceApprove(address(WST_ETH), finalAmount);
93
94         wstEthAmount = IWstETH(WST_ETH).wrap(finalAmount);
95
96         ERC20(ST_ETH).forceApprove(address(WST_ETH), 0);
97
98         emit StEthWrapperFuseEnter(VERSION, finalAmount, wstEthAmount);
99
100        return (finalAmount, wstEthAmount);
101    }

```

Exploit Scenario

The shadowed variable makes the function fragile to refactors. If a future change replaces the explicit tuple `return (finalAmount, wstEthAmount);` with an implicit `return;` and keeps the local `uint256 finalAmount` declaration, the named return variable `finalAmount` would remain uninitialized and be returned as zero while the wrap operation still executes with the local value. Integrated protocols that rely on the returned `finalAmount` for accounting could miscredit balances or bypass limits.

Consider the following attack scenario:

1. Alice calls `enter` with a non-zero `stEthAmount` after a refactor introduced an implicit `return;` while keeping the shadowed local `finalAmount;`

2. Alice receives `wstEthAmount` minted from the actual local `finalAmount` (wrapping succeeded);
3. The integrating system reads the returned `finalAmount` as zero and credits Alice as if no stETH was consumed; and
4. Bob's protocol becomes insolvent because Alice received wstETH without the corresponding debit of stETH, enabling value extraction.

Recommendation

Remove the shadowing to make the intent explicit and future-proof.

Fix the vulnerability by either:

- Renaming the local variable to avoid shadowing (for example, use `amountToWrap`) and keep explicit returns; or
- Removing the names from the return variables in the function signature and only use local variables in explicit returns; or
- Keeping named returns, but stop redeclaring `finalAmount` locally and assign directly to the named return variable.

Additionally, enable or enforce linters that forbid variable shadowing in critical paths to prevent regressions.

[Go back to Findings Summary](#)

I1: Asset allowlist not enforced; error AreodromeSlipstreamModifyPositionFuseUnsupportedAsset is unused

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol	Type:	Unused code

Description

The contract declares `AreodromeSlipstreamModifyPositionFuseUnsupportedAsset(address)` but never reverts with it and does not enforce any asset-level allowlist in `enter/enterTransient`. This contradicts the contract documentation that claims token assets are validated.

Listing 72. Code snippet from contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol

```
90    /// @notice Thrown when attempting to use a token that is not granted as
    /// an asset
91    /// @param asset The address of the token that is not supported
92    error AreodromeSlipstreamModifyPositionFuseUnsupportedAsset(address
    asset);
```

The header comments suggest both assets and pools are validated, but only pool validation is implemented via `validatePool`.

Listing 73. Code snippet from
contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamModifyPositionFuse.sol

```
50 /// @title AreodromeSlipstreamModifyPositionFuse
51 /// @notice Fuse for modifying (increasing or decreasing) liquidity in
    existing Aerodrome Slipstream NFT positions
52 /// @dev This fuse allows users to add or remove liquidity from existing NFT
    positions in Aerodrome Slipstream pools.
53 ///      It validates that tokens are granted assets and that the pool is
    granted as a substrate.
54 ///      Supports both standard function calls and transient storage-based
    calls.
55 /// @author IPOR Labs
```

Wake and repository searches show no references to the error (no `revert AreodromeSlipstreamModifyPositionFuseUnsupportedAsset`). As a result, unsupported assets are not prevented at this layer, which can lead to policy violations or integrator confusion.

Exploit Scenario

The missing asset validation allows operations to proceed with assets that are not granted by governance, despite documentation implying enforcement.

Consider the following attack scenario:

1. Alice passes asset addresses that are not granted as market assets when calling `enter`;
2. The contract performs no asset-level checks and proceeds to approve and call `increaseLiquidity` on the `NonfungiblePositionManager`;
3. The operation succeeds (subject to the position's actual tokens), violating intended asset policy; and
4. Bob's governance and risk controls are bypassed at the fuse layer, creating compliance and risk-management issues.

Recommendation

Enforce asset allowlists and use the custom error.

Fix the vulnerability by either:

- Validating the actual position tokens (read via `INonfungiblePositionManager.positions(tokenId)`) against the market's asset grants using `PlasmaVaultConfigLib.isSubstrateAsAssetGranted(MARKET_ID, token)`. Revert with `AreodromeSlipstreamModifyPositionFuseUnsupportedAsset(token)` if not granted; or
- At minimum, validate the provided `data_.token0` and `data_.token1` addresses against the allowlist before approvals and use the custom error on failure.

Prefer deriving the real `token0/token1` from `positions(tokenId)` and using those for approvals to avoid mismatches between user input and the position's tokens. Apply the same validation path for `enterTransient` via the shared `enter` function.

[Go back to Findings Summary](#)

I2: Dead code: custom error `UnsupportedMethod` is declared but never used

Impact:	Info	Confidence:	High
Target:	contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamCollectFuse.sol	Type:	Unused code

Description

The custom error `UnsupportedMethod()` is declared but never used by any function in this contract. This adds bytecode size and noise without improving safety or functionality. If the intention was to guard unsupported entrypoints, the error should be emitted from a `fallback/receive` function or from specific functions; otherwise, the declaration should be removed.

The contract exposes `enter` and `enterTransient` as its callable methods and does not contain any guard that reverts with `UnsupportedMethod`.

Listing 74. Code snippet from `contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamCollectFuse.sol` (unused error)

```
39 /// @notice Thrown when an unsupported method is called
40 error UnsupportedMethod();
```

Exploit Scenario

There is no direct security exploit. The practical risk is confusion and missed guardrails.

Consider the following operational scenario:

1. Alice expects unsupported entrypoints to revert with a well-defined custom error for monitoring and debugging;

2. Alice calls an unintended function selector and observes a generic revert (or no revert path is present at all for that selector);
3. Tooling and alerting that rely on the custom error signature do not trigger;
4. Bob misclassifies the failure, making incident triage slower and less reliable.

Recommendation

Align the code with the intended behavior.

Fix the issue by either:

- Removing the unused `UnsupportedMethod` error if no such guard is required; or
- Emitting the error from an explicit guard where appropriate, for example in `fallback` and `receive` functions, or at the start of functions that should not be callable in certain modes.

Add a linter or unit test that fails when custom errors are declared but not referenced, to prevent future drift.

[Go back to Findings Summary](#)

I3: Dead private helper `bytes32ToSubstrate` is unused, increasing bytecode size and maintenance risk

Impact:	Info	Confidence:	High
Target:	contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol	Type:	Unused code

Description

The private pure helper `bytes32ToSubstrate(bytes32[])` is never invoked by this contract. It duplicates functionality that is already consumed directly from `VelodromeSuperchainSubstrateLib.bytes32ToSubstrate` in `balanceOf`. Keeping dead code increases bytecode size, audit surface, and the risk of accidental misuse in future refactors (for example, calling the helper with unvalidated inputs). Although this is an informational issue with no direct exploit under the current implementation, removing unused code is a standard hardening step.

The only place that decodes substrates within this contract calls the library directly:

Listing 75. Code snippet from `contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol`

```
68         substrate = VelodromeSuperchainSubstrateLib.bytes32ToSubstrate(  
69             pools[i]  
70         );
```

The private helper below is never referenced:

Listing 76. Code snippet from

contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol

```
201     function bytes32ToSubstrate(  
202         bytes32[] memory bytes32Substrates_  
203     ) private pure returns (VelodromeSuperchainSubstrate[] memory) {  
204         VelodromeSuperchainSubstrate[]  
205             memory substrates = new VelodromeSuperchainSubstrate[](  
206                 bytes32Substrates_.length  
207             );  
208         for (uint256 i; i < bytes32Substrates_.length; ++i) {  
209             substrates[i] =  
210                 VelodromeSuperchainSubstrateLib.bytes32ToSubstrate(  
211                     bytes32Substrates_[i]  
212                 );  
213         }  
214         return substrates;  
215     }
```

Affected elements:

- Function `bytes32ToSubstrate` at lines 201–214 (private, pure, unused).
- No state variables are affected.

Evidence collected:

- Project search shows no call sites for the private helper; only its declaration and its internal call to the library appear.
- The active decoding path in `balanceOf` uses the library-qualified call, confirming the helper is redundant in this contract.

Exploit Scenario

There is no direct exploitation path in the current implementation because the helper is `private` and unreachable via the ABI. The practical risk is operational and forward-looking: dead code increases the chance of future mistakes and reduces test coverage relevance.

Consider the following scenario:

1. Alice studies the contract and notices the unused helper `bytes32ToSubstrate`.
2. Alice monitors releases; in a later refactor, Bob reuses the helper to "simplify" code paths but forgets to replicate validations around substrate decoding.
3. Alice crafts a `bytes32[]` payload that decodes into unexpected substrate types, causing logic to misinterpret entries.
4. Bob's protocol experiences misreported balances or avoidable reverts due to the unvalidated use, which could have been avoided had the dead helper been removed or properly reviewed.

Recommendation

Remove the unused helper to reduce bytecode size and eliminate maintenance risk. If a conversion helper is desired for readability, keep it in the dedicated library and always invoke it with a fully qualified library call.

Fix the vulnerability by either:

- Deleting function `bytes32ToSubstrate(bytes32[])` from this contract; or
- Moving the conversion helper to `VelodromeSuperchainSubstrateLib` (if not already present) and calling it explicitly as `VelodromeSuperchainSubstrateLib.bytes32ToSubstrate` where needed.

Additional hardening:

- Enable and monitor static analysis rules that flag unused functions.
- Add regression tests around substrate decoding and type checks to ensure future refactors cannot bypass validations.

[Go back to Findings Summary](#)

14: Dead code: IERC20/SafeERC20 imported and using directive declared but never used

Impact:	Info	Confidence:	High
Target:	contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamCollectFuse.sol	Type:	Unused code

Description

The contract imports `IERC20` and `SafeERC20` and declares `using SafeERC20 for IERC20;`, but no function in this contract calls any `IERC20` or `SafeERC20` method. This is dead code that increases bytecode size and audit surface without providing functionality. It may also mislead reviewers into assuming `SafeERC20` protections are applied to token interactions, which this module does not perform.

The externally exposed functions `enter` and `enterTransient` interact only with `INonfungiblePositionManager` and transient storage. They do not transfer or approve ERC20 tokens.

Listing 77. Code snippet from contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamCollectFuse.sol (imports)

```
4 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
5 import {SafeERC20} from
  "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
```

Listing 78. Code snippet from contracts/fuses/aerodrome_slipstream/AreodromeSlipstreamCollectFuse.sol (using directive)

```
25 using SafeERC20 for IERC20;
```

No subsequent lines in this file call `safeTransfer`, `safeTransferFrom`, or `safeApprove`. This confirms the `using` directive and the imports are unused in this contract.

Exploit Scenario

This issue does not present a direct on-chain exploit. However, it can cause operational risk and reviewer confusion.

Consider the following operational scenario:

1. Alice, a developer, sees the `using SafeERC20 for IERC20;` directive and assumes SafeERC20 protections are in effect for token transfers in this module;
2. Alice or a teammate later introduces a raw `IERC20.transfer` call in a related code path, assuming SafeERC20 semantics will be applied automatically because of the existing directive;
3. The call interacts with a non-standard ERC20 that returns `false` instead of reverting, and the transfer silently fails;
4. Bob's system proceeds under the assumption that tokens were moved, causing accounting inconsistencies and operational loss.

Recommendation

Remove the dead code or apply the library consistently.

Fix the issue by either:

- Removing the unused imports and the `using` directive if this module will not perform ERC20 operations (delete lines 4-5 and 25 in the referenced file); or
- If ERC20 interactions are intended, replace any raw token calls with `SafeERC20` wrappers (`safeTransfer`, `safeTransferFrom`, `safeApprove`) and add

tests that cover non-standard ERC20 behavior.

Additionally, enable a linter rule in CI that flags unused imports and unused `using` directives to prevent regressions.

[Go back to Findings Summary](#)

I5: Unused immutable MARKET_ID may mislead integrators about market scoping

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/async_action/AsyncActionBalanceFuse.sol	Type:	Unused code

Description

The contract declares a public immutable `MARKET_ID` but never reads it in any function. This leaves dead state and exposes a public getter that may suggest market scoping guarantees that are not enforced by `AsyncActionBalanceFuse`.

Observed behavior:

- `MARKET_ID` is declared at line 25 and written once in the `constructor` at line 30.
- There are no reads of `MARKET_ID` anywhere in this contract; `balanceOf` does not use it for gating or valuation.
- Because the variable is `public`, the compiler auto-generates a getter `MARKET_ID()` which increases the external surface without adding protections.

This can be misleading during integration and monitoring. Off-chain indexers or operators may incorrectly assume that the returned balance is scoped by a specific market, while in reality the function `balanceOf` computes USD value purely from the configured `AsyncExecutor` and price oracle, independent of `MARKET_ID`.

Affected elements:

- State variable `MARKET_ID` at line 25.
- Function `constructor(uint256 marketId_)` at lines 29–31 is the only writer.
- Function `balanceOf` at lines 40–77 does not reference `MARKET_ID`.

Static evidence:

- Repository search shows only the declaration and the assignment within this file. No additional references exist.

Listing 79. Code snippet from

contracts/fuses/async_action/AsyncActionBalanceFuse.sol

```

24    /// @notice Identifier of the market this balance fuse is associated with
25    uint256 public immutable MARKET_ID;
26
27    /// @notice Initializes the balance fuse with the market identifier
28    /// @param marketId_ Unique identifier of the market handled by this fuse
29    constructor(uint256 marketId_) {
30        MARKET_ID = marketId_;
31    }

```

Exploit Scenario

There is no direct on-chain exploit of funds stemming from this pattern in isolation. However, misleading state can create operational and risk-management inconsistencies in downstream systems that believe balances are market-scoped.

Consider the following attack scenario:

1. Alice reviews the contract and learns that `MARKET_ID` is not used by `balanceOf`.
2. Alice participates in Bob's system where off-chain accounting attributes this fuse's `balanceOf` to a market identified by `MARKET_ID`.
3. Alice triggers actions that affect the `AsyncExecutor` associated with a

different market or with shared resources.

4. The fuse still returns a USD value independent of `MARKET_ID`, and Bob's systems misattribute the balance to the wrong market, potentially bypassing market caps, quota checks, or skewing incentives and risk metrics.

Recommendation

Prefer eliminating misleading, unused state or enforcing the intended invariant.

Fix the issue by either:

- Removing `MARKET_ID` and the constructor parameter from this contract if market scoping is not used here, reducing bytecode size, deployment complexity, and confusion.
- If market scoping is required, enforce it: persist a mapping of `marketId` -> `executor/asset` in a trusted component and validate inside `balanceOf` that the resolved executor and/or asset belongs to `MARKET_ID`, reverting on mismatch.
- If the field is intentionally retained for interface homogeneity, document it clearly in NatSpec and rename to `marketId` with `private` or `internal` visibility, exposing an explicit getter only where it is semantically used.
- Add unit tests asserting either that `MARKET_ID` is read in logic, or that it is intentionally absent and cannot mislead integrators.

[Go back to Findings Summary](#)

I6: Dead private helper function substratesToBytes32 increases code surface without benefit

Impact:	Info	Confidence:	High
Target:	contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol	Type:	Unused code

Description

The private helper `substratesToBytes32` is never invoked. Unused code increases audit surface, can hide future mistakes if copied into new contexts, and adds maintenance overhead.

Listing 80. Code snippet from contracts/fuses/velodrome_superchain/VelodromeSuperchainBalanceFuse.sol

```
190 function substratesToBytes32(  
191     VelodromeSuperchainSubstrate[] memory substrates_  
192 ) private pure returns (bytes32[] memory) {  
193     bytes32[] memory bytes32Substrates = new bytes32[](substrates_.length);  
194     for (uint256 i; i < substrates_.length; ++i) {  
195         bytes32Substrates[i] = VelodromeSuperchainSubstrateLib  
196             .substrateToBytes32(substrates_[i]);  
197     }  
198     return bytes32Substrates;  
199 }
```

There is no internal reference to this function within the contract, and it is not externally callable (`private`).

Exploit Scenario

Consider the following attack scenario:

1. Alice copies this contract to extend functionality and assumes `substratesToBytes32` is used and tested.
2. She wires new code to depend on it without adding tests, missing edge cases in `substrate` encoding.
3. The latent, unused helper introduces subtle mis-encodings that only surface in production.
4. Bob's integration suffers from silent misconfiguration and difficult-to-trace bugs.

Recommendation

Remove dead code or integrate it properly:

- If needed, make the helper `internal` and add unit tests that cover round-trip conversions with `VelodromeSuperchainSubstrateLib`.
- Otherwise, delete the function and call `VelodromeSuperchainSubstrateLib.substrateToBytes32` directly where required.
- Enable static analysis rules (e.g., solhint `no-unused-functions`) in CI to prevent reintroduction.

[Go back to Findings Summary](#)

17: Unused SafeCast import and using directive increases bytecode and can mislead reviewers

Impact:	Info	Confidence:	High
Target:	contracts/fuses/fluid_instadapp/FluidInstadappStakingBalanceFuse.sol	Type:	Unused code

Description

The contract imports `SafeCast` and declares `using SafeCast for uint256`; but never invokes any `SafeCast` methods (for example, no calls to `toUint*` helpers). This is dead code that slightly increases deployed bytecode size and, more importantly, suggests that explicit, safe downcasting is performed when it is not. Such misleading signaling can cause reviewers or future contributors to assume safety properties that the code does not actually provide.

Affected scope:

- The import at line 4 and the `using` directive at line 22 are file-scoped and therefore affect the entire contract.
- The contract exposes only two functions (`constructor` and `balanceOf`), and neither performs any `SafeCast` operation.

This is an informational issue with minimal runtime impact today, but it increases maintenance risk. If future changes introduce downcasts or rely on revert-on-overflow semantics, the presence of the `using` directive may lead to the mistaken belief that `SafeCast` protections are in place when they are not.

Listing 81. Code snippet from contracts/fuses/fluid_instadapp/FluidInstadappStakingBalanceFuse.sol

```
4 import {SafeCast} from "@openzeppelin/contracts/utils/math/SafeCast.sol";
```

```

5 import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
6 import {IERC20Metadata} from
  "@openzeppelin/contracts/interfaces/IERC20Metadata.sol";
7 import {IMarketBalanceFuse} from "../IMarketBalanceFuse.sol";
8 import {IPriceOracleMiddleware} from
  "../price_oracle/IPriceOracleMiddleware.sol";
9 import {IporMath} from "../libraries/math/IporMath.sol";
10 import {IFluidLendingStakingRewards} from
  "../ext/IFluidLendingStakingRewards.sol";
11 import {PlasmaVaultConfigLib} from
  "../libraries/PlasmaVaultConfigLib.sol";
12 import {PlasmaVaultLib} from "../libraries/PlasmaVaultLib.sol";
13
14 /**
15
16  * @title Fuse Fluid Instadapp Staking protocol responsible for calculating
    the balance of the Plasma Vault in the Fluid Instadapp Staking protocol based
    on preconfigured market substrates
17  * @notice Calculates the balance of staked assets in Fluid Instadapp Staking
    protocol, converted to underlying asset value in USD
18  * @dev Substrates in this fuse are the staking pool addresses that are used
    in the Fluid Instadapp Staking protocol for a given MARKET_ID.
19  *      This fuse retrieves the staking balance from the first configured
    substrate, converts staking tokens to underlying assets,
20  *      and converts to USD using price oracle middleware. The result is
    normalized to WAD (18 decimals).
21  */
22
23 contract FluidInstadappStakingBalanceFuse is IMarketBalanceFuse {
24     using SafeCast for uint256;

```

Exploit Scenario

There is no direct external exploit vector; the issue is informational. However, misleading unused safety tooling can translate into correctness bugs during future changes or integrations.

Consider the following attack scenario:

1. Alice, a developer, reviews the contract and assumes `SafeCast` is used to enforce safe downcasts in `balanceOf`.
2. Alice later contributes a change that downcasts an intermediate value (for

example, to `uint128`) expecting a revert on overflow because `SafeCast` appears to be enabled.

3. The contract performs a raw Solidity cast instead, does not revert, and silently truncates the value.
4. Bob's protocol deploys the change and suffers under-reported balances and incorrect risk limits due to the silent truncation.

Recommendation

Remove the dead code or apply `SafeCast` consistently.

Recommended steps:

- If no explicit downcasting is required in this contract, delete the `SafeCast` import and the `using SafeCast for uint256;` directive.
- If safe downcasting was intended, replace any raw casts with the corresponding `SafeCast` helpers (for example, `x.toUint128()`), and keep the `using` directive only where those helpers are actually used.
- Keep the codebase consistent: avoid file-scoped `using` statements for libraries that are not used in the file.

[Go back to Findings Summary](#)

l8: instantWithdraw parameter docs mismatch: params[1] must be gauge address (not ERC4626 vault)

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/curve_gauge/CurveChildLiquidityGaugeSupplyFuse.sol	Type:	Code quality

Description

The header comment of function `instantWithdraw` documents `params[1]` as a "vault address", while the implementation interprets `params_[1]` as the Curve child liquidity `gauge` address and then queries the gauge for `lp_token()`. This documentation-to-code mismatch is a footgun for integrators: supplying an ERC4626 vault address (as documented) will cause the function to treat that value as a gauge and invoke `lp_token()` on an incompatible contract.

If the provided address does not implement `lp_token()`, the call reverts and the exit path is blocked. If it does implement `lp_token()` but is not the intended gauge, the function will proceed with a wrong `lpToken` and compute `lpTokenToWithdraw` against an unrelated ERC4626, leading to reverts or inconsistent behavior at `_exit`.

Affected function: `instantWithdraw`. Affected parameter: `params_[1]` (interpreted as `gauge`).

Listing 82. Code snippet from contracts/fuses/curve_gauge/CurveChildLiquidityGaugeSupplyFuse.sol

```
127     /// @dev Could be used only if lpToken is ERC4626.
128     /// @dev params[0] - amount in underlying asset, params[1] - vault
    address
129     function instantWithdraw(bytes32[] calldata params_) external {
130         uint256 amount = uint256(params_[0]);
```

```

131
132     address gauge = PlasmaVaultConfigLib.bytes32ToAddress(params_[1]);
133     address lpToken = IChildLiquidityGauge(gauge).lp_token();
134     uint256 lpTokenToWithdraw =
        ERC4626Upgradeable(lpToken).convertToShares(
135         amount
136     );

```

The comment claims `params[1]` is a vault address, but the code converts it to `gauge` and dereferences `IChildLiquidityGauge(gauge).lp_token()`. Passing a vault address here would cause a call to `lp_token()` on the vault, which is incorrect.

Exploit Scenario

A practical failure scenario arises when integrators wire this fuse following the header comment.

Consider the following attack scenario:

1. Alice configures an automation that calls `instantWithdraw` and, following the comment, sets `params[1]` to the ERC4626 vault address instead of the gauge address;
2. Alice triggers `instantWithdraw` with `params[0]` set to the desired underlying amount and `params[1]` set to the vault address;
3. The contract converts `params_[1]` to `gauge` and calls `IChildLiquidityGauge(gauge).lp_token()` on the vault address;
4. The external call reverts because the vault does not implement `lp_token()`, so the withdrawal path is blocked and the position remains staked.

A subtler variant is possible if `params[1]` points to a contract that implements a function with the `lp_token()` selector but returns an unrelated address. In that case:

1. The call to `lp_token()` succeeds, returning an arbitrary `lpToken` address;

2. The contract calls `ERC4626Upgradeable(lpToken).convertToShares(amount)`, which may revert or yield a meaningless share amount; and
3. The call into `_exit` attempts to withdraw that many shares from the intended gauge, likely reverting. This results in a persistent operational failure until the parameter is corrected.

Impact in production: Denial of service of the fast exit path and potential unintended external calls to arbitrary contracts if misconfigured. Severity is informational because the issue stems from documentation mismatch rather than a direct exploitable state change.

Recommendation

Align the documentation and code to remove ambiguity and prevent misconfiguration.

Fix the vulnerability by either:

- Updating the NatSpec comment to state that `params[1]` must be the Curve child liquidity gauge address; or
- If the intent was to pass an ERC4626 vault address, refactor the implementation to accept the vault and deterministically derive the gauge, or take the vault address explicitly and validate it.

Additionally, harden the interface against misuse:

- Rename variables to `gauge` in the comment and add `require` checks that the provided address is an approved substrate (`PlasmaVaultConfigLib.isSubstrateAsAssetGranted`) before dereferencing;
- Optionally, replace the untyped `bytes32[] params_` with a typed struct or explicitly named parameters to eliminate parameter-ordering errors; and
- Add unit tests that assert `instantWithdraw` reverts with a clear error message when `params[1]` is not a valid gauge.

[Go back to Findings Summary](#)

I9: enterTransient documentation claims outputs are written, but implementation omits setOutputs (transient I/O mismatch)

Impact:	Info	Confidence:	High
Target:	contracts/fuses/burn_request_fee/BurnRequestFeeFuse.sol	Type:	Code quality

Description

The docstring above `enterTransient` claims: "Reads inputs from transient storage, calls `enter()`, and writes outputs to transient storage". The implementation only reads inputs and calls `enter(data)` — it never writes any outputs back to transient storage.

This documentation/implementation drift can mislead integrators who compose multiple fuses within a single execution flow and rely on outputs being present in transient storage for subsequent steps. Such mismatches commonly cause orchestration failures, unexpected reverts, or silent misuse of default values by downstream steps.

Listing 83. Code snippet from contracts/fuses/burn_request_fee/BurnRequestFeeFuse.sol

```
125 /// @notice Burns request fee shares using transient storage for input
    parameters
126 /// @dev Reads inputs from transient storage, calls enter(), and writes
    outputs to transient storage
127 function enterTransient() external {
128     bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
129
130     BurnRequestFeeDataEnter memory data = BurnRequestFeeDataEnter({
131         amount: TypeConversionLib.toUint256(inputs[0])
132     });
133
134     enter(data);
```

Exploit Scenario

A misleading contract interface can break multi-step executions that depend on transient outputs.

Consider the following attack scenario:

1. Alice composes a sequence of fuse calls and, based on the documentation, expects `enterTransient` to publish outputs to transient storage for the next step;
2. Alice executes the sequence; and
3. The next step reads from transient storage, but the expected outputs are missing, causing it to operate on zero/default values or revert unexpectedly, disrupting Bob's workflow and creating a denial-of-service for the composed action.

Recommendation

Align the documentation and behavior of `enterTransient` to avoid integration mistakes.

Fix the issue by either:

- Implementing the missing output write, e.g., calling `TransientStorageLib.setOutputs(VERSION, outputs)` with a clearly specified encoding; or
- Updating the docstring to state that no outputs are produced, and ensuring downstream consumers are not expected to read outputs for this fuse.

Additional robustness:

- Add tests that validate the presence/absence of transient outputs after calling `enterTransient`;
- Prefer strongly-typed structs for transient outputs to make usage explicit and less error-prone.

[Go back to Findings Summary](#)

I10: Inaccurate layout description in decodeAllowedTargets (address not left-aligned; selector in least-significant 4 bytes)

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/async_action/AsyncActionFuseLib.sol	Type:	Code quality

Description

The `decodeAllowedTargets` documentation states that it "Decodes address (20 bytes, left-aligned) and bytes4 selector (4 bytes, right-aligned) from 31 bytes; Layout: [address (20 bytes) | selector (4 bytes) | unused (7 bytes)]".

The implementation, however, interprets the `bytes31` as a `uint248` where the `selector` is in the least-significant 4 bytes and the `address` is obtained by shifting right by 32 bits. This means the 7 unused bytes are the most-significant portion of the `bytes31` word, not trailing after the selector.

Effective layout (most-significant to least-significant):

- Bits [192..247]: 7 unused bytes (zero);
- Bits [32..191]: 20-byte `address` (`data_.target`); and
- Bits [0..31]: 4-byte `selector`.

This discrepancy can cause off-chain decoders that follow the comments to parse a different `target` than the one actually decoded on-chain, undermining allow-list checks, previews, or signing flows that rely on off-chain validation.

Affected surface:

- Function `decodeAllowedTargets` in `contracts/fuses/async_action/AsyncActionFuseLib.sol`.

- Type `AllowedTargets` as represented in `bytes31`.

Listing 84. Code snippet from

contracts/fuses/async_action/AsyncActionFuseLib.sol

```

101 /// @dev Decodes address (20 bytes, left-aligned) and bytes4 selector (4
    bytes, right-aligned) from 31 bytes.
102 ///      Layout: [address (20 bytes) | selector (4 bytes) | unused (7
    bytes)]
103 function decodeAllowedTargets(
104     bytes31 encoded_
105 ) internal pure returns (AllowedTargets memory data_) {
106     uint248 packed = uint248(encoded_);
107     // Extract address from upper 160 bits (shifted right by 32 bits)
108     data_.target = address(uint160(packed >> 32));
109     // Extract selector from lower 32 bits
110     data_.selector = bytes4(uint32(packed & _UINT32_MASK));
111 }

```

Exploit Scenario

A misdocumented layout can be exploited whenever external tooling encodes/decodes `AllowedTargets` based on the comments instead of the actual bit operations.

Consider the following attack scenario:

1. Alice prepares a `bytes31` payload that, when decoded according to the comments, maps to `target = T1` and `selector = S`.
2. Alice presents this payload to Bob's change-management service that validates allow-lists off-chain using the comment-based decoder and green-lights the update for T1.S.
3. Bob signs and sends the transaction including the `bytes31` payload.
4. On-chain, the library decodes with `target = T2` and `selector = S` because the 7 unused bytes are the most-significant, not trailing as assumed by the service.

5. Bob's system executes or authorizes an external call to T2.S, which was never reviewed by the approvers and may transfer funds or grant approvals to an unintended contract.

In practice, such confusion emerges in dashboards, scripts, or SDKs that replicate codec logic from NatSpec comments. The immediate impact is policy drift and the potential for unauthorized external interactions.

Recommendation

Align the documentation with the implementation and remove ambiguity:

- State that the 7 unused bytes are the most-significant bytes of the `bytes31` word (bits [192..247]).
- Document that `target` is decoded as `address(uint160(uint248(encoded_) >> 32))` and `selector` as `bytes4(uint32(uint248(encoded_)))`.
- Provide a worked example in the docs/tests to demonstrate round-trip correctness and the exact byte layout.

Actionable steps:

- Update the NatSpec for `decodeAllowedTargets` to document the precise bit positions.
- Add unit tests that decode the output of `encodeAllowedTargets` and verify that the top 7 bytes in `encoded` are zero.
- Provide an integration test for any off-chain codecs used in tooling to ensure they match the on-chain semantics.

[Go back to Findings Summary](#)

I11: Unused immutable **BALANCER_ROUTER** (dead code) increases maintenance and audit risk

Impact:	Info	Confidence:	High
Target:	contracts/fuses/balancer/ BalancerBalanceFuse.sol	Type:	Unused code

Description

The constructor validates and stores a Balancer router address in the immutable **BALANCER_ROUTER**, but this variable is never read anywhere in the contract. This is dead code that provides no functional effect and can mislead maintainers or integrators into thinking router-specific behavior exists.

Affected elements:

- Variable **BALANCER_ROUTER** (immutable)
- Function **<constructor>** (parameter **router_** and zero-address check)

*Listing 85. Code snippet from
contracts/fuses/balancer/BalancerBalanceFuse.sol*

```
61    /// @notice The Balancer router address
62    /// @dev This address is used for Balancer protocol interactions
63    address public immutable BALANCER_ROUTER;
64
65    /// @notice Constructor to initialize the fuse with a market ID and
66    Balancer router address
67    /// @param marketId_ The unique identifier for the market configuration
68    /// @param router_ The address of the Balancer router contract
69    /// @dev The market ID is used to retrieve the list of substrates
70    (pools/gauges) that this fuse will track.
71    /// VERSION is set to the address of this contract instance for
72    tracking purposes.
73    constructor(uint256 marketId_, address router_) {
74        if (router_ == address(0)) {
75            revert BalancerBalanceFuseInvalidRouterAddress();
76        }
77        BALANCER_ROUTER = router_;
78    }
```

```
73     }
74
75     VERSION = address(this);
76     MARKET_ID = marketId_;
77     BALANCER_ROUTER = router_;
```

Evidence: repository-wide search shows this identifier appears only at declaration and assignment in this file; no reads. Wake MCP confirms the contract has only three functions (<constructor>, `balanceOf`, and `_convertToUsd`), none of which reference `BALANCER_ROUTER` after construction. The parameter therefore has no run-time effect.

Exploit Scenario

While not a direct vulnerability, this layout can lead to unsafe assumptions and configuration mistakes when the fuse is integrated elsewhere.

Consider the following attack scenario:

1. Alice deploys or configures the fuse and supplies a specific `router_`, believing it constrains behavior;
2. Alice communicates this assumption to downstream systems or operations that rely on this fuse's output;
3. The variable is never used, so the actual behavior is unaffected by `router_`; and
4. Bob's protocol relies on the mistaken assumption (for example, believing swaps or valuations are bound to a given router) and makes operational decisions that degrade risk controls or monitoring accuracy.

Recommendation

Make the intent explicit in code:

- If the router address is not needed, remove the constructor parameter and the immutable `BALANCER_ROUTER` entirely.

- If the router is intended to be used, thread it through the logic that requires it (for example, to validate pools/gauges against a router or to perform router-bound calls), and add tests that fail if the variable becomes unused.
- Add a linter or CI check for unused immutables/parameters to prevent dead code from persisting.

[Go back to Findings Summary](#)

I12: NatSpec access annotation mismatch: `updateMarketsBalances` is restricted but documented as public/no restrictions

Impact:	Info	Confidence:	High
Target:	./contracts/vaults/PlasmaVault.sol	Type:	Code quality

Description

The documentation for function `updateMarketsBalances` incorrectly states access as "Public function, no role restrictions" while the implementation applies the `restricted` modifier (from `AccessManagedUpgradeable`), which enforces role-based authorization via the `authority` manager.

This mismatch can mislead integrators, operators, and auditors about who is allowed to call `updateMarketsBalances`. In production, relying on the NatSpec may lead to unexpected authorization failures, stalled operational flows (for example, off-chain agents attempting to refresh market balances without the required role), and inconsistent runbooks or monitoring that assume permissionless access.

`restricted` gates calls through the Access Manager policy and will revert for unprivileged callers. Therefore, the function is not publicly callable despite the current NatSpec.

Listing 86. Code snippet from `contracts/vaults/PlasmaVault.sol`

```
397     /// @param marketIds_ Array of market IDs to update
398     /// @return uint256 Updated total assets after balance refresh
399     /// @custom:access Public function, no role restrictions
400     function updateMarketsBalances(
401         uint256[] calldata marketIds_
402     ) external restricted returns (uint256) {
403         if (marketIds_.length == 0) {
404             return totalAssets();
405         }
406     }
```

Exploit Scenario

An incorrect access annotation can cause operational failures and be abused as part of an availability disruption or misconfiguration.

Consider the following attack scenario:

1. Alice, an external user, reads the documentation and assumes `updateMarketsBalances` is permissionless;
2. Alice (or tooling she operates) repeatedly calls `updateMarketsBalances` from an unprivileged address to force market balance refreshes;
3. The call reverts due to the `restricted` modifier, wasting gas and preventing the intended maintenance action; and
4. Bob's operations team, relying on the same documentation, deploys a health bot without the necessary role, leading to stale balance data and delayed fee realization until a privileged operator intervenes.

While this is not an asset-stealing vector, the confusion can degrade liveness, disrupt monitoring workflows, and increase operational risk in volatile markets where timely balance updates are expected.

Recommendation

Align documentation and access control to eliminate ambiguity:

- If the function must remain role-gated: update the NatSpec to explicitly document `restricted` access, the governing `authority` contract, and the roles allowed to call `updateMarketsBalances`.
- If the function is intended to be publicly callable: remove the `restricted` modifier or provide a dedicated role that maps to public access (for example, granting `PUBLIC_ROLE` via the Access Manager) and update the

NatSpec accordingly.

- Add tests or linting that check consistency between `@custom:access` annotations and actual modifiers. For example, enforce that any function documented as public/no restrictions does not include `restricted`.
- Communicate the access semantics in integration docs and runbooks, and ensure off-chain agents (keepers/bots) are provisioned with the correct role in production.

[Go back to Findings Summary](#)

113: Exit data struct mislabels unit/direction (says "to deposit" instead of "to withdraw")

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol	Type:	Code quality

Description

The field comment in `FluidInstadappStakingSupplyFuseExitData` incorrectly states "fluidTokenAmount to deposit" even though the struct is used for exiting/withdrawing. This mislabeling can confuse integrators about the expected unit and direction, increasing the risk of misuse and incorrect parameterization.

Affected elements:

- Struct `FluidInstadappStakingSupplyFuseExitData` (comment mislabels the purpose of `fluidTokenAmount`);
- Functions `exit`, `exitTransient`, and `instantWithdraw` that consume this struct.

The inconsistency is shown below.

Listing 87. Code snippet from contracts/fuses/fluid_instadapp/FluidInstadappStakingSupplyFuse.sol

```
24 /// @notice Data structure for exiting - withdrawing - the Fluid Instadapp
    Staking protocol
25 struct FluidInstadappStakingSupplyFuseExitData {
26     /// @dev fluidTokenAmount to deposit, in fluidTokenAmount decimals
27     uint256 fluidTokenAmount;
28     /// @dev stakingPool address where fluidToken is staked and farmed token
    ARB
29     address stakingPool;
```

Impact in production:

- Engineers may pass values expressed for deposits rather than withdrawals when populating the struct, leading to incorrect behavior and wasted gas;
- Documentation drift makes maintenance harder and can mask unit mismatches during reviews.

Exploit Scenario

Incorrect documentation can lead to integration mistakes with observable consequences.

Consider the following attack scenario:

1. Alice, an integrator, reads the struct comment and assumes `fluidTokenAmount` is a deposit-sized value in asset units;
2. Alice constructs the exit data using a deposit-oriented assumption and calls a withdraw function;
3. The contract interprets `fluidTokenAmount` as the number of staking token shares to withdraw, not a deposit-sized asset amount, causing an unexpected operation size or no-op;
4. Bob's protocol experiences unexpected behavior and incurs unnecessary gas costs due to the misinterpretation.

Recommendation

Correct the misleading comment and clarify units and direction across the codebase.

Fix the issue by either:

- Updating the comment to "fluidTokenAmount to withdraw, in staking

token share units”; or

- Renaming the field to `sharesToWithdraw` and adjusting documentation accordingly for unambiguous semantics.

Additionally:

- Ensure external-facing documentation and interface comments consistently describe units and rounding expectations to prevent integration errors.

[Go back to Findings Summary](#)

I14: Inaccurate layout description in encodeAllowedTargets (address not left-aligned; unused 7 bytes are most-significant)

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/async_action/AsyncActionFuseLib.sol	Type:	Code quality

Description

The `encodeAllowedTargets` NatSpec claims that it "Encodes address (20 bytes, left-aligned) and bytes4 selector (4 bytes, right-aligned) into 31 bytes; Layout: [address (20 bytes) | selector (4 bytes) | unused (7 bytes)]".

The actual packing logic does not left-align the `address` within `bytes31` and does not place the 7 unused bytes after the `selector`. Instead, the implementation composes a `uint248` as `address << 32 | selector`, which yields the following bit layout inside the 31-byte word (most-significant to least-significant):

- Bits [192..247]: 7 unused bytes, always zero;
- Bits [32..191]: 20-byte `address` (`data_.target`); and
- Bits [0..31]: 4-byte `selector`.

This mismatch between documentation and implementation risks misguiding integrators who implement off-chain encoders/decoders from comments rather than from the code, leading to mispacked values and incorrect authorization decisions when these bytes are used as allow-lists.

Affected surface:

- Function `encodeAllowedTargets` in

contracts/fuses/async_action/AsyncActionFuseLib.sol.

- Type `AllowedTargets` packed into `bytes31`.

Listing 88. Code snippet from

contracts/fuses/async_action/AsyncActionFuseLib.sol

```
89 /// @dev Encodes address (20 bytes, left-aligned) and bytes4 selector (4
    bytes, right-aligned) into 31 bytes.
90 ///     Layout: [address (20 bytes) | selector (4 bytes) | unused (7 bytes)]
91 ///     The remaining 7 bytes are unused but preserved for consistency with
    bytes31 format.
92 function encodeAllowedTargets(
93     AllowedTargets memory data_
94 ) internal pure returns (bytes31 encoded) {
95     // Pack: address shifted left by 32 bits, selector in lower 32 bits
96     uint248 packed = (uint248(uint160(data_.target)) << 32) |
97         uint248(uint32(data_.selector));
98     encoded = bytes31(packed);
99 }
```

Exploit Scenario

A mismatch between the documented layout and the implemented packing can be abused whenever an external system relies on the documentation to encode or validate `AllowedTargets` off-chain.

Consider the following attack scenario:

1. Alice crafts a `bytes31` value for `AllowedTargets` using a third-party encoder that follows the NatSpec comment (treating the address as left-aligned and the 7 unused bytes as trailing).
2. Alice submits this value to Bob's off-chain policy validator which also follows the comment to decode it, concluding that the `target` equals a benign address `T1` and the `selector` equals `S`.
3. Bob, believing the payload is authorized for `T1.S`, signs and relays a transaction that includes the mispacked substrate.

4. The on-chain library decodes using the actual implementation, resolving `target` to a different address T2 (because the 7 unused bytes are most-significant, not trailing) while `selector` stays at the low 4 bytes.
5. Bob's system processes the call against T2.S, enabling an external call path that was not reviewed, which can allow value transfers or approvals to unintended contracts.

In production, this manifests when wallets, backends, or monitoring services build codecs from comments instead of the canonical on-chain implementation. The result is silent policy drift and potentially unauthorized external-call allowance.

Recommendation

Correct the specification so integrators implement the exact bit layout:

- State explicitly that the 7 unused bytes are the most-significant bytes of the `bytes31` word (bits [192..247]).
- Document that the `address` occupies bits [32..191] and the `selector` occupies bits [0..31].
- Provide an explicit reference formula and example bytes:
- Encode: `packed = (uint248(uint160(target)) << 32) | uint248(uint32(selector));`
- Decode: `target = address(uint160(packed >> 32)); selector = bytes4(uint32(packed));`

Actionable steps:

- Update the NatSpec for `encodeAllowedTargets` to describe the actual layout and remove terms like "left-aligned/right-aligned" which are ambiguous across endianness contexts.
- Add a unit test that round-trips random `AllowedTargets` and asserts that

the upper 7 bytes are zero after encoding.

- Consider exposing a small helper that returns (**target**, **selector**) bit positions in comments or docs to eliminate ambiguity for integrators.

[Go back to Findings Summary](#)

Appendix A: How to cite

Please cite this document as:

[Ackee Blockchain Security](#), Wake Arena Report | IPOR-Labs: ipor-fusion, Wake
Arena AI Report.



Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz

